Set Bit

Problem: Write a program to check if a bit is set or not.

Sample Test Case:
Input:
1
4 1

Output:
NO

Explanation:

first line indicates number of test case second line read N and Kth bit set or not.

Basic Approach

```
----------------
read t
loop t times:
    read num, k
    set kbin = 1
    loop (i = 1 to k - 1):
        kbin=kbin*2

    if (num AND kbin)!=0:
        print "Yes"
    else:
        print "No"
```

Better Approach

```
----------------
read t
loop t times:
    read num, k
    num= num >> (k-1)
if (num AND 1)==1:
        print "Yes"
    else:
```

```
        print "No"
```

Best Approach

```
------------------
  if((n AND (1 << (k -1)))!=0):
    Print "Yes"
    else
    Print "No"
```

Count number of Set Bits

Problem:count the number of set bits (i.e., bits with value 1) in the binary representation of a given integer.

Input:
5
Output:
2
Explanation:
5 in binary → 0101 → 2 set bits

```
read n
set count = 0

while n > 0:
    if n %2==1:
        do count++
    n=n/2

print count
```

Time complexity: O(logn)

```
 while(n >0) {
        if((n &1)==1)
            count++
        n = n >>1
 print count
```

Optimized Approach (**Brian Kernighan's Algorithm) O(K) where K is number of set bits**

It works by repeatedly checking the least significant bit and clearing it.
This happens due to how binary subtraction works:

- Subtracting 1 from a number flips all the bits **after** the rightmost set bit (including that bit).
- ANDing with the original number then **clears** that bit and keeps everything else the same.

```
 while(n >0)
        n = n & (n -1)
```

```
            count++
   print count
```

```
n = 52
Binary of 52 → 110100
 n - 1 = 51
Binary → 110011
  n     = 110100
n - 1   = 110011
----------------
n & n-1 = 110000
increment count → count = 1
```

The rightmost set bit in n is at position 2 (counting from right, 0-based).

- That bit is cleared in n & (n - 1).
- All higher bits remain unchanged.
- All lower bits (to the right of the cleared bit) become 0.

```
n = 48 → 110000
n - 1 = 47 → 101111
n & (n - 1) = 110000 & 101111 = 100000 → 32
increment count → count = 2
```

```
n = 32 → 100000
n - 1 = 31 → 011111
n & (n - 1) = 100000 & 011111 = 000000 → 0
increment count → count = 3
```

hence number of set bits in 52 are 3

Sum of Set Bits

Problem:Count the total number of set bits from 1 to given range N

Input:
5

Output:
7

Explanation:

1 → 0001 → 1
2 → 0010 → 1
3 → 0011 → 2
4 → 0100 → 1
5 → 0101 → 2
Total = 1 + 1 + 2 + 1 + 2 = 7 set bits

Naïve approach

```
int totalCount = 0
 for (int i = 1; i <= n; i++)
{
   int num = i;
    while (num > 0)
   {
        totalCount += (num & 1)
        num >>= 1 }
   }
   O(nlogn)
```

Step by step approach: (optimized approach O(logn) using bit pattern approach)

```
Input: Integer n
Initialize total = 0
For each bit position i starting from 0 while (2^i <= n):
    Set cycle_length = 2^(i + 1)
    Calculate complete_cycles = n / cycle_length
    Add to total: complete_cycles * (cycle_length / 2)
    Calculate remainder = n % cycle_length

    If remainder >= 2^i:
        Add to total: remainder - 2^i + 1

Print total
```

Step-by-step Trace for n = 10
Initial:
= 10, total = 0

☑ i = 0 (LSB - bit 0)
1 << 0 = 1
cycle_len = 1 << (0 + 1) = 2
complete_cycles = 10 / 2 = 5
total += 5 * 1 = 5
rem = 10 % 2 = 0
rem ⩾ 1 → false → nothing added
➡ total = 5

☑ i = 1 (bit 1)
1 << 1 = 2
cycle_len = 1 << (1 + 1) = 4
complete_cycles = 10 / 4 = 2
total += 2 * 2 = 4
rem = 10 % 4 = 2
rem ⩾ 2 → true → total += 2 - 2 + 1 = 1
➡ total = 5 + 4 + 1 = 10

☑ i = 2 (bit 2)
1 << 2 = 4
cycle_len = 1 << (2 + 1) = 8
complete_cycles = 10 / 8 = 1
total += 1 * 4 = 4
rem = 10 % 8 = 2
rem ⩾ 4 → false → nothing added
➡ total = 10 + 4 = 14

☑ i = 3 (bit 3)
1 << 3 = 8
cycle_len = 1 << (3 + 1) = 16
complete_cycles = 10 / 16 = 0
total += 0
rem = 10 % 16 = 10
rem ⩾ 8 → true → total += 10 - 8 + 1 = 3
➡ total = 14 + 3 = 17

total += 10 - 8 + 1 = 3
❓ What does this 3 mean?
☑ Answer:
It means there are 3 numbers between 1 and 10 where bit 3 is set to 1 in the incomplete cycle (last part) — specifically, the numbers:

8 → 1000
9 → 1001
10 → 1010
Each of these has bit 3 (value 8) set to 1, so we add 3 to the total.
This calculation is counting how many values in the remainder (last partial cycle) have the bit i (bit 3) set.

If remainder = 6, and 2^i = 4, then range is:
→ Values with bit i set: 4, 5, 6
→ Count: 6 - 4 + 1 = 3 (inclusive count)

If you want to count how many integers are there from a to b, including both ends, the formula is:
Count = b - a + 1

- From 4 to 6 → numbers are: 4, 5, 6
- Count = 3
- 6 - 4 = 2 → wrong
- So, correct: 6 - 4 + 1 = 3

→ That's where +1 comes in: inclusive counting

- The +1 is needed because you're counting an inclusive range from 1 << i to remainder
- Even if the range has just one number, like 10, it should be counted — hence +1
- Without it, you'd miss the last number in that range
- If remainder has crossed 1 << i, then we're in the part where bit i is 1"
- And the first such number is exactly 1 << i

⛔ i = 4
1 << 4 = 16 > n → loop stops

☑ Final Answer:17

📊 Actual Binary Set Bit Count from 1 to 10:

Number	Binary	Set Bits
0	0000	0
1	0001	1
2	0010	1
3	0011	2
4	0100	1
5	0101	2
6	0110	2
7	0111	3
8	1000	1
9	1001	2
10	1010	2
Total		17

for n=8
Pattern Analysis Per Bit:
▨ Bit 0 (LSB):
Repeats every 2 numbers: 01010101
☑ Set in: 1, 3, 5, 7 positions → 4 times

▨ Bit 1:
Repeats every 4 numbers: 00110011

☑ Set in: 2, 3, 6, 7 → 4 times

▨ Bit 2:
Repeats every 8 numbers: 00001111
☑ Set in: 4, 5, 6, 7 → 4 times

▧ Bit 3:
Repeats every 16 numbers: 0000000011111111
☑ Set in: only 8 → 1 time

☑ Total Set Bits:
4 (bit 0) + 4 (bit 1) + 4 (bit 2) + 1 (bit 3) = 13 set bits

Binary Set Bit Patterns from 1 to 16

Bit Position	Pattern Example	Explanation	Numbers Where Bit Is Set	Count
▨ Bit 0 (LSB)	01 01 01 01 01 01 01 01	Repeats every 2 numbers	1, 3, 5, 7, 9, 11, 13, 15	8
▨ Bit 1	00 11 00 11 00 11 00 11	Repeats every 4 numbers	2, 3, 6, 7, 10, 11, 14, 15	8
▧ Bit 2	0000 1111 0000 1111	Repeats every 8 numbers	4, 5, 6, 7, 12, 13, 14, 15	8
▩ Bit 3	00000000 11111111	Repeats every 16 numbers	8, 9, 10, 11, 12, 13, 14, 15	8
▨ Bit 4	0000000000000001	Repeats every 32 numbers	16	1

📊 Total Set Bits from 1 to 16:

Bit 0: 8
Bit 1: 8
Bit 2: 8
Bit 3: 8
Bit 4: 1

Total: 33 ☑

- For a given bit position i, the bit pattern repeats every cycleLength = 2^(i + 1) numbers
- In each cycle of length 2^(i + 1), the bit at position i is set to 1 for half of those numbers
- So we add:

completeCycles * (cycleLength / 2)
Which is the total number of set bits at bit i from all the full cycles.

Let's say:
- i = 2 → Bit 2
- So cycleLength = 2^(2 + 1) = 8

Bit 2 (i=2) pattern across 0–7:

000 (0)
001 (1)
010 (2)
011 (3)

100 (4) ☑
101 (5) ☑
110 (6) ☑
111 (7) ☑ ← bit 2 is 1 from 4 to 7 → 4 times

- That's cycleLength = 8
- Bit 2 is 1 in half of the cycle → 8 / 2 = 4 = 2^2

Count set bits and store in Array

Problem:Count of Set bits for each number in the range upto N and Store it in array

Input: 5
Output: [0, 1, 1, 2, 1, 2]

Naïve approach

```
int[] arr = new int[n + 1];

 for (int i = 0; i <= n; i++)
{
            int count = 0
            int num = i
         while (num > 0)
           {
                if (num % 2 == 1)
                   count++
                num = num / 2
            }
            arr[i] = count
}
print arr
```

O(nlogn)

Optimized approach using DP (Tabulation)

→ The number of set bits in i equals:
   (number of set bits in i / 2) plus 1 if i is odd, 0 if i even

---

```
int[] arr = new int[n + 1]
arr[0]=0
for (int i = 1; i <= n; i++)
{
       arr[i] = arr[i/2] + (i & 1);  //arr[i>>1]
}
print arr
```

Time Complexity O(n)

tracing for N=5
i = 0 → 0
i = 1 → dp[1 >> 1] + (1 & 1) = dp[0] + 1 = 1

```
i = 2 → dp[1] + 0 = 1
i = 3 → dp[1] + 1 = 2
i = 4 → dp[2] + 0 = 1
i = 5 → dp[2] + 1 = 2
       Final Output: 0 1 1 2 1 2
```

```
        int[] arr = new int[n + 1];

        arr[0] = 0;

        for (int i = 1; i <= n; i++) {
            arr[i] = arr[i / 2] + (i & 1);
        }

        for (int i = 0; i <= n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Find unique element in the array

Problem:Given an array of size N find the single unique element in the array

Input:4 1 2 1 2 4 7
Output: 7

Explanation:

Basic Approach:

Compare each element with all others and find the one which has no duplicate.
but time consuming since it take O(n^2)

```
for i = 0 to n-1
    count = 0
    for j = 0 to n-1:
        if arr[i] == arr[j] then
            count = count + 1
    if count == 1 then
        print arr[i]
        break
```

Using Hashmap O(n)time O(n) space

Input:4 1 2 1 2 4 7
Map: {4=2, 1=2, 2=2, 7=1}
Unique: 7 → output

Using Xor O(n) time O(1) space
---
a ^ a = 0, a ^ 0 = a
So all duplicate elements cancel each other and the unique remains.

```
for(int i = 0; i < n; i++)
 xor ^= arr[i]
```

4 ^ 1 = 5
5 ^ 2 = 7
7 ^ 1 = 6
6 ^ 2 = 4
4 ^ 4 = 0
0 ^ 7 = 7 ← Unique

Find missing number in the given array

Problem: Given an array nums containing n distinct numbers in the range [0, n], find the only number that is missing from the given array.

Input: 3
3 0 1
Output:2

Explanation:
0 is always included in the array. missing number in the range is 2.

Naïve Approach

```
for i from 0 to n do
    found =false

    for j from 0 to n - 1 do
        if nums[j] == i then
            found ← true
            break

    if found is 0 then
        print i
        break
```

for example: if array is [3, 0, 1]
for third iteration i=2
found = 0
Inner loop:
  j = 0: nums[0] != 2
  j = 1: nums[1] != 2
  j = 2: nums[2] != 2
found == 0, so 2 is missing and printed as the result.

boolean nested array

```
 boolean[] present = new boolean[n+1];

        for(int i=0; i<n; i++) {
            nums[i] = sc.nextInt();
            present[nums[i]] = true;
        }

        for(int i=0; i<=n; i++) {
            if(!present[i]) {
                System.out.println(i);
                return;
```

Better approach using summation forumula

Input: nums = [3, 0, 1]
n = 3

```
Step 1: Calculate the sum of numbers from 0 to n
Using the formula sum = n * (n + 1) / 2:
sum = 3 * (3 + 1) / 2 = 3 * 4 / 2 = 6
Step 2: Calculate the actual sum of elements in nums:
actual_sum = 3 + 0 + 1 = 4
Step 3: Find the difference:
missing = sum - actual_sum = 6 - 4 = 2
The missing number is 2.


Best Approach using XoR
-------------------------
```

Naive Approach	O(n²)	O(1)	Slow, not efficient
Sum Formula	O(n)	O(1)	Efficient but requires sum
XOR (Best)	O(n)	O(1)	Most efficient and elegant

```
initialize xor as 0
for i = 0 to n:
    xor1 = xor1 ^ i

initialize xor1 as 0

for i 0 to n-1:
    xor2 = xor2 ^ nums[i]

missingNumber = xor1 ^ xor2
```

```
 int[] nums = new int[n]

        int xor_full = 0
        int xor_array = 0

        for(int i=0; i<n; i++) {
            nums[i] = sc.nextInt()
            xor_array ^= nums[i]
        }

        for(int i=0; i<=n; i++)
            xor_full ^= i


   System.out.println(xor_full ^ xor_array)
```

```
  Input: nums = [3, 0, 1]
  n = 3
Step 1: XOR all numbers from 0 to n:
  xor = 0
  xor ^= 0 → xor = 0
  xor ^= 1 → xor = 1
  xor ^= 2 → xor = 3
  xor ^= 3 → xor = 0 (XOR of all numbers from 0 to 3)
Step 2: XOR all elements in nums:
  xor ^= 3 → xor = 3
  xor ^= 0 → xor = 3
```

```
  xor ^= 1 → xor = 2 (final XOR result)
The missing number is 2.
```

X1sY0s to decimal

Problem:Given X number of 1's followed by Y number of 0's find the decimal value

Input:2 2 (1100)
Output:12

Naïve approach

```
read x and y as integer values
initialize empty result string
loop from i= 0 to x-1 do
  result= result+ "1"

loop from i=0 to y-1 do
  result = result + "0"

  int dec=Integer.parseInt(result,2)
```

- **Time Complexity**: O(x + y)
- **Space Complexity**: O(x + y) (binary string stored in memory)

Optimized apprach

```
int ones = (1 << x) - 1  (1<<X raise power to the respective bit and subtracting 1 from it gives
                           the number of 1's as LSB)
int result = ones << y   (shifting ones again y times gives us number of 0's at MSB)

print result
```

No string operations
Uses direct bit manipulation
Time: **O(1)**
Space: **O(1)**

2 **<< 3**
It means:
Shift the binary representation of 2 to the left by 3 positions.

2 >> 3
This means:
Shift the binary representation of 2 to the right by 3 bits.

XY set bit to decimal

Problem:You are given two integers, x and y, representing bit positions.
Your task is to create a number where **only the x-th and y-th bits are set to 1** (rest all bits are 0).

Input:2 5

Output:36

Explanation:binary value will be 100100 which is 36.. (2nd and 5th bit are set)

Naïve Approach

Input: x = 2, y = 5
take 2^x and 2^y and sum them.

val1 = 2^2 = 4
val2 = 2^5 = 32
result = 4 + 32 = 36

- Uses **floating-point math (Math.pow)**, then casts to int.
- **Slower** and **less precise** for larger values.

Optimized approach

(1 << x) | (1 << y)
This directly sets bits using fast bit manipulation without relying on floating-point computation.

OR operation allows to combine the set bits from x and y into a single number

XoR of sum of Pairs

Problem: Given an integer array arr of size n, compute the XOR of all pairwise sums of the elements in the array.
Note: Here (A[i], A[i]), (A[i], A[j]), (A[j], A[i]) all are considered as different pairs.

Naive approach O(n^2)

```
result = 0
for i from 0 to n-1:
    for j from 0 to n-1:
        sum = arr[i] + arr[j]
        result = result ^ sum
print result
```

suppose array size is 2 and elements are 1 and 2: [1,2]

i	j	arr[i]	arr[j]	arr[i] + arr[j]
0	0	1	1	2
0	1	1	2	3
1	0	2	1	3
1	1	2	2	4

ans = 0 ^ 2 ^ 3 ^ 3 ^ 4
= ((2 ^ 3) ^ 3) ^ 4
= (1 ^ 3) ^ 4
= 2 ^ 4
= 6

Let's count how many times the inner loop runs.

- When i = 0, j runs n times
- When i = 1, j runs n - 1 times
- When i = 2, j runs n - 2 times
- ...
- When i = n - 1, j runs 1 time

So total operations:

= n + (n-1) + (n-2) + ... + 1
= n(n+1)/2
= O(n²)

Optimized Approach:O(n)

```
xor = 0
    for i from 0 to n-1:
        xor = xor ^ arr[i]
    print(xor * 2)
```

Input: arr[] = {1, 5, 6}
Output: 4
all possible pairs = { 1+1, 1+5, 1+6, 5+1, 5+5, 5+6, 6+1, 6+5, 6+6}
{ 2, 6, 7, 6, 10, 11, 7, 11, 12}
2 ^ 6 ^ 7 ^ 6 ^ 10 ^ 11 ^ 7 ^ 6 ^ 11 ^ 12 = 4
as XoR property, a^a=0 hence same elements gets cancelled out..
and remaining are 2^10^12=4

as u can observe remaining elements each are the twice of original elemnts..
hence its same as finding xor of 1 5 6 and then multiplying by 2

Input:
3
5 1 2
Output:
12

Sum of Xor of all unique pairs

Problem: Given an array of n integers, find the sum of xor of all pairs of numbers in the array.

Input : {7, 3, 5}
Output : 12
7 ^ 3 = 4
3 ^ 5 = 6
7 ^ 5 = 2
Sum = 4 + 6 + 2
= 12

Naive Approach:

```
for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++)
                Sum += arr[i] ^ arr[j]

        print Sum
```

Optimized Approach:

```
--------------------------
total = 0

for bit = 0 to 31:
    count_set = 0

    for i = 0 to n - 1:
        if ((arr[i] >> bit) & 1) == 1:
            count_set = count_set + 1

    count_unset = n - count_set

    pair_contribution = count_set * count_unset * (1 << bit) (Equals to 2^bit)

    total = total + pair_contribution

return total
```

Java use **32-bit signed integers**, where:

- The range of an int in Java is:
  -2^31 to 2^31 - 1
  i.e., -2147483648 to 2147483647
- So, even the largest positive value 2^31 - 1 takes **31 bits** (0 to 30) for magnitude, and the **31st bit (index 31)** is used for the **sign bit** (1 for negative, 0 for positive).
- We go from **bit 0 to 31** to **cover all bits in a 32-bit integer**

arr = {2, 3, 4, 5, 7}
Binary forms:
2 → 0010 ☑ bit 1 set
3 → 0011 ☑ bit 1 set
4 → 0100 ❌ bit 1 not set
5 → 0101 ❌
7 → 0111 ☑

🔍For bit 1:
Set (bit 1 = 1): 2, 3, 7 → total = 3 → countSet = 3
Unset (bit 1 = 0): 4, 5 → total = 2 → countUnset = 2

Now list **every pair** (x, y) such that:

- x has bit 1 set
- y has bit 1 unset

From set: 2

- (2, 4)
- (2, 5)

From set: 3

- (3, 4)
- (3, 5)

From set: 7

- (7, 4)
- (7, 5)

☑ Total such pairs: **3 × 2 = 6**
Each of these pairs, when XORed, will have **bit 1 set to 1**.

- So: countSet * countUnset = 6 valid pairs
- formula: countset*countunset* 2^1 (bit 1 weight)

And since **bit 1 represents 2¹ = 2**,
➡**Each such pair contributes 2 to the total sum** of all XORs.

Total contribution from bit 1 = 3 * 2*2^1 = 12
So, when bit 1 is set in the XOR result, it adds a value of 2 to the total sum.

example 2:
---
arr = {7, 3, 5}
n = 3

🔁 bit = 0
int countSet = 0;
▶ Inner loop (i = 0 to 2):
i = 0: arr[0] = 7, 7 >> 0 = 7, 7 & 1 = 1 → ☑ countSet = 1

i = 1: arr[1] = 3, 3 >> 0 = 3, 3 & 1 = 1 → ☑ countSet = 2

i = 2: arr[2] = 5, 5 >> 0 = 5, 5 & 1 = 1 → ☑ countSet = 3

int countUnset = n - countSet = 3 - 3 = 0;
total += 3 * 0 * (1 << 0) = 0
✔ total = 0

🔁 bit = 1
countSet = 0;
▶ Inner loop:
i = 0: 7 >> 1 = 3, 3 & 1 = 1 → ☑ countSet = 1

i = 1: 3 >> 1 = 1, 1 & 1 = 1 → ☑ countSet = 2

i = 2: 5 >> 1 = 2, 2 & 1 = 0 → ❌

countUnset = 3 - 2 = 1;
total += 2 * 1 * (1 << 1) = 2 * 1 * 2 = 4;
✔ total = 0 + 4 = 4

🔁 bit = 2

countSet = 0;
▶️ Inner loop:
i = 0: 7 >> 2 = 1, 1 & 1 = 1 → ☑️ countSet = 1

i = 1: 3 >> 2 = 0, 0 & 1 = 0 → ❌

i = 2: 5 >> 2 = 1, 1 & 1 = 1 → ☑️ countSet = 2
countUnset = 3 - 2 = 1;
total += 2 * 1 * (1 << 2) = 2 * 1 * 4 = 8;
✔️ total = 4 + 8 = 12

🔁 bit = 3 to bit = 31
All bits higher than 2 in 7, 3, 5 are 0.
For each bit:
countSet = 0, countUnset = 3
total += 0 * 3 * (1 << bit) = 0

✔️ **total Sum remains 12**

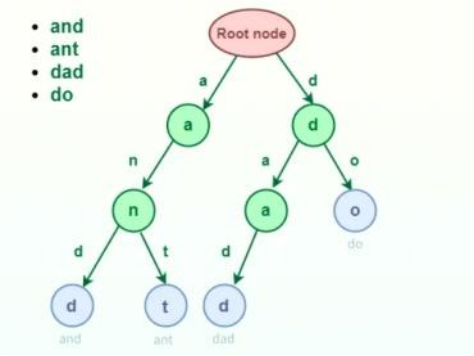
input= 5 9 7 6
output=47

This is for the **optimized XOR sum of all pairs**:
- **Time:** O(32 * n) → Since we go over 32 bits for each of n elements.
- **Space:** O(1)

Trie Data Structure

- A Trie (or prefix tree) is a tree-like data structure used to store sequences (like strings or bit sequences).
- Each edge represents one element (character or bit).
- Common prefixes share the same path from the root.
- Very useful for prefix-related queries.

Trie supports operations such as insertion, search, deletion of keys, and prefix searches.



- **Insert/Search/Prefix Search** all run in **O(L)**, where **L** is length of the word or prefix.
- This is **faster than hash maps** in some cases, especially for prefix searches and when collisions are costly

- Space depends on total nodes, up to **O(ALPHABET_SIZE × N × L)** worst case.(N: length of word)
- Usually large because each node has an array of child pointers.

Trie Applications

1. **Autocomplete / Auto-suggestion**
   Suggest words based on a prefix typed by the user.
2. **Spell Checking**
   Verify if a word exists and suggest corrections.
3. **Implementing Dictionaries**
   Storing and retrieving words efficiently.
4. **Finding Longest Prefixes in dictionary**
   For example, in search engines or IP address lookup. etc

Representation of Trie Node

- **Trie data structure** consists of nodes connected by edges.
- Each node represents a character or a part of a string.
- The root node acts as a starting point and does not store any character.

TrieNode Class
Purpose:
Represents a **single node** in the Trie.
Contains:

- An array of child nodes (usually size 26 for lowercase letters).
- A flag to indicate if the node represents the **end of a valid word**

We’re inserting "apple" into the Trie.

```
TrieNode curr = root;
for (char ch : word.toCharArray()) {
    int index = ch - 'a';
    if (curr.children[index] == null) {
        curr.children[index] = new TrieNode();
    }
    curr = curr.children[index];
}
```

Step 1: 'a'

- ch = 'a'
- index = 'a' - 'a' = 0
- curr = root

```
if (curr.children[0] == null)
    curr.children[0] = new TrieNode();
curr = curr.children[0];
```

🔁 curr now points to the 'a' node.

Step 2: 'p'

Now, we're processing the **next character 'p'**.

- ch = 'p'
- index = 'p' - 'a' = 15

```
if (curr.children[15] == null)
    curr.children[15] = new TrieNode();
curr = curr.children[15];
```

👉 But here’s the **key**:

- At this point, curr is already pointing to the 'a' node (from the previous step).
- So curr.children[15] is accessing the 'p' **child of the 'a' node.**

Max Subarry XoR

Problem:
Given an array of integers. The task is to find the **maximum subarray XOR** value in the given array.

Input: *arr[] = {1, 2, 3, 4}*
Output: *7*
Explanation: *The subarray {3, 4} has maximum XOR value*

Input:arr[] = {8, 1, 2, 12, 7, 6}
Output: 15
Explanation: The subarray {1, 2, 12} has maximum XOR value

Naive Approach:

```
int max = 0
        for (int i = 0; i < n; i++)
            int xor = 0
            for (int j = i; j < n; j++)
                xor = xor ^ arr[j]
               if (xor > max)
                  max = xor;
            }
        }

     print max
```

Optimized Approach:
Insert each number: O(32)
Query each number: O(32)
Total: **O(n)** for n = array size

A ^ A = 0
A ^ 0 = A
And most importantly:
(A ^ B ^ C) ^ (A ^ B) = C
(A ^ B ^ C) ^ (A ^ B)
= (A ^ B) ^ C ^ (A ^ B)
= ((A ^ B) ^ (A ^ B)) ^ C
= 0 ^ C = C

Given an array arr[], we define:

prefix[0] = arr[0]
prefix[1] = arr[0] ^ arr[1]
prefix[2] = arr[0] ^ arr[1] ^ arr[2]
...
prefix[j] = arr[0] ^ arr[1] ^ ... ^ arr[j]

For array: [1, 2, 3, 4]
Prefix XORs:
prefix[0] = 1
prefix[1] = 1 ^ 2 = 3
prefix[2] = 3 ^ 3 = 0
prefix[3] = 0 ^ 4 = 4

So when you XOR two prefix values:

prefix[j] ^ prefix[i - 1]
You effectively **cancel out the first i elements**, and are left with:
arr[i] ^ arr[i+1] ^ ... ^ arr[j]
Which is exactly the **XOR of the subarray from i to j.**

arr = [1, 2, 3, 4]
Assume We want to compute the XOR of subarray from i = 1 to j = 3, i.e.:
Normal calcuation: arr[1] ^ arr[2] ^ arr[3] = 2 ^ 3 ^ 4 = 5

Step 1: **Compute prefix XORs**

index	prefix[i] (XOR from 0 to i)
0	arr[0] = 1
1	arr[0] ^ arr[1] = 1 ^ 2 = 3
2	1 ^ 2 ^ 3 = 3 ^ 3 = 0
3	1 ^ 2 ^ 3 ^ 4 = 0 ^ 4 = 4

so prefix[3] = 4
prefix[i-1]=prefix[0] = 1

Step 2: Apply formula
To compute XOR from index i=1 to j=3:

subarray_xor = prefix[3] ^ prefix[0] = 4 ^ 1 = 5 ✔

Explaination of cancel out:
---
prefix[3] = arr[0] ^ arr[1] ^ arr[2] ^ arr[3]
= 1 ^ 2 ^ 3 ^ 4

prefix[0] = arr[0]
= 1

Now,
prefix[3] ^ prefix[0]
= (1 ^ 2 ^ 3 ^ 4) ^ 1

Apply XOR associativity:
= 1 ^ 2 ^ 3 ^ 4 ^ 1

= (1 ^ 1) ^ 2 ^ 3 ^ 4
= 0 ^ 2 ^ 3 ^ 4
= 2 ^ 3 ^ 4 = 5 ✔

Hence
To get XOR of subarray from index i to j, do:
XOR(i, j) = prefix[j] ^ prefix[i - 1]
Because the earlier terms cancel out using XOR properties

Idea:

For the current prefix XOR (say, prefix[j]), we want to **find a previous prefix XOR** (prefix[i - 1]) in such a way that their XOR is **as large as possible.**
So:

- We build a Trie with all the **previous prefix XORs**
- For every new prefix XOR, we **query the Trie** to find the **best matching XOR** (i.e., most differing bits from high to low)
- This lets us compute max(prefix[j] ^ best_match) in **O(32)** time per element.
- By checking for **opposite bits at each level,** you greedily maximize XOR.

Visual Diagram after for trie based on binary values (only last 3 bits):

We'll start with xor = 0 and update it with each element of the array:

i	arr[i]	xor (prefix)	3-bit binary
0	–	0	000
1	1	0 ^ 1 = 1	001

2	2	1 ^ 2 = 3	011
3	3	3 ^ 3 = 0	000
4	4	0 ^ 4 = 4	100

We will insert each of these XORs into the Trie using **3 bits** from MSB to LSB (bit 2 → 1 → 0).

🔁 **Step-by-step Insertion**

🔹 **Insert 000 (prefix 0)**
Binary: 000

Bit Position (i)	Bit	Action
2	0	Create child[0] at root
1	0	Create child[0] under previous
0	0	Create child[0] under previous

```
root
 └── 0
     └── 0
         └── 0 (represents 000)
```

🔹 **Insert 001 (prefix 1)**
Binary: 001

Bit Position (i)	Bit	Action
2	0	Already exists
1	0	Already exists
0	1	Create child[1] under previous node

```
root
 └── 0
     └── 0
         ├── 0 (000)
         └── 1 (001)
```

🔹 **Insert 011 (prefix 3)**
Binary: 011

Bit Position (i)	Bit	Action
2	0	Already exists
1	1	Create child[1] under first 0
0	1	Create child[1] under previous

Trie:

```
root
 └── 0
     ├── 0
     │   ├── 0 (000)
     │   └── 1 (001)
     └── 1
         └── 1 (011)
```

🔹 **Insert 000 again (prefix 0)**
Binary: 000
☑ Path already exists: no new nodes created.

🔹 **Insert 100 (prefix 4)**
Binary: 100

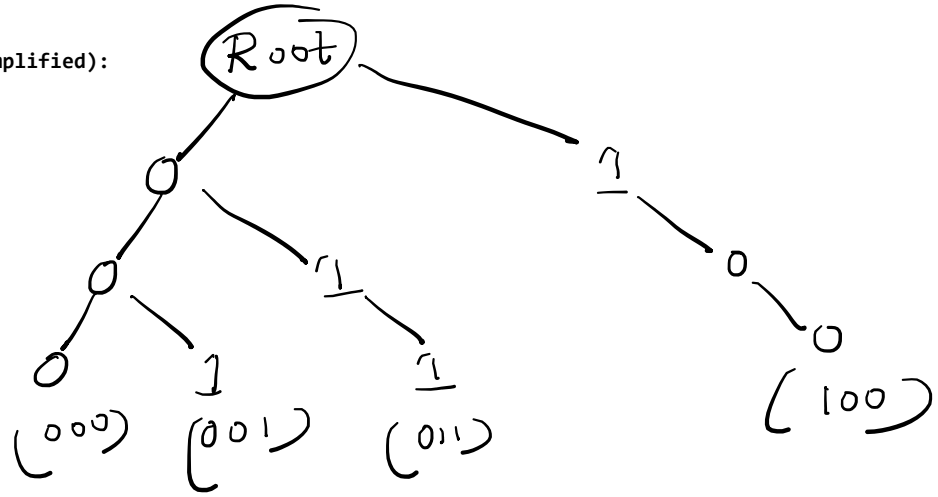
Bit Position (i)	Bit	Action
2	1	Create child[1] at root
1	0	Create child[0]
0	0	Create child[0]

```
root
 ├── 0
 │   ├── 0
 │   │   ├── 0 (000)
 │   │   └── 1 (001)
 │   └── 1
 │       └── 1 (011)
```

```
    └── 1
        └── 0
            └── 0 (100)
```

☑ **Final Trie Diagram (simplified):**

```
root
 ├── 0
 │   ├── 0
 │   │   ├── 0 (000)
 │   │   └── 1 (001)
 │   └── 1
 │       └── 1 (011)
 └── 1
     └── 0
         └── 0 (100)
```



query() greedily walks the Trie from top to bottom, and at each level:

- It looks at the current bit in val
- Tries to go to the child node with the **opposite bit** (to maximize XOR)
- If it can't, it takes the same bit path

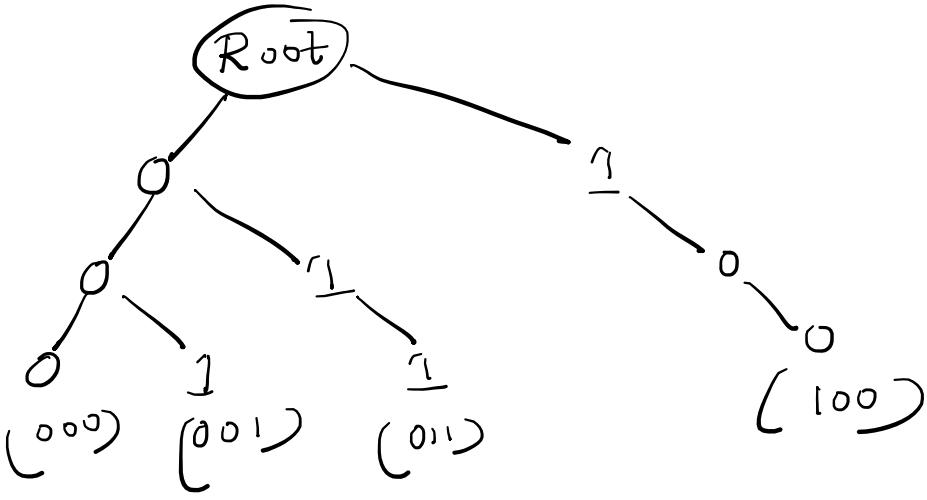
The result is the **maximum XOR** value it can make using any prefix stored in the Trie.

To **maximize XOR**, we want to choose **opposite bits** at each level.

- If current bit is 0, try 1 in Trie.
- If current bit is 1, try 0 in Trie.

Because 1 ^ 0 = 1 (better than 1 ^ 1 = 0)

Let's Run query(root, 4)
we want to find the number in the Trie that maximizes 4 ^ x, where x is a prefix already in the Trie.
int xor = 0;
We're now trying to find the number in the Trie that gives the **maximum XOR** with 4.
4 = 100 (in 3-bit view: bits 2,1,0)
We start from **bit 2 (MSB) → bit 0 (LSB)**



▶ **Bit 2:**
bit = (4 >> 2) & 1 = 1
opp = 0
We want **opposite** of 1 → that is 0
Check if curr.child[0] exists from root:
☑ YES (0, 1, 3 all go through bit 2 = 0)
So:

xor |= (1 << 2); // 0 | 4 = 4
curr = curr.child[0];
We now move down into the child[0] branch.
☑ We just locked in a 1 at **bit 2** in XOR.

▶ **Bit 1:**
bit = 0
opp = 1
We want child[1]
☑ YES (3 has bit1 = 1)
So:
xor |= (1 << 1); // 4 | 2 = 6
curr = curr.child[1];
☑ Locked in a 1 at **bit 1**

▶ **Bit 0:**
bit = 0
opp = 1
We want child[1]
☑ YES (3 has bit0 = 1)

xor |= (1 << 0); // 6 | 1 = 7

curr = curr.child[1];
☑ Locked in a 1 at **bit 0**
So, the number in the Trie that gives the **best XOR with 4** is 3
Because:

4 ^ 3 = 7 ☑
Full XOR Tracing:

Bit Position	Bit in val (4)	Opp Exists?	XOR Gained
2 (MSB)	1	0 ☑	+4 (100)
1	0	1 ☑	+2 (010)
0 (LSB)	0	1 ☑	+1 (001)

Total XOR = 4 + 2 + 1 = 7 (4^3=7)

`xor |= (1 << i)`
signifies that **at bit i**, we found a value in the Trie that had the **opposite bit** — which helps set bit i in the result to 1
That increases the total XOR value.

- At each bit:
  - If the **opposite bit exists**, we go there (to maximize XOR).
  - **Because XOR of opposite bits gives 1**, we **add (1 << i)** to the result.

- You query with current prefix XOR (like 12) to find which **previous prefix** gives the **maximum XOR** when XORed with it.
- This helps find the **maximum subarray XOR ending at current index**.
- You repeat this for every index to get the global max.

arr = [3, 10, 5]
prefix[0] = 0 (initial)
prefix[1] = 3 (0 ^ 3)
prefix[2] = 9 (3 ^ 10)
prefix[3] = 12 (9 ^ 5)
At i = 2, you have:

xor = 12

You call: query(12)

Trie has previous prefix values: {0, 3, 9}

Then:

12 ^ 0 = 12

12 ^ 3 = 15

12 ^ 9 = 5

So the best you can get is 15

Hence, query(12) is returning 15, and you update your max = 15

XorSUM of all pairs with AND

Problem:
You are given two integer arrays, arr1 and arr2, both containing non-negative integers. Define a new list containing the bitwise AND of every possible pair formed by taking one element from arr1 and one from arr2

Your task is to find the XOR sum of all elements in this new list, where the XOR sum is the bitwise XOR of all the elements.

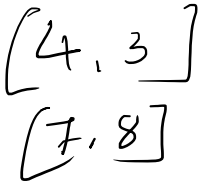
Input:
arr1 = [4, 7]
arr2 = [3, 8]

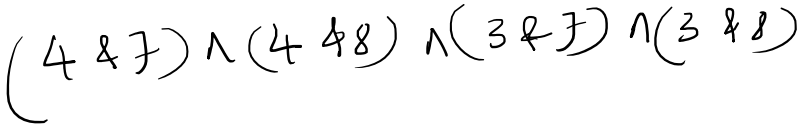
Output: 4

Naive Approach:

```
int xor_sum = 0;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                xor_sum ^= (arr1[i] & arr2[j])

        System.out.println(xor_sum)
```

Pairs:
4 & 3 = 0
4 & 8 = 0
7 & 3 = 3
7 & 8 = 0
List = [0, 0, 3, 0]
XOR sum = 0 ^ 0 ^ 3 ^ 0 = 3

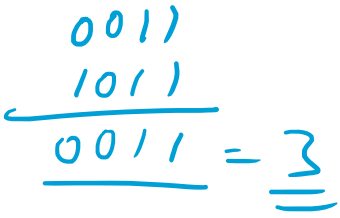




Optimized Approach:
Bitwise property:
(a1 & b1) ^ (a1 & b2) ^ ... ^ (a2 & b1) ^ (a2 & b2) ...
can be rewritten using distributivity:
= (a1 ^ a2 ^ ... ^ an) & (b1 ^ b2 ^ ... ^ bm)
So the final XOR sum is:
(arr1_xor) & (arr2_xor)
Time Complexity: O(n + m)
Space Complexity: O(1)

- XOR(arr1) = 4 ^ 7 = 3
- XOR(arr2) = 3 ^ 8 = 11 (binary 1011)
- Result = 3 & 11 = 3

Output matches XOR sum = 3



(a1 & b1) ^ (a1 & b2) ^ (a2 & b1) ^ (a2 & b2)
= (a1 ^ a2) & (b1 ^ b2)
Let:

a1 = 4, a2 = 7

b1 = 3, b2 = 8

```
arr1 = [1, 2, 3]
arr2 = [6, 5]
Trace (Naive)
Compute AND of all combinations:
1 & 6 = 0
1 & 5 = 1
2 & 6 = 2
2 & 5 = 0
3 & 6 = 2
3 & 5 = 1
Resulting list: [0, 1, 2, 0, 2, 1]
XOR sum: 0 ^ 1 ^ 2 ^ 0 ^ 2 ^ 1 = 0
```

Distributive property is similar for AND and XOR.
(a1^a2) & (b1^b2) = (a1&b1) ^ (a1&b2) ^ (a2&b1) ^ (a2&b2)

XOR Distributive Diagram
(1&2&3) ^ (6&5)

```
XOR(arr1) = 1 ^ 2 ^ 3 = 0
XOR(arr2) = 6 ^ 5 = 3
Final Result = 0 & 3 = 0
```

```
initialize xor1 to 0
for i from 0 to n-1 do
    xor1 = xor1 ^ arr1[i]

initialize xor2 to 0
for j from 0 to m-1 do
    xor2 = xor2 ^ arr2[j]

result = xor1 AND xor2
print result
```

Bitwise XoR of all Pairings

Problem:
You are given two arrays nums1 and nums2 containing non-negative integers.
Consider all pairs (i, j) where i ranges over indices of nums1 and j ranges over indices of nums2. For each pair, compute the bitwise XOR of nums1[i] and nums2[j].
print bitwise xor of all possible pairs from num1 and num2.

Input:
3 4
2 1 3
Output:
13

Naive Approach:

```
 for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                result ^= (nums1[i] ^ nums2[j]);
            }
        }

        System.out.println(result);
```

Optimized Approach:

$(a \wedge b) \wedge c = a \wedge (b \wedge c)$

XOR is commutative and associative:
a ^ b = b ^ a, and (a ^ b) ^ c = a ^ (b ^ c)
When you XOR the same number multiple times:

- XORing a number **even** times results in **0**
  (because x ^ x = 0, so pairs cancel out)
- XORing a number **odd** times results in the number itself
  (because if you have an odd number of x's, one x remains after pairs cancel)

Let's say:
nums1 = [a1, a2, ..., an]
nums2 = [b1, b2, ..., bm]
All pairwise XORs:
(a1 ^ b1), (a1 ^ b2), ..., (a1 ^ bm), (a2 ^ b1), ..., (an ^ bm)
Rewriting the total XOR:
(a1 ^ a2 ^ ... ^ an) repeated m times ^ (b1 ^ b2 ^ ... ^ bm) repeated n times
So the total XOR is:
if m is odd: use XOR of all nums1
if n is odd: use XOR of all nums2

Input:
nums1 = [2, 1, 3]
nums2 = [10, 2, 5, 0]

- n = length of the first array (nums1)
- m = length of the second array (nums2)

- XOR(nums1) = 2 ^ 1 ^ 3 = 0
- XOR(nums2) = 10 ^ 2 ^ 5 ^ 0 = 13
- m = 4 (even), so ignore XOR(nums1)
- n = 3 (odd), so take XOR(nums2)

Result = 0 ^ 13 = 13

If n is odd and m is even, why do elements from nums1 cancel out, and only xor(nums2) remains?

Each element of nums1 is XORed with **all** elements of nums2.
So every nums1[i] is repeated m times in pairing.
Likewise, every nums2[j] is repeated n times in pairing.
hence When n is odd and m is even → only xor(nums2) contributes to the final result.

So the final result is:

$$\text{result} = \begin{cases} XOR(nums1) \oplus XOR(nums2) & \text{if both } m, n \text{ are odd} \\ XOR(nums1) & \text{if } m \text{ is odd and } n \text{ is even} \\ XOR(nums2) & \text{if } n \text{ is odd and } m \text{ is even} \\ 0 & \text{if both } m, n \text{ are even} \end{cases}$$

0 0 1 0
0 0 1 0

a ^ a = 0

nums1 = [2, 3] → xor1 = 2 ^ 3 = 1
nums2 = [5, 7, 1] → xor2 = 5 ^ 7 ^ 1 = 3
n = 2 (even), m = 3 (odd)

result = xor1 ^ 0 = 1

- all pairs are: (2^5)^(2^7)^(2^1)^(3^5)(3^7)^(3^1)
  as u observe:5 7 and 1 are appeared even times hence they cancel out. (this is bcz length of n is 2)
  Hence remaining ans will be 2^3 which is 1.

- Since m is odd, XOR of xor1 repeated m times equals xor1 itself.
- Since n is even, XOR of xor2 repeated n times cancels out to 0.
- XORing with zero leaves xor1 unchanged.

2 ^ 2 ^ 2 ^ 3 ^ 3 ^ 3
0 ^ 2 ^ 3 ^ 3 ^ 3
2 ^ 3 ^ 3 ^ 3
2 ^ 3

Logic

Input: n, m
Input: nums1[0...n-1]
Input: nums2[0...m-1]

Initialize xor1 = 0
For i from 0 to n-1:
    xor1 = xor1 XOR nums1[i]

Initialize xor2 = 0
For j from 0 to m-1:
    xor2 = xor2 XOR nums2[j]

Initialize result = 0

If m is odd:
    result = result XOR xor1

If n is odd:
    result = result XOR xor2

Output result

O(n+m) time

even $[2, 5]$
$[7, 8]$

$(2,7), (2,8), (5,7), (5,8)$

$(2 \wedge 7) \wedge (2 \wedge 8) \wedge (5 \wedge 7) \wedge (5 \wedge 8)$

$= 0$

Longest consecutive 1's

- Every iteration **shrinks the group of 1s** by one from the **right**
- So each step removes **1 bit from the group**, hence we count iterations = length of the longest streak

Let's say n = 29
→ Binary: 11101
Longest consecutive 1s = 3 (from left)

Naive:

```
      while (n > 0) {
            if ((n & 1) == 1) {
                curr_len++;
                if (curr_len > max_len)
                    max_len = curr_len;
            } else {
                curr_len = 0;
            }
            n = n >> 1;
        }

        System.out.println(max_len);
```

Time Complexity: O(log n)
Space Complexity: O(1)

```
   while (n != 0) {
            n = n & (n << 1);
            count++;
        }
```

Time Complexity:O(k) where k is length of max 1s

- n = n & (n << 1) shrinks each group of 1s by 1 from the right
- Number of times we can repeat this until n == 0 is equal to the length of the longest streak
- It’s not removing the 0, but actually trimming 1s from each group

Problem:

Input:

Output:

Naive Approach:

Optimized Approach:

Problem:

Input:

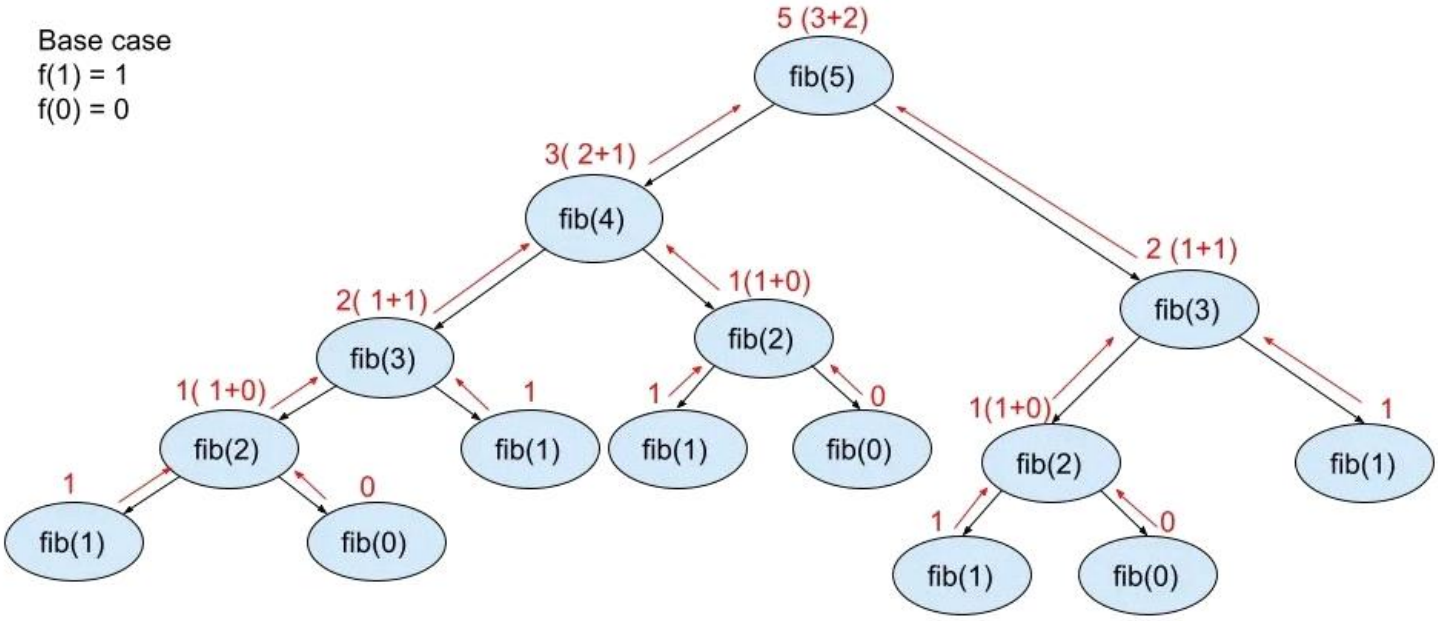
Output:

Naive Approach:

Optimized Approach:

fibanacci series recursion

```
public class FibonacciRecursive {
    public static int fibonacci(int n) {
        if (n == 0)
            return 0;
        else if (n == 1)
            return 1;
        else
            return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        int n = 10;

        System.out.println("Fibonacci series up to " + n + " terms:");
        for (int i = 0; i < n; i++) {
            System.out.print(fibonacci(i) + " ");
        }
    }
}
```

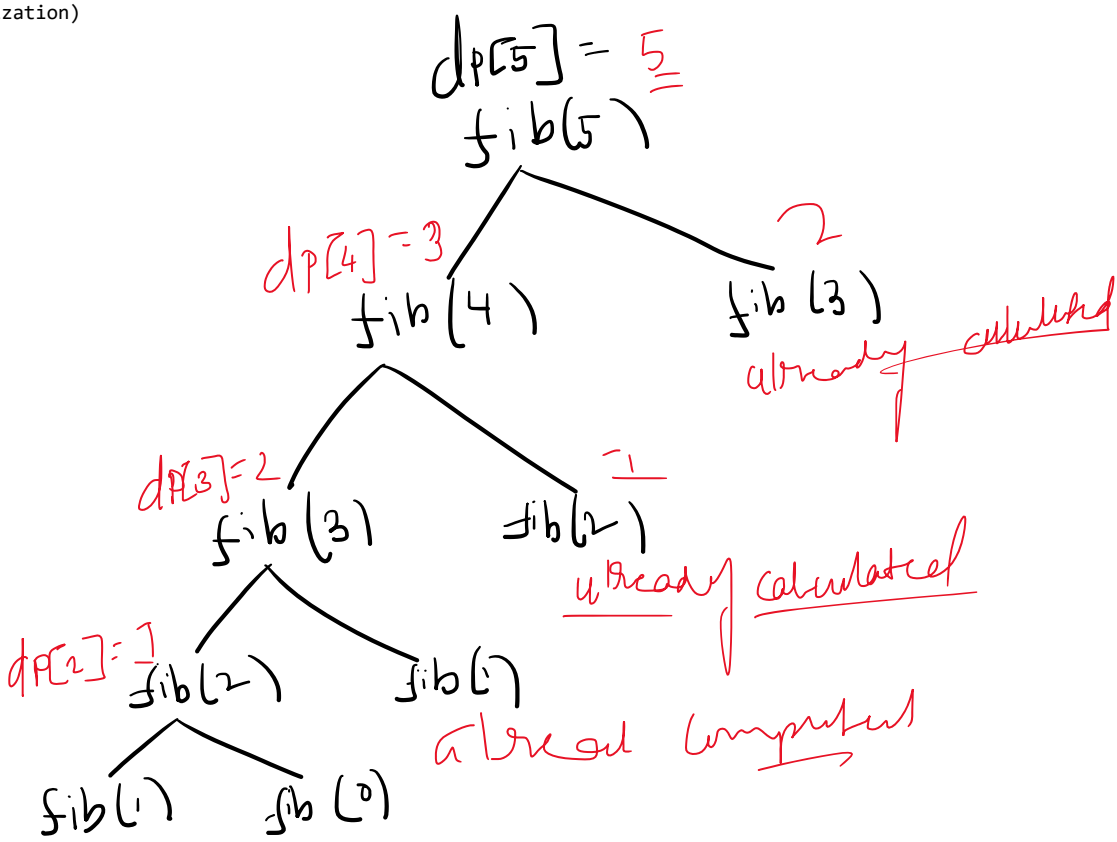


Exponential O(2^n)

Approach	Time Complexity	Space Complexity
Naive Recursion	O(2^n)	O(n)
Recursion + Memoization	O(n)	O(n)
Iterative DP (Bottom-Up)	O(n)	O(n)
Iterative with Constant Space	O(n)	O(1)

fibanacci series (top down dp memoization)

24 May 2025 14:49




```
public class fib_memo {
    static int[] dp;

    public static int fib(int n) {
        if (n <= 1) return n;
        if (dp[n] != -1)
            return dp[n];
        dp[n] = fib(n - 1) + fib(n - 2);
        return dp[n];
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        dp = new int[n + 1];
        Arrays.fill(dp, -1);
        System.out.println(fib(n));
    }
}
```

n	dp[n]
0	0
1	1
2	1
3	2
4	3
5	5

Final dp[] after computation for n = 5:

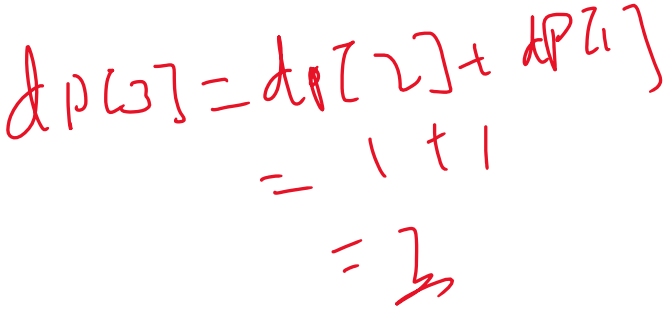
Time Complexity:
O(n)
Why?
- Each fib(n) is computed **only once**, and stored in dp[].
- Space complexity: O(n) for the array and call stack.

- You store each Fibonacci number in dp[n] once.
- So each fib(k) from 0 to n is computed **only once**.

fibanacci series (tabulation bottom up)

24 May 2025 14:50

```
import java.util.*;

public class fib_tab {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] dp = new int[n + 1];

        if (n > 0) dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        System.out.println(dp[n]);
    }
}
```



Time Complexity:
O(n)
Space Complexity:
O(n)

fibancci series space optimized version of tabulation

24 May 2025 14:52

```
import java.util.*;

public class fib_best {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n == 0) {
            System.out.println(0);
            return;
        }
        int a = 0, b = 1;
        for (int i = 2; i <= n; i++) {
            int sum = a + b;
            a = b;
            b = sum;
        }
        System.out.println(b);
    }
}
```

Time Complexity:
O(n)
Space Complexity:
O(1)

0/1 knapsack problem using recursion

24 May 2025 09:12

Given **n** items where each item has some weight and profit associated with it and also given a bag with capacity **W**, [i.e., the bag can hold at most **W** weight in it]. The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.

Note: The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Input: *W = 4, profit[] = [1, 2, 3], weight[] = [4, 5, 1]*
Output: *3*

Explanation: *There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.*

1. **Base Case**: If n == 0 or W == 0, return 0.
2. **If current item can't be included** (weight > current capacity):
    - Skip it: call recursively with n - 1
3. **Else**:
    - Include current item: add its value and reduce capacity and recursively call n-1
    - Exclude current item and directly call n-1
    - Return max of Include and Exclude calls

```
public static int knapsack(int n, int W, int[] wt, int[] val) {
       if(n == 0 || W == 0)
           return 0;
       if(wt[n - 1] > W)
           return knapsack(n - 1, W, wt, val);
       else {
           int include = val[n - 1] + knapsack(n - 1, W - wt[n - 1], wt, val);
           int exclude = knapsack(n - 1, W, wt, val);
           return Math.max(include, exclude);
       }
   }
```

How it works:

- At every item, you make **2 choices**:
    - Include the item
    - Exclude the item
- Then **recurse** on remaining items and updated capacity

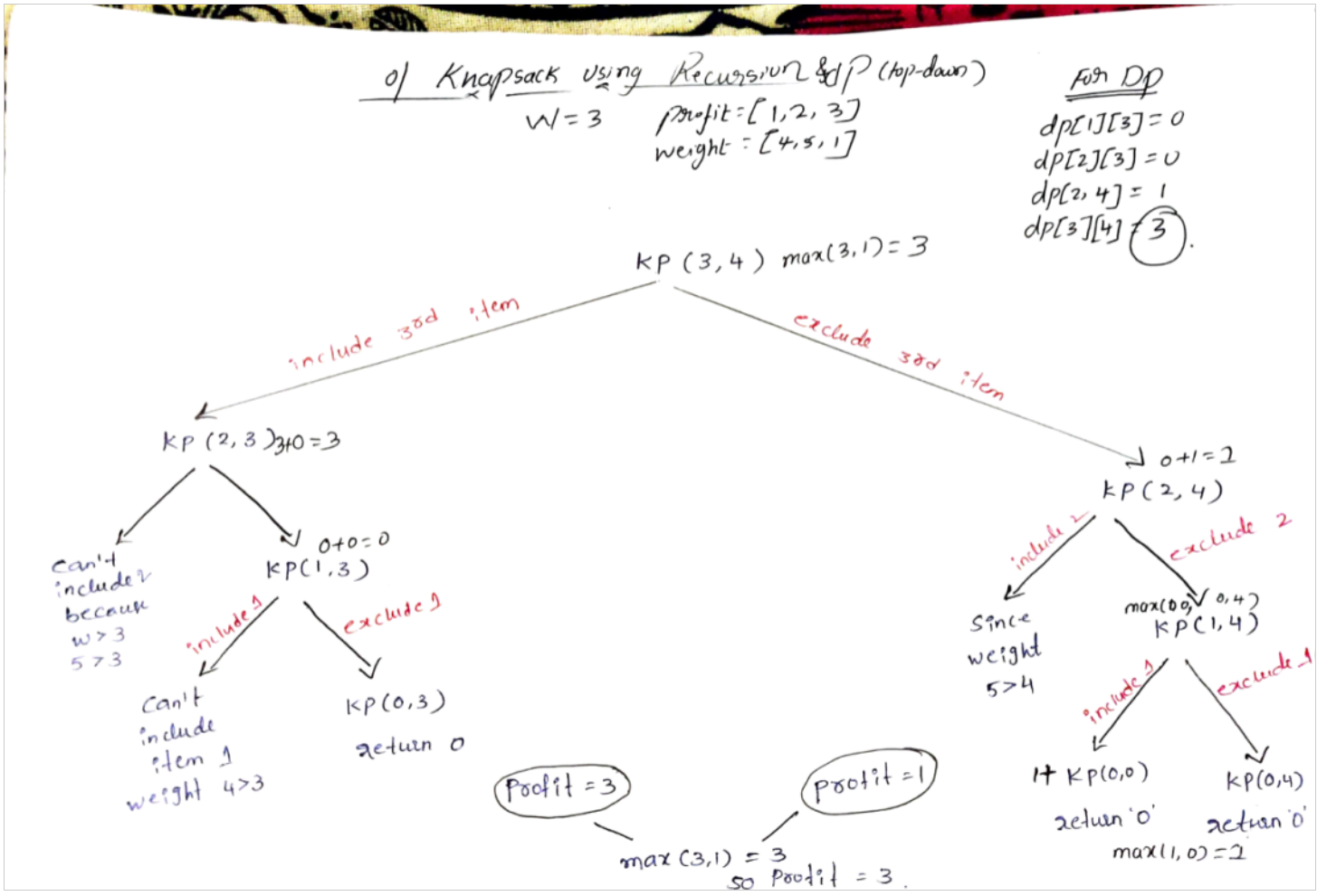
🔁 **What happens:**

- Subproblems **repeat multiple times**
- No memory of already computed values

for example:
For n=3, W=4, you may call:

```
knapsack(3, 4)
  ↙           ↘
(2, 3)       (2, 4)
  ↙   ↘        ↙   ↘
(1,3)(1,2)  (1,4)(1,3)
```

➡️ Subproblem (1,3) is solved **twice**, wasting time.


0/ Knapsack using Recursion & dp (top-down)
W = 3
profit = [1, 2, 3]
weight = [4, 5, 1]
For DP
dp[1][3] = 0
dp[2][3] = 0
dp[2, 4] = 1
dp[3][4] = 3
KP (3, 4) max(3, 1) = 3
include 3rd item
exclude 3rd item
KP (2, 3) 3+0 = 3
0+1 = 1
KP (2, 4)
Can't include 2 because w > 3 5 > 3
0+0 = 0
KP(1, 3)
include 2
exclude 2
Since weight 5 > 4
max(0, 0, 0, 4)
KP(1, 4)
include 1
exclude 1
Can't include item 1 weight 4 > 3
KP(0, 3)
return 0
If KP(0,0)
return '0'
KP(0,4)
return '0'
max(1, 0) = 1
Profit = 3
Profit = 1
max (3, 1) = 3
so Profit = 3.

Subset with sum K

Problem: Given n integers and an integer k, determine if there exists a subset of the given integers whose sum is exactly k.

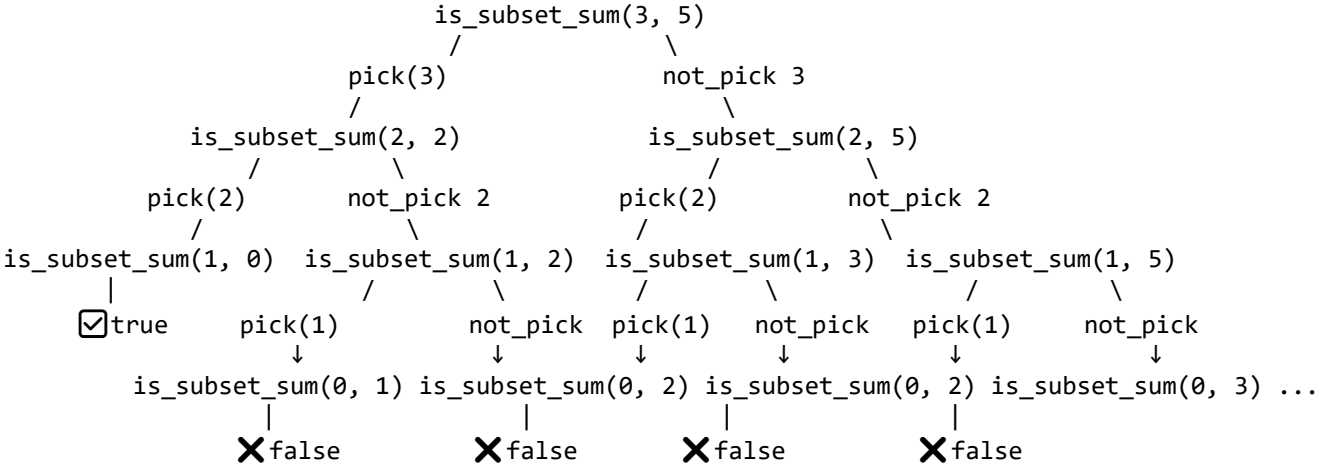
Input:
n = 4
arr = [3, 1, 5, 9]
k = 9
Output: true
Because subset [3, 1, 5] or [9] sums to 9.

n = 3
arr = [1, 2, 3]
k = 5
Output: true

Recursive Approach Using Recursion – O(2^n) Time and O(n) Space

```
boolean is_subset_sum(int[] arr, int n, int sum)
        if (sum == 0) return true;
        if (n == 0) return false;           //or arr[0]==sum..here n is index
         boolean nt_pick=is_subset_sum(arr, n - 1, sum);
        if(arr[n-1]<=k)
         boolean pick=is_subset_sum(arr, n - 1, sum - arr[n-1]);
         return pick||nt_pick;
```

recusive decision tree (input= 1 2 3 and sum=5)



other variations

- count of subsets: for this just return pick+not_pick

- print the actual subset
- Variants like: , min number of elements summing to k.

subset sum using memoization Top-Down DP O(sum*n) Time and O(sum*n) Space

Optimized Approach using memoization
In computing, "memoize" means to store and reuse the results of expensive function calls.
(Top-Down DP)

- **Memoization = storing results of expensive recursive calls so that we don't recompute them.**
- **It uses a recursive (function call) approach, and stores the result in a dp array or map.**
  - Use a 2D boolean DP array dp[n][k+1] to store results of subproblems.
  - dp[i][sum] indicates whether a subset of arr[0..i] can sum to sum.
  - Avoid recomputation by storing results.

Time Complexity: O(n*k*)
Space Complexity: *O(n*k) for dp + O(n) recursion stack

*When implementing a **recursive approach** to solve the subset sum problem, we observe that many subproblems are computed multiple times. For instance, when computing **isSubsetSum(arr, sum)**, where arr[] = {2,3,1,1} and sum = 4 we might need to compute **isSubsetSum(1,3)** multiple times.*

- The recursive solution involves changing two parameters: the **current index** in the array (n) and the **current target sum** (sum). We need to track both parameters, so we create a **2D array** of size **(n) x (sum+1)** because the value of n will be in the range **[0, n]** and sum will be in the range **[0, sum].**
- We initialize the 2D array with -1 to indicate that no subproblems have been computed yet.
- We check if the value at **dp[n][sum]** is -1. If it is, we proceed to compute the result. otherwise, we return the stored result.

Idea:
Store results of already solved subproblems in a DP table:
- *Use a 2D array dp[n][k+1]*
- *dp[i][sum] means: Is there a subset of arr[0..i] that adds up to sum?*

Before computing a state, check:

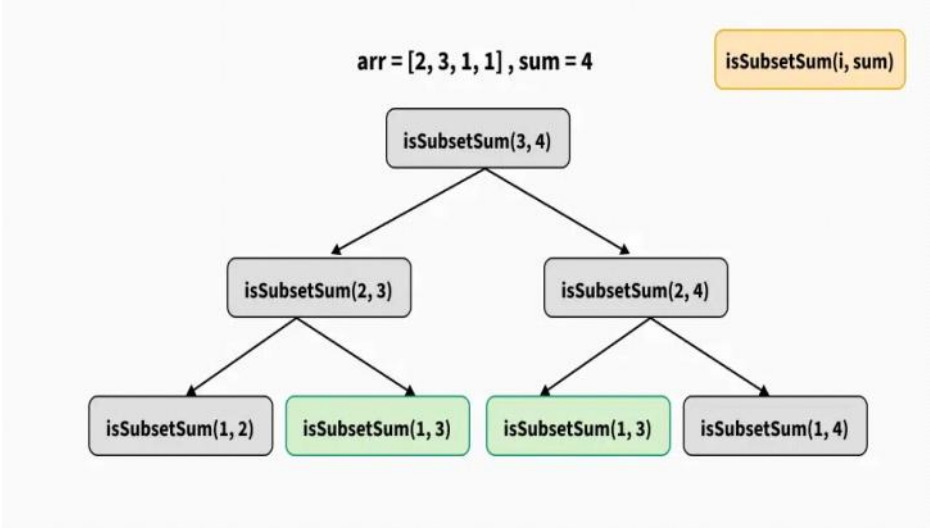
if (dp[i][sum] != -1) return dp[i][sum] == 1;

arr = [1, 2, 3], k = 5
Possible i (indices): 0, 1, 2 → 3 values
Possible sum: 0 to 5 → 6 values
*Total unique states = 3 * 6 = 18*
*So the recursive function will run **maximum 18 times***

*arr[] = {2,3,1,1} and sum = 4 we might need to compute **isSubsetSum(1,3)** multiple times.*
Key reuse: for example:
- ***subset_sum(1, 3) is first computed using base case subset_sum(0, 0)***
- ***Its result (dp[1][3] = 1) is stored***

- *Later, inside subset_sum(2, 3) and again in subset_sum(2, 4), the result is reused from dp[1][3]*

This is the core benefit of memoization: avoiding duplicate computations by remembering earlier answers.




```
subset_sum(3, 4)
├── not_pick → subset_sum(2, 4)
│   ├── not_pick → subset_sum(1, 4)
│   │   ├── not_pick → subset_sum(0, 4) = 0 → dp[0][4] = 0
│   │   └── pick → subset_sum(0, 1) = 0 → dp[0][1] = 0
│   │  → dp[1][4] = 0
│   └── pick → subset_sum(1, 3)
│       ├── not_pick → subset_sum(0, 3) = 0 → dp[0][3] = 0
│       └── pick → subset_sum(0, 0) = 1 → ☑ base → dp[0][0] = 1
│      → dp[1][3] = 1
│  → dp[2][4] = 1 ← (from dp[1][3])
├── pick → subset_sum(2, 3)
│   ├── not_pick → subset_sum(1, 3) = ☑ reused → dp[1][3] = 1
│   └── pick → subset_sum(1, 2)
│       ├── not_pick → subset_sum(0, 2) = 1 → dp[0][2] = 1
│       └── pick → subset_sum(0, 1) = 0 → dp[0][1] = 0
│      → dp[1][2] = 1
│  → dp[2][3] = 1
→ dp[3][4] = 1 (from dp[2][4] or dp[2][3])
```

stack function calls

Step	Function Call	Condition / Explanation	Result	dp[i][k] Updated
1	subset_sum(0, 0)	Base case (k=0)	true	dp[0][0] = 1
2	subset_sum(0, 1)	arr[0]=2 ≠ 1 → false	false	dp[0][1] = 0
3	subset_sum(0, 2)	arr[0]=2 == 2 → true	true	dp[0][2] = 1
4	subset_sum(0, 3)	arr[0]=2 ≠ 3 → false	false	dp[0][3] = 0
5	subset_sum(0, 4)	arr[0]=2 ≠ 4 → false	false	dp[0][4] = 0
6	subset_sum(1, 1)	not_pick=0, pick from subset_sum(0, -2) invalid	false	dp[1][1] = 0

```
7    subset_sum(1, 2) not_pick = dp[0][2] = 1, pick = invalid        true   dp[1][2] = 1
8    subset_sum(1, 3) pick from subset_sum(0, 0) = 1                 true   dp[1][3] = 1
9    subset_sum(1, 4) not_pick = 0, pick = dp[0][1] = 0              false  dp[1][4] = 0
10   subset_sum(2, 3) not_pick = dp[1][3] = 1, pick = dp[1][2] = 1   true   dp[2][3] = 1
11   subset_sum(2, 4) not_pick = dp[1][4] = 0, pick = dp[1][3] = 1   true   dp[2][4] = 1
12   subset_sum(3, 4) not_pick = dp[2][4] = 1, pick = dp[2][3] = 1   true   dp[3][4] = 1
```

initial call subset(n-1,k) where k is the sum

```
Function subset_sum(n, k):
    If k == 0:
        return True

    If n == 0:
        return arr[0] == k (or false)

    If dp[i][k]!=-1:
        return dp[i][k]

    not_pick = subset_sum(n - 1, k)

    pick = False
    If arr[i] <= k:
        pick = subset_sum(n - 1, k - arr[n])

    dp[n][k] = pick || not_pick

    return dp[i][k]==1
```

the **sum k can range from 0 to k** inclusive.
This means we need to store results for all these values:

k = 0, 1, 2, ..., k
=> total (k + 1) values
If you only create dp[n][k], then the **index k is out of bounds**, because array indices go from 0 to k - 1.

In top-down recursion:

- You start with the **last index i = n - 1**
- You recurse down to **index 0**

- So valid i values are 0 to n - 1 → total n rows
- Therefore, dp[n][...] (0-based indexing) is **exact**.

For input arr = [2, 3, 1, 1], k = 4, we call:

```
subset_sum(3, 4)   ← this is the top-level problem
   ↓
subset_sum(2, 4)
   ↓
subset_sum(1, 4)
   ↓
subset_sum(0, 4)
```

We **start at the top** (i = n - 1, k = sum) and go downward into subproblems (i - 1, smaller k) — hence called **top-down**.

for counting number of such subsets.

```
dp[n][k] = pick + not_pick;
return dp[n][k];
```

subset sum with K Tabulation approach(Bottom Up)

Total operations:
- For each i (0 to n-1): **n iterations**
- For each j (0 to k): **k+1 iterations**
- Total = roughly n * k operations.

```
boolean[][] dp = new boolean[n][k + 1];

 for (int i = 0; i < n; i++) dp[i][0] = true
  // for all other sums, dp[0][sum] remains false by default (boolean array)
        if (arr[0] <= k) dp[0][arr[0]] = true
        for (int i = 1; i < n; i++)
            for (int sum = 1; sum <= k; sum++)
                boolean not_pick = dp[i - 1][sum]
                boolean pick = false
                if (arr[i] <= sum)
                    pick = dp[i - 1][sum - arr[i]]

                dp[i][sum] = pick || not_pick;


        return dp[n - 1][k]
```

This line sets dp[i][0] = true for all i because:
- The **subset sum problem** asks: "Is there a subset of the first i elements that sums to j?"
- When j = 0 (sum = 0), the answer is **always YES** regardless of which elements you consider.

Why?
- Because the **empty subset** (no elements chosen) always has sum 0.

So for **any i**, it is always possible to form sum 0 by choosing no elements.

What does filling the table signify in each iteration?
- Each iteration considers one more element of the array.
- For every possible sum from 0 to k, it answers:
  "Can I form this sum using elements up to index i?"
- It **builds up knowledge** from smaller subproblems (previous rows) to solve bigger subproblems (current row).
- By the time the table is fully filled, it contains answers
- for **all subsets and all sums** up to k.

Quick visual for example arr=[2,3,1,1], k=4

i \ sum	0	1	2	3	4
0 (2)	T	F	T	F	F
1 (3)	T	F	T	T	T
2 (1)	T	T	T	T	T
3 (1)	T	T	T	T	T

In the DP table for the **subset sum problem**, each cell dp[i][sum] represents:
Is there a subset among the first i+1 elements (arr[0..i]) that sums up to sum?

dp[2][1] = true — What does this mean?
Using the first 3 elements of the array (arr[0], arr[1], and arr[2]), we can form a subset whose sum is exactly

1.

dp[1][2] = true
It means:
Using the first two elements of the array (arr[0], arr[1]), it is possible to form a subset whose sum is exactly 2.

📘 **Given:**
Array = [2, 3, 1, 1]
Index 1 means second element → arr[1] = 3
So we are checking whether using only [2, 3], we can form sum = 2.

Initial dp table after base setup (T = true, F = false):

i \ sum	0	1	2	3	4
0	T	F	T	F	F
1	T	F	F	F	F
2	T	F	F	F	F
3	T	F	F	F	F

Loop trace:
For i = 1 (element = 3):

- sum = 1:
  - not_pick = dp[0][1] = F
  - pick: arr[1] = 3 > 1 → no pick → F
  - dp[1][1] = F || F = F
- sum = 2:
  - not_pick = dp[0][2] = T
  - pick: arr[1] = 3 > 2 → no pick → F
  - dp[1][2] = T || F = T
- sum = 3:
  - not_pick = dp[0][3] = F
  - pick: arr[1] = 3 ≤ 3 → dp[0][3-3]=dp[0][0] = T
  - dp[1][3] = F || T = T
- sum = 4:
  - not_pick = dp[0][4] = F
  - pick: arr[1] = 3 ≤ 4 → dp[0][4-3] = dp[0][1] = F
  - dp[1][4] = F || F = F

dp after i=1:

i \ sum	0	1	2	3	4
0	T	F	T	F	F
1	T	F	T	T	F
2	T	F	F	F	F
3	T	F	F	F	F

Expression	Meaning
dp[i - 1][sum]	Can I make sum **without** using arr[i] (not pick)?
dp[i - 1][sum - arr[i]]	Can I make sum **by including** arr[i]? (pick)

Suppose:
arr = [2, 3, 1, 1], sum = 4, you're at i = 2 → arr[2] = 1
You want dp[2][4] = ?

Can we form sum = 4 using the first 3 elements: [2, 3, 1]?

1. not_pick:
→ dp[1][4]
→ Did we already form 4 using just [2, 3]?

2. pick:
→ Only valid if arr[2] <= 4 → ☑ true
→ We check: dp[1][4 - arr[2]] = dp[1][3]
So we need:
  - dp[1][4] = ?
  - dp[1][3] = ?

- dp[1][4] (using [2,3]) is false because neither 2 nor 3 nor 2+3=5 equals 4.
- dp[1][3] (using [2,3]) is true because subset [3] sums to 3.

So dp[2][4] = false || true = true

- At dp[2][4], we get true because we can pick arr[2] = 1 and combine with sum 3 from previous elements (subset [3]) to get sum 4.
- So the subset is [3, 1] — from first 3 elements.

Subset sum with K space optimized approach

Using Space Optimized DP – O(sum*n) Time and O(sum) Space

- Instead of a 2D dp[n][k+1] table, we use a 1D boolean array dp[k+1].
- dp[sum] means:
  Is there a subset (from elements processed so far) with sum sum?
- We update this array for each element.
- We update sums in **reverse order** (from k down to arr[i]) to avoid using the updated values of the same iteration.

Imagine we go **forward** from sum = arr[i] up to k. When we update dp[sum] = dp[sum] || dp[sum - arr[i]], the value of dp[sum - arr[i]] might have just been updated in the current iteration **using the same element arr[i]**. That means the current element could be counted multiple times for the same subset sum, which is wrong (we want each element counted once per subset).

To **avoid reusing the same element multiple times** in one iteration
First, arr[0] = 2:
→ dp[2] = true
Now, arr[0] = 2 again (due to forward iteration):
→ dp[4] = dp[2] = true
This falsely concludes:
→ dp[4] = true ⇒ subset {2, 2} exists
But 2 appears **only once** in input!
That's wrong. The subset {2, 2} isn't valid in 0/1 case.

- That condition checks if a subset with sum (sum - arr[i]) already exists, then by adding the current element arr[i], a subset with sum sum can also be formed.
-
- Explanation:
  - dp[sum - arr[i]] == true means:
    There exists a subset among the elements processed so far whose sum equals sum - arr[i].
  - If you add the current element arr[i] to that subset, the sum becomes (sum - arr[i]) + arr[i] = sum.
  - So, by including arr[i], we can form a subset with sum sum.

Suppose:
- arr[i] = 3
- We want to check if a subset sum of sum = 5 is possible.

If dp[5 - 3] = dp[2] is true (meaning subset with sum 2 exists), then by adding 3, subset sum 5 can be formed, so dp[5] = true.

```
boolean[] dp = new boolean[k+1];
        dp[0] = true;
dp[0] = true
It represents:
An empty subset can always give sum = 0.
So it's always true from the start.

        if (arr[0] <= k) dp[arr[0]] = true;
        for (int i=1; i<n; i++) {
            for (int sum=k; sum>=arr[i]; sum--) {
                if (dp[sum - arr[i]]) {
                    dp[sum] = true;
                }
            }
        }
```

```
        System.out.println(dp[k]);
```

arr = [2,3,1,1] and k = 4

Index (Sum)	**0**	**1**	**2**	**3**	**4**
dp (start)	T	F	F	F	F

Index (Sum)	**0**	**1**	**2**	**3**	**4**
dp after arr[0]=2	T	F	T	F	F

Step 2: Process arr[1] = 3
Loop from 4 to 3:
- dp[3] = dp[0] = true
- So update dp[3] = true

Index (Sum)	**0**	**1**	**2**	**3**	**4**
dp after arr[1]=3	T	F	T	T	F

Step 3: Process arr[2] = 1
Loop from 4 to 1:
- dp[4] = dp[3] = true
- dp[3] = dp[2] = true (already true)
- dp[1] = dp[0] = true

Index (Sum)	**0**	**1**	**2**	**3**	**4**
dp after arr[2]=1	T	T	T	T	T

☑ We found dp[4] = true ⇒ subset exists.

similarly step 4: arr[3]=1

Final DP Table

Sum	**0**	**1**	**2**	**3**	**4**
dp	T	T	T	T	T

Meaning:

You can form all sums from 0 to 4 using subsets of {2, 3, 1, 1}.
Some subsets:
- Sum 1 → {1}
- Sum 2 → {2}
- Sum 3 → {2,1} or {3}
- Sum 4 → {3,1} or {2,1,1}

String Partitioning

Given a string s and a dictionary wordDict, determine if s can be segmented into a space-separated sequence of one or more dictionary words.

Input: s = "applepenapple", wordDict = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.
Example 3:
Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false

Approach 1: Recursion without memoization (O(2^n))

◇ **Idea:**

At every position i in the string s, try **all possible prefixes** s[0...j] and:

- If prefix is in dictionary ☑, recursively check the **remaining suffix** s[j+1:].

Input:
s = "applepenapple"
wordDict = ["apple", "pen"]

☑ **Goal:**
Check if "applepenapple" can be segmented using words from the dictionary ["apple", "pen"].

☑ **Dictionary:**
dict = {"apple", "pen"}

🔍 **RECURSION TRACE:**
We will recursively try all **prefixes** of the current string and proceed only when the prefix is in the dictionary.

Step 1: Call with s = "applepenapple"
Check all prefixes:

- "a" ✗
- "ap" ✗
- "app" ✗
- "appl" ✗
- "apple" ☑ → in dictionary → Recurse on "penapple"

Step 2: Call with s = "penapple"
Check all prefixes:

- "p" ✗
- "pe" ✗
- "pen" ☑ → in dictionary → Recurse on "apple"

Step 3: Call with s = "apple"
Check all prefixes:

- "a" ✗
- "ap" ✗
- "app" ✗
- "appl" ✗
- "apple" ☑ → in dictionary → Recurse on ""

Step 4: Call with s = "" → Reached empty string → return true
Now we backtrack all the way up:

- Step 3 returns true
- Step 2 returns true
- Step 1 returns true

FINAL RESULT: true
We could segment:
"apple" + "pen" + "apple"

CALL STACK:

```
word_break("applepenapple")
  └─ prefix = "apple" ☑
      └─ word_break("penapple")
          └─ prefix = "pen" ☑
              └─ word_break("apple")
                  └─ prefix = "apple" ☑
                      └─ word_break("") → ☑ base case
```

```
public static boolean word_break(String s, Set<String> dict)
        if (s.length() == 0)
            return true
        for (int i = 1; i <= s.length(); i++)
            String prefix = s.substring(0, i)
            if (dict.contains(prefix))
                if (word_break(s.substring(i), dict))
                    return true


        return false
```

Approach 2 using DP (same time complexity)

We use a dp[i] array where:

- dp[i] = true means s[0..i-1] can be segmented using words from the dictionary.

s = "applepenapple"
wordDict = ["apple", "pen"]

Setup:
s.length() = 13

dp array size = 14 (0 to 13)

dp[0] = true (empty string can be segmented)

dp[i] means s[0..i-1] can be segmented into dictionary words.

```
Initially:
dp = [T, F, F, F, F, F, F, F, F, F, F, F, F, F]
Index: 0  1  2  3  4  5  6  7  8  9 10 11 12 13
Iteration over i from 1 to 13:
i = 1
Check j = 0: s[0..0] = "a" not in dict → dp[1] = false

i = 2
j = 0: "ap" no

j = 1: "p" no → dp[2] = false

i = 3
j=0: "app" no

j=1: "pp" no

j=2: "p" no → dp[3] = false

i = 4
j=0: "appl" no

j=1: "ppl" no

j=2: "pl" no

j=3: "l" no → dp[4] = false

i = 5
j=0: "apple" → YES, in dict and dp[0] = true
→ dp[5] = true, break inner loop

dp = [T, F, F, F, F, T, F, F, F, F, F, F, F, F]
i = 6
j=0: "applep" no

j=1: "pplep" no

j=2: "plep" no

j=3: "lep" no

j=4: "ep" no

j=5: "p" no → dp[6] = false

i = 7
similarly check substrings ending at 7 → no match
dp[7] = false

i = 8
j=0 to 4 no matches

j=5: s[5..7] = "pen" YES in dict, dp[5] = true
→ dp[8] = true, break inner loop

dp = [T, F, F, F, F, T, F, F, T, F, F, F, F, F]
i = 9
check substrings ending at 9 → no matches
dp[9] = false

i = 10
no matches
dp[10] = false

i = 11
no matches
dp[11] = false

i = 12
```

```
no matches
dp[12] = false

i = 13
j=0 to 7 no matches

j=8: s[8..12] = "apple" in dict, dp[8] = true → YES
→ dp[13] = true


dp = [T, F, F, F, F, T, F, F, T, F, F, F, F, T]
Final dp array:
r
Index:  0  1  2  3  4  5  6  7  8  9 10 11 12 13
dp[] = [T, F, F, F, F, T, F, F, T, F, F, F, F, T]
dp[13] = true means the whole string "applepenapple" can be segmented as dictionary words.

Summary:
"apple" at indices [0..4]

"pen" at indices [5..7]

"apple" at indices [8..12]
```

```
  Function String_partitioning(s: string, dict: set of words):
      n = length of s

      dp = array of boolean of size (n + 1)
      dp[0] = true   // base case

      for i from 1 to n:
          for j from 0 to i:
              if dp[j] == 1 AND substring s[j to i-1] in dict
                  dp[i] = true
                  break
      return dp[n]
```

Minimum number of partitions.

Given a string s
find the minimum number of partitions of a string s such that each partition is a word from the dictionary,
You only want the minimum number of valid words to cover the entire string.

s = "pineapplepenapple"
dict = ["pine", "apple", "pen", "applepen", "pineapple"]
Partitions:

"pine" + "apple" + "pen" + "apple" → 4 partitions

"pineapple" + "pen" + "apple" → 3 partitions (minimum)

"pine" + "applepen" + "apple" → 3 partitions (minimum)

Minimum partitions = 3, but some partitions have 4 parts.

- len = 13 (length of "applepenapple")

We fill dp[] from i=1 to 13.

i = 1:
Check substrings s[j..i-1]:
- j=0: s[0..0] = "a" → not in dict → no update → dp[1] = ∞

i = 2:
- j=0: "ap" → no
- j=1: "p" → no

  dp[2] = ∞

i = 3:
- j=0: "app" → no
- j=1: "pp" → no
- j=2: "p" → no

  dp[3] = ∞

i = 4:
- j=0: "appl" → no
- j=1: "ppl" → no
- j=2: "pl" → no
- j=3: "l" → no

  dp[4] = ∞

i = 5:
- j=0: "apple" → yes in dict and dp[0] = 0

  → dp[5] = min(∞, 0 + 1) = 1
- other j's ignored since dp[j] is ∞

i = 6:
- j=0: "applep" → no
- j=1..5 substrings no match

  dp[6] = ∞

i = 7:
- j=0..6 no substring in dict

  dp[7] = ∞

i = 8:
- j=0..7 no substring in dict

  dp[8] = ∞

i = 9:
- j=0..4 no

- j=5: "pen" → yes in dict and dp[5] = 1
  → dp[9] = min(∞, 1 + 1) = 2

i = 10..12:
No valid substrings in dict
dp[10] = ∞, dp[11] = ∞, dp[12] = ∞

i = 13:
- j=0..8 no
- j=9: "apple" → yes and dp[9] = 2
  → dp[13] = min(∞, 2 + 1) = 3

- dp[5] = 1 means minimum 1 partition for "apple"
- dp[9] = 2 means minimum 2 partitions for "applepen"
- dp[13] = 3 means minimum 3 partitions for full string
  So the minimum partitions to segment "applepenapple" into dictionary words = 3

```
int len = s.length();
        int[] dp = new int[len + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;
        for (int i = 1; i <= len; i++) {
            for (int j = 0; j < i; j++) {
                String sub = s.substring(j, i);
                if (dp[j] != Integer.MAX_VALUE && dict.contains(sub)) {
                    dp[i] = Math.min(dp[i], dp[j] + 1);
                }
            }
        }
        if (dp[len] == Integer.MAX_VALUE) {
            System.out.println(-1);
        } else {
            System.out.println(dp[len]);
        }
```

Number of ways to Partition String

Given a string s and a set of dictionary words dict, Count how many ways the string can be broken into valid dictionary words.
Note: all segmentations must start from the beginning of the string s.
So it is mandatory to begin matching from index 0 of the string and proceed left to right

Input:
s = "applepenapple"
dict = ["apple", "pen"]
Output:
1
Because only one valid way: "apple pen apple"

Input:
s = "pineapplepenapple"
dict = ["apple", "pen", "applepen", "pine", "pineapple"]
Output:
3
Explanation:
The string "pineapplepenapple" can be segmented in 3 ways:

"pine" + "apple" + "pen" + "apple"

"pineapple" + "pen" + "apple"

"pine" + "applepen" + "apple"

```
int len = s.length()
int[] dp = new int[len + 1]
dp[0] = 1
```

```
for (int i = 1; i <= len; i++)
    for (int j = 0; j < i; j++)
        String sub = s.substring(j, i)
        if (dict.contains(sub)) {
            dp[i] += dp[j]
```

```
s = "pineapplepenapple"
Length = 17
Initialize:

dp[0] = 1 (empty string)

dp[1] to dp[17] = 0
Now iterate:
At i = 4:
 • s[0..4] = "pine" is in dict
   → dp[4] += dp[0] = 1
   → dp[4] = 1
At i = 9:
 • s[4..9] = "apple" is in dict
   → dp[9] += dp[4] = 1
   → dp[9] = 1
 • s[0..9] = "pineapple" is in dict
   → dp[9] += dp[0] = 1
   → dp[9] = 2
At i = 12:
 • s[9..12] = "pen" is in dict
```

→ dp[12] += dp[9] = 2
→ dp[12] = 2

At i = 13:

- s[4..13] = "applepen" is in dict
  → dp[13] += dp[4] = 1
  → dp[13] = 1

At i = 17:

- s[12..17] = "apple" is in dict
  → dp[17] += dp[12] = 2
  → dp[17] = 2
- s[13..17] = "apple" is in dict
  → dp[17] += dp[13] = 1
  → dp[17] = 3

☑ **Final DP Table**

Index (i)	dp[i]	s[0..i-1]
0	1	""
4	1	"pine"
9	2	"pineapple"
12	2	"pineapplepen"
13	1	"pineapplepena"
17	3	"pineapplepenapple"

→ So, dp[17] = 3 = **number of ways to partition.**

☑ **Time and Space Complexity**

- **Time:** O(n^2) due to nested loops over substring generation
- **Space:** O(n) for dp[] array

Why is dp[12] = 2 for:

s = "pineapplepenapple"
dict = ["apple", "pen", "applepen", "pine", "pineapple"]
We're checking all possible substrings ending at index 12.
Let's try **all valid segmentations** of "pineapplepen":

Option 1:

- "pine" (0..3) + "applepen" (4..11)
- "pine" and "applepen" are both in dict → ☑

→ So one valid segmentation

Option 2:

- "pineapple" (0..8) + "pen" (9..11)
- Both in dict → ☑

→ Second valid segmentation

☑ **Therefore:**

- dp[12] = 2 because:
  - "pine" + "applepen"
  - "pineapple" + "pen"

These are **2 valid ways** to split "pineapplepen"

dp[9] = dp[0] + dp[4] = 1 + 1 = 2
This means there are two ways to segment "pineapple":

"pine" + "apple"

"pineapple"

```
Function String_partitioning(s: string, dict: set of words):
    n = length of s

    dp = array of boolean of size (n + 1)
    dp[0] = true   // base case

    for i from 1 to n:
        for j from 0 to i:
            if dp[j] == 1 AND substring s[j to i-1] in dict
                dp[i] = true
                break
    return dp[n]
```

Min fib terms to sum upto K

Problem:
Given a number **k**, the task is to find the minimum number of Fibonacci terms (including repetitions) whose sum equals **k**.

Note: You may use any Fibonacci number multiple times.

Examples:

Input: *k = 7*

Output: *2*

Explanation: *Possible ways to sum up to 7 using Fibonacci numbers:*

5 + 2 = 7 (2 terms)

3 + 3 + 1 = 7 (3 terms)

2 + 2 + 2 + 1 = 7 (4 terms)

The minimum number of terms is 2, using 5 + 2.

- Use **Fibonacci numbers** (like 1, 2, 3, 5, 8, ...) ≤ k
- Can use any Fibonacci number **multiple times**
- Return the **minimum number of terms** required to form sum = k
- Try all Fibonacci numbers ≤ k. For each, recursively subtract and find the minimum number of terms.
  In each recursive call:
  - You try all valid Fibonacci numbers f such that f ≤ k
  - You do: min = min(min, 1 + solve(k - f))
  - The 1 + is because you've used one term (f)
  - You return the smallest number of terms needed to build k

solve(7)
Fibonacci numbers ≤ 7: [1, 2, 3, 5]
You try:

Choice (f)	Subproblem	Terms	Total Terms
f = 1	solve(6)	?	1 + solve(6)
f = 2	solve(5)	1	1 + 1 = 2 ☑
f = 3	solve(4)	2	1 + 2 = 3
f = 5	solve(2)	1	1 + 1 = 2 ☑

Now among all these, the **minimum** is:

min = min(3, 2, 2) = 2
So solve(7) = 2

```
time complexity: O(2^k)
space: O(k)
```

Backtracking Intro

Backtracking algorithms are like problem-solving strategies that help explore different options to find the best solution. They work by trying out different paths and if one doesn't work, they backtrack and try another until they find the right one. It's like solving a puzzle by testing different pieces until they fit together perfectly.

There are three types of problems in backtracking:

- Decision Problem – In this, we search for a feasible solution.
- Optimization Problem – In this, we search for the best solution.
- Enumeration Problem – In this, we find all feasible solutions.

Recursion does not always need backtracking	Backtracking always uses recursion to solve problems

Recursion usually involves O(n) stack space.	Backtracking can be O(n!) or more depending on constraints.

Print all valid parantheses

Problem:Given a number n, print all combinations of balanced parentheses of lengthn.
Note: A sequence of parentheses is balanced if every opening bracket (has a corresponding closing bracket) in the correct order.

Examples:

Input: *n = 2*

Output: *"( )"*

Explanation: *Only one valid sequence can be formed using 1 pair of parentheses, and it is balanced.*

Input : *n = 4*

Output: *"(( ))", "( )( )"*

Explanation: *Two valid balanced sequences are possible with 2 pairs: one with nested parentheses, and one with pairs side by side.*

Approach 1: Naive – Generate All Sequences → Filter Valid Ones
Idea:
Generate all strings of length n using only (and), then **check each** for balance using a counter.
Time Complexity:
- Total sequences = 2^n
- For each, O(n) to check balance
  ⇒ **O(n * 2^n)**

Step 1: **Generate All Strings of (and) of length n**
Each position has 2 choices:
- **Either '('**
- **Or ')'**

So total strings =
⇒2^n strings

Step 2: For Each String, Check if It Is Balanced
To validate a string (e.g., "(()))("), we must check balance:
- **Use a counter:**
  - **+1 for '('**
  - **-1 for ')'**
- **At any point, if counter < 0 → invalid**
- **At the end, if counter == 0 → valid**

This check takes O(n) time per string.

Hene:
Total Time
- **2^n strings**
- **Each string takes O(n) time to validate**

Space Complexity:
- O(n) for recursion stack and string builder

Let's now **trace the naive approach** for n = 2.

☑ **Step 1: Generate All Strings of Length 2 Using '(' and ')'**
Total possible combinations = 2^2 = 4
All strings of length 2:
1. ((
2. ()

3.)(
4.))

there is only one valid path= ()

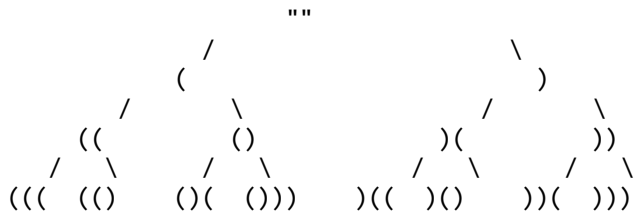
☑ **Step 2: Check for Balance**
We use a **counter method**:
- +1 for '(', -1 for ')'
- If counter < 0 at any point → invalid
- At end, if counter == 0 → valid

Note: if closing bracket appears before opening its invalid pair as its not balanced paranthesis.

String	Steps	Valid
((	+1, +1 → 2	No
()	+1, -1 → 0	☑ Yes
)(	-1 → invalid at start	No
))	-1, -2 → invalid	No

TRACE FOR n = 4
Generated Strings:
- ((((– ✗
- (((), (()(– ✗
- (()) – ✓
- ()() – ✓
- All others → invalid due to imbalance or wrong order

Total combinations: 2^4 = 16
All combinations (binary tree traversal):



Psuedo Code Naive Approach

```
if n % 2 != 0
            System.out.println("No valid sequences")
            return
creat empty arraylist result
        generate("", n)
        for String s : result
            System.out.println(s)

    static void generate(String current, int n)
        if current.length() == n
            if is_valid(current)
                result.add(current)
            return
        generate(current + "(", n)
        generate(current + ")", n)

    static boolean is_valid(String str)
        int count = 0
        for int i = 0; i < str.length(); i = i + 1
            char ch = str.charAt(i)
            if ch == '('
                count = count + 1
            else
                count = count - 1
            if count < 0
                return false
        return count == 0
```

Optimized Approach: using recursion with pruning (only generate valid strings)

- Maintain counts of how many '(' and ')' used so far.
- Only add '(' if count of '(' used < n/2.
- Only add ')' if count of ')' used < count of '(' used (to keep balance).
- This avoids generating invalid sequences altogether, improving efficiency drastically.

Explanation:

- max = number of pairs = n/2.
- open = number of '(' used.
- close = number of ')' used.
- Recursively build string, only adding valid parentheses.

Time Complexity:

- O(4^n / sqrt(n)) – Catalan number complexity (number of balanced parentheses strings).
- Much better than naive O(n * 2^n).

Explaination:

- Each balanced parentheses sequence consists of **pairs** of parentheses.
- One **pair** is "()" which has length 2 characters.
- If you want to generate sequences with **n pairs**, total length of the string is always **2 × n** characters.

Why 4?

- Because there are 2 choices for each of the 2n positions: total combinations = $2^{2n} = 4^n$

The total number of balanced parentheses sequences with **n pairs** is the **nth Catalan number**:

```
ArrayList<String> result = new ArrayList

    if n % 2 != 0
        print "No valid sequences"
        return
    generate("", 0, 0, n / 2)
    for each String s in result
        print s

void generate(String current, int open, int close, int max)
    if current.length == max * 2
        result.add(current)
        return
    if open < max
        generate(current + "(", open + 1, close, max)
    if close < open
        generate(current + ")", open, close + 1, max)
```

Why max * 2?

- max = number of pairs (e.g., 2 pairs means max = 2)
- Total length of balanced parentheses string = 2 * max (because each pair has an opening and a closing bracket)

- The "backtracking" is **in the recursion itself** for this problem.
- Because strings are immutable, each recursive call has its own current state.
- After returning from recursion, the caller tries other branches without needing to undo changes explicitly.

Magic Square

Given a matrix, check whether it's Magic Square or not. A Magic Square is a n x n matrix of the distinct elements from 1 to n2 where the sum of any row, column, or diagonal is always equal to the same number.

Examples:

Input : n = 3
2 7 6
9 5 1
4 3 8

Output : Magic matrix
Explanation: In matrix sum of each row and each column and diagonals sum is same = 15.

```
Naive approach( O(n^2)
-------------------
boolean[] seen = new boolean[n * n + 1];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                mat[i][j] = sc.nextInt()
                if (mat[i][j] < 1 || mat[i][j] > n * n || seen[mat[i][j]])
                    System.out.println("Not a Magic Matrix");
                    return
                }
                seen[mat[i][j]] = true
            }
        if (isMagicSquare(mat, n))
            System.out.println("Magic matrix");
        else
            System.out.println("Not a Magic Matrix");
    }
    static boolean isMagicSquare(int[][] mat, int n)
        int sumd1 = 0, sumd2 = 0
        for (int i = 0; i < n; i++)
            sumd1 += mat[i][i]
            sumd2 += mat[i][n - 1 - i]

        if (sumd1 != sumd2)
            return false;

        for (int i = 0; i < n; i++)
            int rowSum = 0, colSum = 0;
            for (int j = 0; j < n; j++)
                rowSum += mat[i][j]
                colSum += mat[j][i]

            if (rowSum != colSum || colSum != sumd1)
                return false

        return true
```

Minimum cost to convert to a Magic Square

Problem:

Consider a 3 X 3 matrix, **s**, of integers in the inclusive range [1, 9] . We can convert any digit, **a**, to any other digit, **b**, in the range [1, 9] at cost **|a - b|**.

Given **s**, convert it into a magic square at minimal cost by changing zero or more of its digits. The task is to find minimum cost.

Note: The resulting matrix must contain distinct integers in the inclusive range [1, 9].

Input:

Input : mat[][] = { { 4, 9, 2 },
{ 3, 5, 7 },
{ 8, 1, 5 }};

Output : 1

Given matrix s is not a magic square. To convert it into magic square we change the bottom right value, s[2][2], from 5 to 6 at a cost of | 5 - 6 | = 1.

mat[][] = { { 4, 8, 2 },
{ 4, 5, 7 },
{ 6, 1, 6 }};

Output : 4

naive Approach:

- There are **only 8 possible 3×3 magic squares** with numbers from 1 to 9.
- So, we can:

1. Store all 8 valid magic squares.
2. For each, compute the cost to convert the given matrix to that square.
3. Return the minimum cost.

Why Only 8 Magic Squares?

1. **Fixed sum condition:**
   For a 3×3 magic square with **numbers 1 to 9**, the **sum of all numbers** is:
   1+2+3+4+5+6+7+8+9 = 45

2. Since there are **3 rows**, each row must sum to: 45/3=15
3. So each **row, column, and diagonal must sum to 15.**
4. All 9 digits are **distinct** and must be in range 1 to 9.
5. Mathematically, for 3×3 with these constraints, there's **only one base magic square**:
   8 1 6
   3 5 7
   4 9 2

6. From this square, you can get **7 more** by applying **symmetry operations**:
   - 3 rotations (90°, 180°, 270°)
   - 4 reflections (horizontal, vertical, and diagonal)

   These transformations **don't create new unique configurations**, just symmetric versions.

- 1 original
- 3 rotations
- 4 reflections

1+3+4=8 distinct 3×3 magic squares

Rotated 90°:

4 3 8
9 5 1
2 7 6

Rotated 180°:

2 9 4
7 5 3
6 1 8

270° Clockwise Rotation = 90° Counter-Clockwise
Rotating **270° clockwise** is the same as rotating **90° counter-clockwise**.
To do this:

- Take each **column from right to left**, and turn it into a row **from top to bottom**.

8 1 6
3 5 7
4 9 2

More formally:
The element at position mat[i][j] moves to mat[cols - 1 - j][i].

6 7 2
1 5 9
8 3 4

What do "4 reflections" mean in a 3×3 magic square?
A **reflection** means "flipping" the matrix across some axis, like a mirror.
For a 3×3 grid, there are **4 meaningful reflections** that produce different (but valid) magic squares from the original:

☑ **Original magic square:**

8 1 6
3 5 7
4 9 2

🔁 **1. Horizontal Reflection (flip top-to-bottom)**

4 9 2
3 5 7
8 1 6

🔁 **2. Vertical Reflection (flip left-to-right)**

6 1 8
7 5 3
2 9 4

🔁 **3. Main Diagonal Reflection (top-left to bottom-right)**
Swap mat[i][j] with mat[j][i]

8 3 4
1 5 9
6 7 2

🔁 **4. Anti-diagonal Reflection (top-right to bottom-left)**
Swap mat[i][j] with mat[n-1-j][n-1-i]

2 7 6

```
9 5 1
4 3 8
8 1 6
3 5 7
4 9 2
```

```
int[][] mat = new int[3][3]

for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        mat[i][j] = sc.nextInt()

int[][][] magic_squares = {
    { {8, 1, 6}, {3, 5, 7}, {4, 9, 2} },
    { {6, 1, 8}, {7, 5, 3}, {2, 9, 4} },
    { {4, 9, 2}, {3, 5, 7}, {8, 1, 6} },
    { {2, 9, 4}, {7, 5, 3}, {6, 1, 8} },
    { {8, 3, 4}, {1, 5, 9}, {6, 7, 2} },
    { {4, 3, 8}, {9, 5, 1}, {2, 7, 6} },
    { {6, 7, 2}, {1, 5, 9}, {8, 3, 4} },
    { {2, 7, 6}, {9, 5, 1}, {4, 3, 8} }
};

int min_cost = Integer.MAX_VALUE;

for (int[][] magic : magic_squares)
    int cost = 0;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            cost += Math.abs(mat[i][j] - magic[i][j])
    min_cost = Math.min(min_cost, cost)
}

System.out.println(min_cost)
```

We subtract **to find how much adjustment is needed** at each cell to reach the corresponding number in the magic square. Summing these differences gives the total cost to convert the input matrix to that magic square.

Approach Using Backtracking: (O(9!)

- Generates **all permutations of numbers 1 to 9.**
- Each permutation represents a candidate 3×3 matrix (flattened as a 1D array of length 9).
- For each permutation, it checks if it forms a magic square.
- If yes, it computes the cost to convert the input matrix to this magic square.

- Keeps track of the minimal cost found so far.

Why 9! (factorial 9)?
- You are arranging **9 distinct numbers** (from 1 to 9) in **9 positions**.
- The number of ways to arrange n distinct elements in n positions is **n!** (n factorial).
- So for 9 numbers, the total number of permutations is:

$9!=9\times8\times7\times6\times5\times4\times3\times2\times1=362,880$

Intuition
- For the first position, you can choose any of the 9 numbers.
- For the second position, you can choose from remaining 8 numbers.
- For the third, remaining 7 numbers,and so on...

So in backtracking:
- You try all 9! permutations of numbers 1 to 9.
- Each permutation forms a possible 3x3 matrix.
- You check which ones are magic squares.

At each recursion level idx (from 0 to 8), we choose one number from 1 to 9 **not yet used.**
We maintain:
- arr[]: the current forming permutation.
- visited[]: tracks what numbers are already used.

Example:
- idx = 0 → try 1
- idx = 1 → try 2 (since 1 visited)
- idx = 2 → try 3 (1, 2 visited)
- ...
- idx = 8 → try 9 (all others visited)

One permutation: [1, 2, 3, 4, 5, 6, 7, 8, 9]

- The visited array ensures each number is used exactly once.
- Recursive calls explore all possible sequences.
- Backtracking unmarks numbers so they can be reused in other permutations.
- Base case triggers when all positions are filled.

Without resetting visited[i] to false, once a number is used, it would never be available again for other permutations, and you won't explore all possibilities.
Backtracking **restores the state to how it was before choosing that number**, so the algorithm can explore other branches.

```
Stack Pushes (recursive calls)
--------------------------------
idx = 0 → arr[0] = 1, visited[1] = true
    idx = 1 → arr[1] = 2, visited[2] = true
        idx = 2 → arr[2] = 3, visited[3] = true
            idx = 3 → arr[3] = 4, visited[4] = true
                idx = 4 → arr[4] = 5, visited[5] = true
                    idx = 5 → arr[5] = 6, visited[6] = true
                        idx = 6 → arr[6] = 7, visited[7] = true
                            idx = 7 → arr[7] = 8, visited[8] = true
                                idx = 8 → arr[8] = 9, visited[9] = true
                                    idx = 9 → complete arr: [1,2,3,4,5,6,7,8,9]
```

We return to:
idx = 8
- arr = [1,2,3,4,5,6,7,8]
- visited[9] = true, visited[8] = true
- Now we unmark 9:

  visited[9] = false
  arr = [1,2,3,4,5,6,7,8]
- At this point, loop in idx=8 has tried:
  - i = 1 to 8 → already visited
  - i = 9 → tried, now backtracked

→ So **no new values left to try at idx=8**, so we backtrack again.

Backtrack to:
idx = 7
- arr = [1,2,3,4,5,6,7,8]
- visited[8] = true → unvisit

  visited[8] = false
  arr = [1,2,3,4,5,6,7]

Now in the loop for idx=7, we had:
- i = 8 → tried before
- So next: **i = 9** → not visited

arr[7] = 9
visited[9] = true
→ arr = [1,2,3,4,5,6,7,9]
permutations(arr, visited, idx + 1, input); // i.e., idx = 8
→ Move to **idx = 8**

At idx = 8:
- visited[1 to 7] = true
- visited[9] = true
- i = 1 → visited
- i = 2 → visited
  ...
- i = 8 → not visited

→ We pick:
arr[8] = 8
visited[8] = true
Now:
arr = [1,2,3,4,5,6,7,9,8]
Reached base case!
- idx = 9 → Full permutation ready
- Check if it's a magic square
- Whether it's valid or not, we **now backtrack to idx=6 and so on and compute 9! permutations.**

Key Detail:
When backtracking, the for loop **remembers where it was.** If i = 8 was the last tried value at idx = 7, then in the next iteration it tries i = 9.
So it doesn't **skip 8 arbitrarily** — it just **doesn't retry it** because it already tried i = 8 in that position **before**

At idx = 7:
- **You had already done:**

  i = 8 → arr[7] = 8 → leads to [1..8,9]

  That was fully explored.
- **After backtracking:**

- visited[8] = false
- Loop now continues with:

  i = 9 → arr[7] = 9

subset sum with K print all subsets using backtracking

```
void printSubsetSum(int i, int n, int[] arr, int targetSum, List<Integer> subset)
 {
    if (targetSum == 0) {
        // Print current subset
        for (int x : subset) System.out.print(x + " ");
        System.out.println();
        return;
    }
    if (i == n || targetSum < 0)
        return;


    // Not pick current element
    printSubsetSum(i + 1, n, arr, targetSum, subset);

    // Pick current element if it is <= targetSum
    if (arr[i] <= targetSum) {
        subset.add(arr[i]);
        printSubsetSum(i + 1, n, arr, targetSum - arr[i], subset);
        subset.remove(subset.size() - 1); // backtrack
    }
}
```

`subset.remove(subset.size() - 1);` means:

- Remove the **last element** added to the list subset.
- This is done **after** the recursive call returns, to **undo** the previous addition.
- It's part of **backtracking**, restoring the state so other recursive paths can try different subsets without interference.

In simpler terms:

- When you add an element to subset and explore further,
- After exploring that path, you remove that element to **go back** and try other possibilities.

Trace with Input [1, 2, 1], sum = 3:

- Start: i=0, targetSum=3, subset = []

1) At i=0 (value = 1), targetSum=3

- **Not pick** 1:
  - Call: i=1, targetSum=3, subset=[]

2) At i=1 (value = 2), targetSum=3

- **Not pick** 2:
  - Call: i=2, targetSum=3, subset=[]

3) At i=2 (value = 1), targetSum=3

- **Not pick** 1:
  - Call: i=3, targetSum=3, subset=[]
  - i == n, targetSum != 0 → return (dead end)
- **Pick** 1 (because 1 ≤ 3):
  - subset = [1]
  - targetSum = 3 - 1 = 2
  - Call: i=3, targetSum=2, subset=[1]
  - i == n, targetSum != 0 → return (dead end)

Backtrack: subset = []

Back to i=1 (value=2), targetSum=3

- **Pick** 2 (2 ≤ 3):
  - subset = [2]
  - targetSum = 3 - 2 = 1
  - Call: i=2, targetSum=1, subset=[2]

4) At i=2 (value=1), targetSum=1

- **Not pick** 1:
  - Call: i=3, targetSum=1, subset=[2]
  - i == n, targetSum != 0 → return (dead end)
- **Pick** 1 (1 ≤ 1):
  - subset = [2, 1]
  - targetSum = 1 - 1 = 0
  - Call: i=3, targetSum=0, subset=[2,1]

Since targetSum == 0 → **print [2 1]**
Backtrack: subset = [2]
Backtrack: subset = []

Back to i=0 (value=1), targetSum=3

- **Pick** 1 (1 ≤ 3):
  - subset = [1]
  - targetSum = 3 - 1 = 2
  - Call: i=1, targetSum=2, subset=[1]

5) At i=1 (value=2), targetSum=2

- **Not pick** 2:
  - Call: i=2, targetSum=2, subset=[1]

6) At i=2 (value=1), targetSum=2

- **Not pick** 1:
  - Call: i=3, targetSum=2, subset=[1]
  - i == n, targetSum != 0 → return (dead end)
- **Pick** 1 (1 ≤ 2):
  - subset = [1, 1]
  - targetSum = 2 - 1 = 1
  - Call: i=3, targetSum=1, subset=[1,1]
  - i == n, targetSum != 0 → return (dead end)

Backtrack: subset = [1]

- **Pick** 2 (2 ≤ 2):
  - subset = [1, 2]
  - targetSum = 2 - 2 = 0
  - Call: i=2, targetSum=0, subset=[1,2]

Since targetSum == 0 → **print [1 2]**
Backtrack: subset = [1]
Backtrack: subset = []

Output subsets for sum = 3:

- [2 1]
- [1 2]

print all subsets backtracking 1

Problem:Print all valid subsets of a given array.

Input:

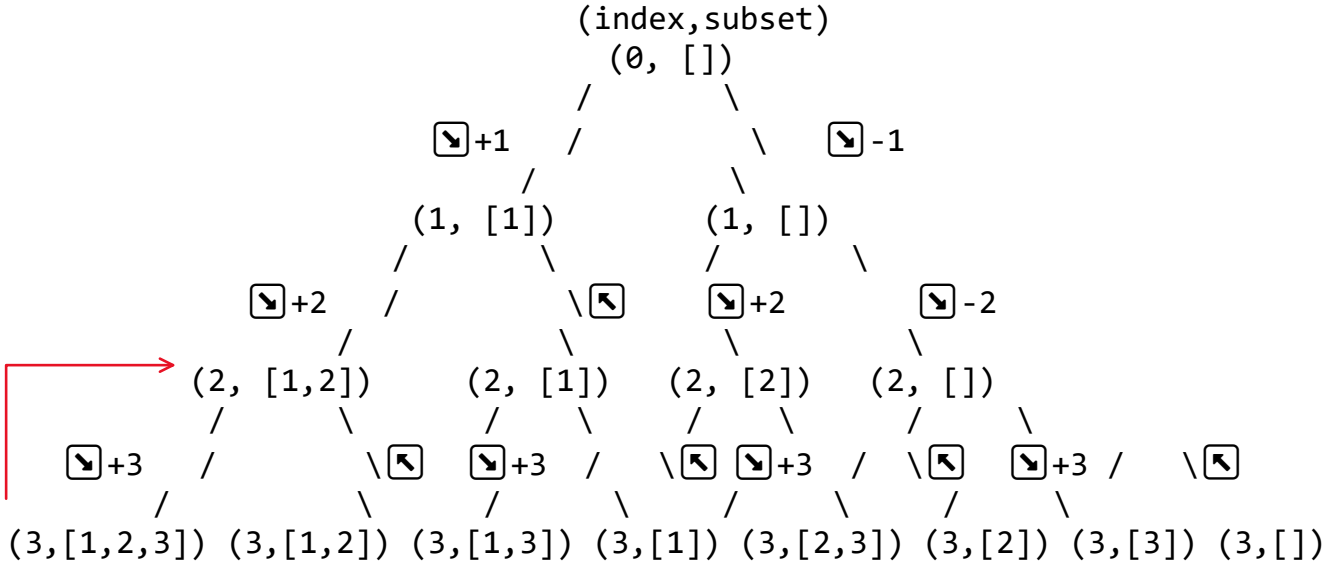
Output:

Naive Approach:

Optimized Approach:

Call Stack Level	index	subset (current)	Action	Next calls
0	0	[]	Include nums[0] = 1	Call generateSubsets(nums,1,[1])
1	1	[1]	Include nums[1] = 2	Call generateSubsets(nums,2,[1,2])
2	2	[1,2]	Include nums[2] = 3	Call generateSubsets(nums,3,[1,2,3])
3	3	[1,2,3]	index == nums.length → print [1, 2, 3]	Return
2 (backtrack)	2	[1,2,3]	Remove last: 3 → subset = [1,2]	Exclude nums[2], call generateSubsets(nums,3,[1,2])
3	3	[1,2]	index == nums.length → print [1, 2]	Return
1 (backtrack)	1	[1,2]	Remove last: 2 → subset = [1]	Exclude nums[1], call generateSubsets(nums,2,[1])
2	2	[1]	Include nums[2] = 3	Call generateSubsets(nums,3,[1,3])
3	3	[1,3]	index == nums.length → print [1, 3]	Return
2 (backtrack)	2	[1,3]	Remove last: 3 → subset = [1]	Exclude nums[2], call generateSubsets(nums,3,[1])
3	3	[1]	index == nums.length → print [1]	Return
0 (backtrack)	0	[1]	Remove last: 1 → subset = []	Exclude nums[0], call generateSubsets(nums,1,[])
1	1	[]	Include nums[1] = 2	Call generateSubsets(nums,2,[2])
2	2	[2]	Include nums[2] = 3	Call generateSubsets(nums,3,[2,3])
3	3	[2,3]	index == nums.length → print [2, 3]	Return
2 (backtrack)	2	[2,3]	Remove last: 3 → subset = [2]	Exclude nums[2], call generateSubsets(nums,3,[2])
3	3	[2]	index == nums.length → print [2]	Return
1 (backtrack)	1	[2]	Remove last: 2 → subset = []	Exclude nums[1], call generateSubsets(nums,2,[])
2	2	[]	Include nums[2] = 3	Call generateSubsets(nums,3,[3])

3	3	[3]	index == nums.length → print [3]	Return
2 (backtrack)	2	[3]	Remove last: 3 → subset = []	Exclude nums[2], call generateSubsets(nums,3,[])
3	3	[]	index == nums.length → print []	Return



- **+1, +2, +3** → means we **included** the next number (e.g., included 1, 2, or 3).
- ↘**-1, -2, -3** → means we **excluded** the number at that step.
- ↖ → the stack **returns** from that recursive call to explore other possibilities (backtracking).

Call Stack:

```
generateSubsets([1,2,3], 0, [])
|
├── include 1 → [1]
|   └── generateSubsets([1,2,3], 1, [1])
|       ├── include 2 → [1,2]
|       |   └── generateSubsets([1,2,3], 2, [1,2])
|       |       ├── include 3 → [1,2,3]
|       |       |   └── generateSubsets([1,2,3], 3, [1,2,3])
|       |       |       └── index == 3 → print [1,2,3]
```

So now:

We hit base case index == nums.length

Print [1,2,3]

Function returns to previous level

⬅ BACK BACKTRACKING: LEVEL 2
We go one level up, where:
index = 2

subset = [1, 2, 3]

Now we execute:
subset.remove(subset.size() - 1);
Now subset becomes: [1, 2]

And now we move to:
generateSubsets(nums, 3, subset); // Option 2: exclude nums[2]
So now call becomes:
generateSubsets([1,2,3], 3, [1,2]) again base case reached
→ print [1,2]

Backtrack to Index = 1, Subset = [1,2]
We go one level up:
We were at:
generateSubsets([1,2,3], 2, [1,2])
Now, we **backtrack** by removing 2:
subset = [1]
Now we're back at:
generateSubsets([1,2,3], 1, [1])
- Include 2 → [1,2] → explored fully ☑

Now try:
➤ **Exclude 2**
generateSubsets([1,2,3], 2, [1])

Now at Index = 2, Subset = [1]
- Include 3 → [1,3] → generateSubsets(3, [1,3]) → print ☑
- Backtrack → subset = [1]
- Exclude 3 → generateSubsets(3, [1]) → print [1] ☑

call generateSubsets(nums, 0, subset)

```
static void generateSubsets(int[] nums, int index, List<Integer> subset)
       if (index == nums.length) {
           System.out.println(subset);
           return;

       // Option 1: Include nums[index]
       subset.add(nums[index]);
       generateSubsets(nums, index + 1, subset);
       // Backtrack: Remove last added element
       subset.remove(subset.size() - 1);
       // Option 2: explore further options
       generateSubsets(nums, index + 1, subset);
```

Print all subsets in the lexicographical order 2

Problem:

Input:

3

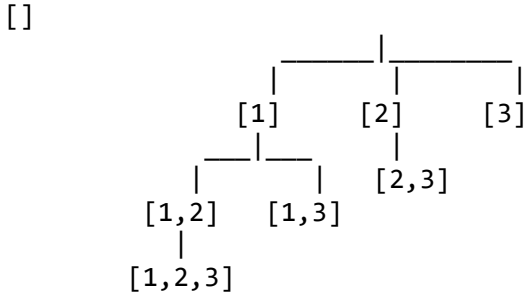
arr = [2, 1, 3]

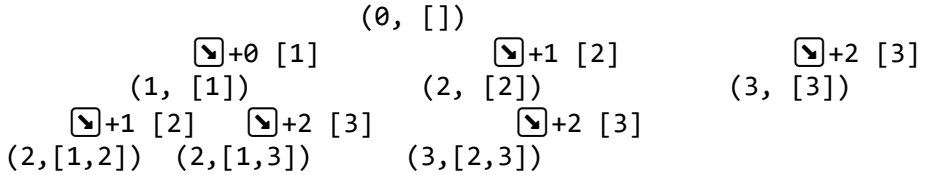
Output:

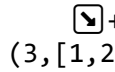
[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3],[3]

- Read input array arr of size n from user.
- Sort the array arr to ensure subsets are in lexicographical order.
- Initialize:
  - ans: list to store all subsets.
  - temp: temporary list to build each subset.
- Call generate(idx = 0, temp = []) to start recursive generation.
- Inside generate(idx, temp):
  - Add current subset temp to ans.
  - Loop from i = idx to n - 1:
    - Add arr[i] to temp.
    - Recurse: generate(i + 1, temp)
    - Backtrack: remove last element from temp list.

For each subset in ans:
  Print a left square bracket [
Iterate through each element of the current subset:
  Print the element
  If it is **not the last element**, print a comma followed by a space
After printing all elements of a subset, print the right bracket] and move to the next line







Call Stack Level	idx	temp	Action Taken	Next Calls (i from idx)
0	0	[]	Add [] to ans	i=0 → Add 1
1	1	[1]	Add [1] to ans	i=1 → Add 2
2	2	[1, 2]	Add [1,2] to ans	i=2 → Add 3
3	3	[1, 2, 3]	Add [1,2,3] to ans	i=3 ends → backtrack 3
2	2	[1, 2]	Remove 3 from temp	i=2 ends → backtrack 2
1	1	[1]	Remove 2 from temp	i=2 → Add 3
2	3	[1, 3]	Add [1,3] to ans	i=3 ends → backtrack 3
1	1	[1]	Remove 3 from temp	i=2 ends → backtrack 1
0	0	[]	Remove 1 from temp	i=1 → Add 2
1	2	[2]	Add [2] to ans	i=2 → Add 3
2	3	[2, 3]	Add [2,3] to ans	i=3 ends → backtrack 3
1	2	[2]	Remove 3 from temp	i=2 ends → backtrack 2
0	0	[]	Remove 2 from temp	i=2 → Add 3
1	3	[3]	Add [3] to ans	i=3 ends → backtrack 3
0	0	[]	Remove 3 from temp	Loop ends, done

Print all valid String partitions
Backtracking 3

Print all valid partitions of a string s using a given dictionary
Given:

- String: s
- Dictionary: wordDict

Task: Print all ways to split s into dictionary words such that the entire string is used and every substring in the partition exists in the dictionary.

Input
pineapplepenapple
5
apple
pen
applepen
pine
pineapple
Output
pine apple pen apple
pineapple pen apple
pine applepen apple

s = "pineapplepenapple"
dict = ["apple", "pen", "applepen", "pine", "pineapple"]
We are calling:

backtrack("pineapplepenapple", 0, dict, [])

We'll build the string from start = 0, and at each step we check every substring s[start:end] and recurse when it's a valid dictionary word.

Step-by-step Trace:
At start = 0:
Check substrings:

s[0:1] = "p" → ❌

...

s[0:4] = "pine" → ☑

→ Add "pine" to path → path = ["pine"]

At start = 4:
Check substrings:

s[4:5] = "a" → ❌

...

s[4:9] = "apple" → ☑

→ Add "apple" to path → path = ["pine", "apple"]

At start = 9:
Check substrings:

s[9:12] = "pen" → ☑

→ Add "pen" to path → path = ["pine", "apple", "pen"]

At start = 12:
Check substrings:

s[12:17] = "apple" → ☑

→ Add "apple" to path → path = ["pine", "apple", "pen", "apple"]

✔ Reached end → start == s.length()

☑ Output this partition:

text
Copy
Edit
pine apple pen apple
Now backtrack:

Remove last: "apple" → ["pine", "apple", "pen"]

No more substrings at start=12, backtrack

Remove "pen" → ["pine", "apple"]

Back to start = 9, try more:
s[9:14] = "pena" ✖

...

Done → backtrack

Backtrack again to ["pine"] and start = 4, now try:
s[4:12] = "applepen" → ☑

→ Add "applepen" → ["pine", "applepen"]

At start = 12:
s[12:17] = "apple" → ☑

→ Add "apple" → ["pine", "applepen", "apple"]

✔ Reached end → print:

pine applepen apple
Backtrack → remove "apple" → ["pine", "applepen"]

→ No more substrings → remove "applepen" → ["pine"]

Backtrack to start = 0, try:
s[0:9] = "pineapple" → ☑

→ Add "pineapple" → ["pineapple"]

At start = 9:
s[9:12] = "pen" → ☑

→ Add "pen" → ["pineapple", "pen"]

At start = 12:
s[12:17] = "apple" → ☑

→ Add "apple" → ["pineapple", "pen", "apple"]

✔ Print:

text
Copy
Edit
pineapple pen apple
☑ Final Output (3 Ways)
nginx
Copy
Edit
pine apple pen apple
pine applepen apple
pineapple pen apple

N Queens Problem

Given an integer **n**, the task is to find the solution to the **n-queens problem**, where **n** queens are placed on an **n*n** chessboard such that no two queens can attack each other.

The **N** Queen is the problem of placing **N** chess queens on an **N×N** chessboard so that no two queens attack each other.

In N-Queens, this means:

1. **They lie on the same row**
2. **They lie on the same column**
3. **They lie on the same diagonal**
   There are two types of diagonals:
   - **Main diagonal**
   - **Anti-diagonal**

n = 4
Output=

```
. Q . .
. . . Q
Q . . .
. . Q .
```

APPROACH 1: Naive Backtracking
Idea:

Try placing queens row by row and for every row, try all columns. Check for safety by scanning the board.
Time Complexity: O(N!)
(Each row has N options, then N-1, then N-2, … → N!)
Space Complexity: O(N²) (Board matrix)

[N-Queens Problem | using Backtracking | Leetcode Hard]


#45
N
QUEENS
Backtracking Problem
Approach
Code
Complexity

Intro to DP

19 May 2025 16:27

Dynamic programming has two main approaches: top-down (memoization) and bottom-up (tabulation). These methods solve problems by storing and reusing solutions to overlapping subproblems, but they differ in how they approach the problem.

1. Top-Down (Memoization):

- This approach is also known as "recursive dynamic programming".
- It starts with the main problem and recursively breaks it down into smaller subproblems.
- It stores the solutions to these subproblems in a cache (like array) to avoid recalculating them.
- It checks if the result for a subproblem is already in the cache before solving it recursively.

2. Bottom-Up (Tabulation):

- This approach is also known as "iterative dynamic programming".
- It starts with the base cases and builds the solution iteratively, working its way up to the main problem.
- It uses a table to store the solutions to subproblems, filling it in a systematic way.
- It calculates the solution to larger problems based on the solutions to smaller problems already stored in the table.

Explain Rod cutting problem for both approaches before subset with sum k.

Top-Down	Recursive + Memoization	Start from the final problem, solve smaller
Bottom-Up	Iterative + Tabulation	Start from base cases, build up to answer

0/1 knapsack problem using DP memoization top down

24 May 2025 11:19

What Happens in Recursion?

In recursive or top-down approaches:

- The order of subproblem solving is dynamic, controlled by the call stack.
- A subproblem is solved only when it's needed (lazy evaluation).
- Memoization helps avoid recomputation, but you don't control which subproblems get solved first.

This can sometimes lead to:

- Deep recursion (stack overflow)
- Harder debugging and tracing
- Unpredictable order of execution

Dynamic Programming (Top-Down or Bottom-Up)

▶ **How it works:**

- DP approach **stores** solutions to subproblems (in a dp[n][W] table)
- If a subproblem is already solved, just **return** the stored result (memoization)

☑ **Efficiency:**

- **No redundant calls**
- Fills only each unique (n, W) state once

Approach	Time Complexity	Space Complexity
Recursion	O(2^n)	O(n) (call stack)
Top-Down DP	O(n * W)	O(n * W)
Bottom-Up DP	O(n * W)	O(n * W)
Space Optimized	O(n * W)	O(W)

Both recursive approach and DP versions have the **same recursive structure.**
BUT, the DP version **remembers the answer** for (n, W) and reuses it.

Why It Matters

Let's say:

- You call knapsack(3, 4)
- It internally calls knapsack(2, 3) and knapsack(2, 4)
- Without memoization: these may both call knapsack(1, 3) → redundant
- With memoization: knapsack(1, 3) is solved once and reused

```
Approach
---------
dp[i][w] = max value using items 0...i with remaining capacity w

function:
knapsack(int n, int W, int[] wt, int[] val, int[][] dp)

        if(n == 0 || W == 0)
            return 0
```

```
        if(dp[n][W] != -1)
            return dp[n][W]
        if(wt[n - 1] > W)
            dp[n][W] = knapsack(n - 1, W, wt, val, dp);
        else {
            int include = val[n - 1] + knapsack(n - 1, W - wt[n - 1], wt, val, dp)
            int exclude = knapsack(n - 1, W, wt, val, dp);
            dp[n][W] = Math.max(include, exclude);
        }
        return dp[n][W]
```

```
n = 3
w = 5
wt = [1, 2, 3]
val = [10, 15, 40]

We want dp[2][5] → max value with first 3 items and capacity 5.
1. Try including item 2 (wt=3): val[2] + dp[1][2] = 40 + dp[1][2]
2. Try excluding item 2: dp[1][5]
Continue evaluating dp[1][2], dp[1][5], ... and so on recursively.
```

Climbing Stairs to reach to the top

24 May 2025 09:21

Count ways to reach the nth stair using step 1, 2 or 3

24 May 2025 09:22

knapsack Space Optimized DP

24 May 2025 13:12

Why Optimize?
In the 2D DP table dp[i][j], each row i depends **only on the previous row** i−1.
So we can reduce the space from **O(n * W)** → **O(W)** using a **1D array**.

In the **0/1 Knapsack problem**, we use a 2D DP table dp[i][j]:

- i = index of item.
- j = capacity from 0 to W.

Observation:

- dp[i][j] depends **only on row i-1** (i.e., previous row).
- So, at any point, to compute dp[i][j], only dp[i-1][...] values are required.

When updating dp[j], we don't want to reuse the **updated dp[j - wt[i]]** from the same iteration.
When we go backward (j = W to 0):

- dp[j - weight[i]] has **not been updated yet** in this iteration.
- It still holds the value **from previous item** (i−1) — exactly what we want.

dp = [0 0 0 0 0 0]
Now loop j = 5 to 3:
dp[5] = max(0, 2 + dp[2]) → dp[5] = 2
dp[4] = max(0, 2 + dp[1]) → dp[4] = 2
dp[3] = max(0, 2 + dp[0]) → dp[3] = 2
At each step:

- dp[j - weight[i]] is still the **old value** (from previous state).
- We **don't touch** dp[2], dp[1], or dp[0] until after they're read.

Each item can be included **only once**.
So when computing dp[j], it must depend on the values **before including the current item**.

☑ **Scenario: Why Reverse Loop?**
Let's walk through a **concrete example** and compare **forward vs reverse iteration**.
Example:

Items: wt = [2], val = [5]
Capacity = 4
We'll build the dp array of size 5 (0 to 4).

WRONG: Forward Iteration (j = 0 to W)

```
for(int j = 0; j <= w; j++) {
    if(j >= wt[i]) {
        dp[j] = Math.max(dp[j], val[i] + dp[j - wt[i]]);
    }
}
```

Let's simulate:

- Initially: dp = [0, 0, 0, 0, 0]
- Process item wt = 2, val = 5

Forward update:
j = 2 → dp[2] = max(0, 5 + dp[0]) = 5 → dp[2] = 5
j = 3 → dp[3] = max(0, 5 + dp[1]) = 5 → dp[3] = 5
j = 4 → dp[4] = max(0, 5 + dp[2]) = 10 → ❌ wrong!

Problem:
- dp[4] is using **dp[2] = 5**, which was **just updated this round** (for this item).
- This means it's **using the same item more than once**, which is invalid in 0/1 Knapsack.

☑ **CORRECT: Reverse Iteration (j = W to 0)**

```
for(int j = w; j >= wt[i]; j--) {
    dp[j] = Math.max(dp[j], val[i] + dp[j - wt[i]]);
}
```

Let's simulate the correct version:
- Initially: dp = [0, 0, 0, 0, 0]

Reverse update:
- j = 4 → dp[4] = max(0, 5 + dp[2]) = 0 + 5 = 5
- j = 3 → dp[3] = max(0, 5 + dp[1]) = 0 + 5 = 5
- j = 2 → dp[2] = max(0, 5 + dp[0]) = 5

Final dp = [0, 0, 5, 5, 5]

☑ Now we used each item **only once**, and dp[4] uses dp[2] **from previous state** — not the one updated this round.

Idea:
Why j >= weight[i]?
Because:
- We can **only pick** item i if the **remaining capacity j is at least weight[i].**
- Otherwise, the item **won't fit** in the knapsack.

```
n = 4
W = 8
profit[] = {2, 3, 4, 1}
weight[] = {3, 4, 5, 6}
```

```
 int[] dp = new int[W + 1]

for(int i = 0; i < n; i++)
            for(int j = W; j >= weight[i]; j--)
                dp[j] = Math.max(dp[j], profit[i] + dp[j - weight[i]]);
```

```
dp = [0, 0, 0, 0, 0, 0, 0, 0, 0]
After Item 1 (profit = 2, weight = 3)
j = 8 to 3 → dp[j] = max(dp[j], 2 + dp[j - 3])
dp[8] = max(0, 2 + dp[5]) = 2
```

```
dp[7] = max(0, 2 + dp[4]) = 2
dp[6] = max(0, 2 + dp[3]) = 2
dp[5] = max(0, 2 + dp[2]) = 2
dp[4] = max(0, 2 + dp[1]) = 2
dp[3] = max(0, 2 + dp[0]) = 2

dp = [0, 0, 0, 2, 2, 2, 2, 2, 2]

After Item 2 (profit = 3, weight = 4)
j = 8 to 4 → dp[j] = max(dp[j], 3 + dp[j - 4])
dp[8] = max(2, 3 + dp[4]) = max(2, 3 + 2) = 5
dp[7] = max(2, 3 + dp[3]) = max(2, 3 + 2) = 5
dp[6] = max(2, 3 + dp[2]) = max(2, 3 + 0) = 3
dp[5] = max(2, 3 + dp[1]) = 3
dp[4] = max(2, 3 + dp[0]) = 3
dp = [0, 0, 0, 2, 3, 3, 3, 5, 5]

After Item 3 (profit = 4, weight = 5)
j = 8 to 5 → dp[j] = max(dp[j], 4 + dp[j - 5])
dp[8] = max(5, 4 + dp[3]) = max(5, 6) = 6
dp[7] = max(5, 4 + dp[2]) = 5
dp[6] = max(3, 4 + dp[1]) = 4
dp[5] = max(3, 4 + dp[0]) = 4
dp = [0, 0, 0, 2, 3, 4, 4, 5, 6]

After Item 4 (profit = 1, weight = 6)
j = 8 to 6 → dp[j] = max(dp[j], 1 + dp[j - 6])
dp[8] = max(6, 1 + dp[2]) = max(6, 1) = 6
dp[7] = max(5, 1 + dp[1]) = 5
dp[6] = max(4, 1 + dp[0]) = 4

dp = [0, 0, 0, 2, 3, 4, 4, 5, 6]
 Final Answer: dp[8] = 6
```

knapsack using Tabulation bottom up DP

24 May 2025 13:13

Top-down (memoized recursion):
You only solve what's needed:

```
solve(0, W) → solve(1, W), solve(1, W - wt[0]), .
```

What Happens in Tabulation?
In **tabulation (bottom-up)**:

- You solve all subproblems in a **fixed, predetermined order**.
- You **build the solution step by step**, from the smallest subproblems to the biggest.
- You can **observe and control** how values evolve in the dp[][] table.

This gives:

- Better predictability
- Easier tracing/debugging
- Efficient space optimization (1D arrays)
- No risk of stack overflow

Bottom-up:

```
t
for i = 0 to n:
   for j = 0 to W:
       dp[i][j] = ...
```

You **explicitly control** the order: row-by-row, left-to-right (or right-to-left in space optimization).

Tabulation = Iterative Bottom-Up

- You fill a **dp table iteratively**, solving smaller subproblems first.
- You know exactly **which previous values** are needed to compute the current value.

Because of this controlled flow:

- You can **identify redundancy**
- You can determine if **earlier rows** of the table can be reused

Why Tabulation Helps Space Optimization
Let's use **0/1 Knapsack** as an example.
In Full Tabulation:

```
int[][] dp = new int[n+1][W+1];
```

- Time: O(nW)
- Space: O(nW)

Each dp[i][j] depends only on:

```
dp[i-1][j]               // exclude
dp[i-1][j - wt[i]]       // include
```

So only **row i-1** is needed. This insight comes **because tabulation exposes dependencies** explicitly.
From this, we optimize:

int[] dp = new int[W+1];
We reuse the same row (backwards traversal), maintaining correctness.
This space optimization is only possible because tabulation clearly shows the subproblem dependency direction.

Bottom-Up DP (Tabulation) – Key Idea
We use a dp[n+1][W+1] table where:
- dp[i][j] = maximum profit using first i items and capacity j

For every item i and capacity j:
- **If the item's weight is more than capacity j**, we **can't pick** it
  dp[i][j] = dp[i-1][j]
- **Else**, we have 2 choices:
  - **Include it**: profit = profit[i-1] + dp[i-1][j - weight[i-1]]
  - **Exclude it**: profit = dp[i-1][j]
    Take maximum:
    dp[i][j] = max(dp[i-1][j], profit[i-1] + dp[i-1][j - weight[i-1]])

Initialization
- If i == 0 or j == 0, then dp[i][j] = 0
  (0 items or 0 capacity ⇒ 0 profit)

Final Answer
- The value at dp[n][W] contains the **maximum profit**

Example
W = 4
profit = [1, 2, 3]
weight = [4, 5, 1]

Time and Space Complexity
- **Time**: O(n * W) → loop over all items and capacities
- **Space**: O(n * W) for full 2D table

If you **include** the i-th item, then:
- You gain its profit: profit[i - 1]
- But you reduce available capacity by weight[i - 1]
- So now, you can only get additional profit from the remaining capacity

So, the **total profit if you include this item** is:

profit[i - 1] + profit_from_remaining_capacity
Which is:

profit[i - 1] + dp[i - 1][j - weight[i - 1]]

Min absoulte difference of 2 subsets

27 May 2025 15:21

Given an array arr[]:

- Divide into two subsets S1 and S2
- Goal: **Minimize** |sum(S1) - sum(S2)|

Let:

- total_sum = sum of all array elements
- sum1 = sum(S1)
- sum2 = sum(S2) = total_sum - sum1

|sum1 - sum2| = |sum1 - (total_sum - sum1)| = |2*sum1 - total_sum|

We want to **minimize** |2 * sum1 - total_sum|
So, for minimum absolute difference:

So the problem becomes:
Find a subset sum sum1 as close as possible to total / 2.

Example:
arr = [1, 6, 11, 5]
total_sum = 23
We want sum1 as close to 11.5 as possible.
If we pick:

- sum1 = 11 → sum2 = 12 → difference = 1
- sum1 = 12 → sum2 = 11 → difference = 1
- sum1 = 10 → sum2 = 13 → difference = 3
- sum1 = 9 → sum2 = 14 → difference = 5

Only around 11.5 is the minimum difference achieved

Final Logic
Use 1D DP to find all subset sums ≤ 11
Then find the maximum j ≤ 11 such that dp[j] = true

- We loop from target = total_sum / 2 down to 0
- First j such that dp[j] == true gives **closest possible subset sum**
- **dp[j] = true means: it is possible to form a subset sum equal to j.**
  - **using some elements of the array**
- We call this value s1, and s2 = total_sum - s1
- Then difference = |s1 - s2|

This guarantees **minimum absolute difference**

We need to partition the array such that the sum of one subset (sum1) is as close as possible to total_sum / 2.

|2 * sum1 - 23|
Try a few values:

sum1	2sum1	|2sum1 - 23|
0	0	23
6	12	11
11	22	1 ☑
12	24	1 ☑
13	26	3

You can see minimum difference at sum1 = 11 or 12

```
arr = [1, 6, 11, 5]
total = 23, target = 11

After DP filling:

dp[11] = true
dp[10] = false
dp[9] = false
dp[8] = true
...
→  s1 = 11
→ Compute diff = 23 - 2 * 11 = 1
{1, 6, 5} and {11} → diff = 1
```

```
arr = [1, 3, 5]
n = 3
total = 9
target = total / 2 = 4 (we will use this as upper bound in dp)
boolean[] dp = new boolean[5]; // 0 to 4
dp[0] = true; // subset sum of 0 is always possible (empty set)
dp = [T, F, F, F, F]   → We can make only sum 0 at the start
        0  1  2  3  4
```

Now process arr[0] = 1

```
We update in reverse:
for (j = 4 to 1):
    dp[j] = dp[j] || dp[j - 1]
→ j = 1: dp[1] = dp[1] || dp[0] = F || T = T
Updated:

dp = [T, T, F, F, F]
This means:
```

- ☑ We can now make **sum 1** (using element 1)

Process arr[1] = 3

```
For j = 4 to 3:
→ j = 4: dp[4] = dp[4] || dp[1] = F || T = T
→ j = 3: dp[3] = dp[3] || dp[0] = F || T = T
```

Updated:
dp = [T, T, F, T, T]
This means:

- ☑ We can now make:
  - sum 3: using only 3
  - sum 4: using 1 + 3

Process arr[2] = 5
We skip it since our target is 4 (and 5 > 4)

☑ **Final dp array:**

```
dp = [T, T, F, T, T]
       0  1  2  3  4
```

☑ **Justifying:**
dp[j] = true ⇒ There exists a subset with sum j

j	dp[j]	Possible Subset
0	true	∅ (empty set)
1	true	[1]
2	false	–
3	true	[3]
4	true	[1, 3]

Min fib terms to sum upto K

30 May 2025 12:07

1. Create DP array dp[0…k]
2. Set dp[0] = 0 (0 terms needed to make sum 0)
3. For every i from 1 to k:
    - For every Fibonacci number f ≤ i:
        - Update dp[i] = min(dp[i], 1 + dp[i - f])
4. Return dp[k]

k = 7
Fibonacci numbers: [1, 2, 3, 5]
Let's fill dp[0...7]

i	**f ≤ i**	**Best dp[i]**
0	–	0
1	1 → dp[0]+1 = 1	1
2	1→dp[1]+1=2, 2→dp[0]+1=1	1
3	1+dp[2]=2, 2+dp[1]=2, 3+dp[0]=1	1
4	1+dp[3]=2, 2+dp[2]=2, 3+dp[1]=2	2
5	1+dp[4]=3, 2+dp[3]=2, 3+dp[2]=2, 5+dp[0]=1	1
6	1+dp[5]=2, 2+dp[4]=3, 3+dp[3]=2, 5+dp[1]=2	2
7	1+dp[6]=3, 2+dp[5]=2, 3+dp[4]=3, 5+dp[2]=2	2

→ dp[7] = 2 is final answer.

Time O(k × f) (f=fib nums ≤ k)
Space O(k)

Climbing Stairs (with order)

02 June 2025 10:44

You are climbing a staircase. It takes n steps to reach the top.
Each time you can either climb **1 or 2 steps**.
Find the number of distinct ways you can climb to the top.

Input=5
output=8

Explaination:
All combinations of 1-step and 2-steps:
1. 1+1+1+1+1
2. 1+1+1+2
3. 1+1+2+1
4. 1+2+1+1
5. 2+1+1+1
6. 1+2+2
7. 2+1+2
8. 2+2+1
Total = **8 ways**

Input: n = 3
Output: 3
Explanation:

1. 1+1+1
2. 1+2
3. 2+1
Total= **3 ways**

You are at step 0 and need to reach step n.
At each move, you can:
- Climb **1 step**, or
- Climb **2 steps**

You need to compute the **total number of distinct sequences** of such moves to reach step n.

Approach 1: Naive Recursive
Idea: Try both options (1 or 2 steps) from current step.
We define a function f(n) = number of ways to reach the top from step 0 to n.
At any step n, the last move could be:
- A 1-step from n-1, or
- A 2-step from n-2

So,

f(n) = f(n - 1) + f(n - 2)
This is exactly the **Fibonacci recurrence**.

- f(0) = 1: 1 way to stay at step 0 (do nothing)
- f(1) = 1: 1 way to climb 1 step (just one 1-step move)

Recursion	O(2^n)	O(n)
DP with Array	O(n)	O(n)
Optimized DP	O(n)	O(1)

DP top down

Algorithm:

- Define count_ways(n) which returns number of ways to climb n steps.
- If n == 0 or 1, return 1.
- If memo[n] already computed, return it.
- Else, compute count_ways(n-1) + count_ways(n-2) and store in memo[n].

Time and Space:

- Time: O(n)
- Space: O(n) for recursion stack + O(n) for memo array

```
memo = new int[n + 1]
Arrays.fill(memo, -1)


        if (n == 0 || n == 1)
            return 1
        if (memo[n] != -1)
            return memo[n]
        memo[n] = count_ways(n - 1) + count_ways(n - 2);
        return memo[n]
    }
```

Dp bottom up

- Create dp array of size n + 1.
- Set dp[0] = 1, dp[1] = 1.
- For each i from 2 to n, dp[i] = dp[i-1] + dp[i-2].
- Return dp[n].

Time and Space:

- Time: O(n)
- Space: O(n)

```
        int n = sc.nextInt();
        int[] dp = new int[n + 1]
        dp[0] = 1
        dp[1] = 1
        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2]
        }
        System.out.println(dp[n])
    }
}
```

Optimized dp using 2 variables

```
        int n = sc.nextInt()
        int prev = 1, curr = 1
        for (int i = 2; i <= n; i++) {
            int temp = curr
            curr = curr + prev
            prev = temp
        }
        System.out.println(curr)
```

Using Mathematical Formula - O(1) Time and O(1) Space

*The basic idea is that we can first try to reach the nth stair using as many steps of size **2** as possible. So, if n is even we can reach nth stair by using **(s = n/2)** steps of size 2 else if n is odd, we can reach **(s = n/2)** steps of size 2 and 1 step of size 1. Now, for every subsequent way we can break the steps of size 2 into two steps of size 1.*

Climibing stairs order doesnt matter

02 June 2025 11:50

You want to know **how many ways to reach the top** with steps of size 1 or 2.

- E**xample:** For n=3, these are the ways:

  1 + 1 + 1
  1 + 2
  2 + 1
- Here, **1 + 2 and 2 + 1 are different** ways because order is different.
- The count for n=3 is **3.**

Order does not matter (combinations)
For the same n=3, combinations are:

1 + 1 + 1
1 + 2

- Here, 1 + 2 and 2 + 1 are considered the **same** because order is ignored.
- The count for n=3 is **2.**

for n=4
Order matters:

- 1 + 1 + 1 + 1
- 1 + 1 + 2
- 1 + 2 + 1
- 2 + 1 + 1
- 2 + 2

Count = 5

Order does not matter:

- 1 + 1 + 1 + 1
- 1 + 1 + 2
- 2 + 2

Count = 3

Idea:
simple formula works for combinations

- **Maximum pairs of 2 steps = n / 2.**
- **For each k pairs of 2-steps, rest are 1-steps.**
- **So total ways = number of possible pairs from 0 to n/2 → (n/2) + 1.**

Rod Cutting Problem

28 May 2025 12:43

Given:

- A rod of length n.
- An array price[], where price[i] is the price of a rod piece of length i.

Task:

- Cut the rod into smaller pieces (any number of cuts allowed) to **maximize the total price** obtained.

Example:

N=4
prices=[1,5,8,9]
output=
10

- **Maximum revenue = 10**
- **Optimal cuts = [2, 2]**

This means:

- Cut rod into two pieces of length 2
- Each sells for 5 → total = 10

Explaination:
We want to find the best combination of cuts that sum to 4 and give **maximum total price.**
All combinations:
1. Cut [1,1,1,1] → 1 + 1 + 1 + 1 = 4
2. Cut [2,2] → 5 + 5 = **10**
3. Cut [1,3] → 1 + 8 = 9
4. Cut [4] → 9
5. Cut [1,1,2] → 1 + 1 + 5 = 7
6. Cut [2,1,1] → 5 + 1 + 1 = 7

Input: *price[] = [1, 5, 8, 9, 10, 17, 17, 20]*
Output: *22*
Explanation*: The maximum obtainable value is 22 by cutting in two pieces of lengths 2 and 6, i.e., 5 + 17 = 22.*

Maximum Revenue = 22
Optimal Cuts = [2, 6]
This means

- *You have a **rod of length 8 inches**.*
- *You cut it into **two pieces**:*
  - *One of **length 2 inches (price=5 for 2 inches)***
  - *One of **length 6 inches (price=17 for 6 inches)***

Algorithm
1. For a rod of length n, try every possible first cut (from length 1 to n).
2. Recursively calculate maximum value for the remaining rod.
3. Take maximum of all such combinations.

Time Complexity:
Exponential → O(2^n)
Space Complexity:

O(n) (for recursion stack)

Better Approach (Bottom Up DP)

Algorithm

1. Create a dp[] array where dp[i] = maximum price for length i.
   - dp[i] stores the **maximum money** you can earn for a rod of length i.

   We fill it from dp[1] to dp[4].

2. For each length i, try all cuts j (from 1 to i).

   j represents a **possible first cut length**. That is:
   - If you're solving for rod of total length i,
   - You're trying **every way you can cut off a piece of length j (from 1 to i),**
   - Then combining that with the best answer for the **remaining rod of length i - j**.

3. Maximize dp[i] using:
   dp[i] = max(dp[i], price[j-1] + dp[i-j])

- **Time Complexity:** O(n^2) – for each rod length i, we try all cuts j = 1 to i.
- **Space Complexity:** O(n) – only one 1D dp[] array is used.

Test Case (n = 4, price = [1,5,8,9])

dp[1] = max(1) → 1
dp[2] = max(1+dp[1]=2, 5) → 5
dp[3] = max(1+dp[2]=6, 5+dp[1]=6, 8) → 8
dp[4] = max(1+dp[3]=9, 5+dp[2]=10, 8+dp[1]=9, 9) → 10

You're solving a **small problem first** (length 1), saving its answer, and **using that saved answer** to solve the next bigger problem (length 2), and so on…

You’re building the solution **bottom-up** – from small pieces to the whole.

Each time, you’re asking:

What if I cut the first piece of size j, and then use the best I already know for the remaining part.

Imagine this:

You have a rod of length n.

You also have a **price list** like this:

Length:	1	2	3	4	5	6	7	8
Price:	1	5	8	9	10	17	17	20

Now, you want to **cut the rod in pieces** such that the **total money you get is the maximum possible.**

But instead of solving it all at once, you think:

“Hmm, let me first figure out the **best way to make money from a rod of length 1**.”

Then:

“Now let's figure out the best for **length 2**, using what I already know for length 1.”

And then for 3, and 4… all the way up to 8.

Each time, you’re **just adding one more inch**, and trying all ways to use that new

length using the pieces you already figured out.
So basically, you're **filling a table** like this:

```
best[0] = 0                → no rod, no money
best[1] = 1                → best for 1 inch
best[2] = 5                → best for 2 inches
best[3] = 8
best[4] = 10
best[5] = 13
best[6] = 17
best[7] = 18
best[8] = 22              → final best value
```

By the time you reach length 8, you already know the **best prices** for all smaller lengths, and you use that info to decide what's best for 8.

for example:

Price list (0-based indexing):

```
Length:  1   2   3   4   ...
Price:   1   5   8   9   ...
```

Suppose we have:
best[0] = 0
best[1] = 1
best[2] = 5
Now we want to compute:
best[3] = ?

Try All Possible First Cuts
We'll try every first cut from 1 to 3 and use previously solved best[] values.

➤ **Option 1: First Cut = 1 inch**
- Cut rod: [1] + [2 remaining]
- Price = price[0] + best[2] = 1 + 5 = 6

➤ **Option 2: First Cut = 2 inches**
- Cut rod: [2] + [1 remaining]
- Price = price[1] + best[1] = 5 + 1 = 6

➤ **Option 3: First Cut = 3 inches**
- Cut rod: [3] + [0 remaining]
- Price = price[2] + best[0] = 8 + 0 = 8

☑ **Compare All Options**
Option 1: 6
Option 2: 6
Option 3: 8
→ Max = 8

So:
best[3] = 8

What Just Happened?
We **used our previous best answers** for rods of length 1 and 2,
tried combining them with new cuts,
and then **picked the max** out of all combinations.

Intro to Greedy

23 May 2025 11:48

ore idea: Always take the best immediate (local) choice hoping for global optimal.
Analogy: Choosing most valuable items to pack when traveling.
☑ **Real-world Example:**

Coin change problem (Greedy version):
You want to give change using the **fewest coins** possible.
Coins available: 25, 10, 5, 1 cents.
You always take the largest coin ≤ remaining amount.
📌 **Visualization for 63 cents:**

Take 25 → Remaining: 38
Take 25 → Remaining: 13
Take 10 → Remaining: 3
Take 1 → Remaining: 2
Take 1 → Remaining: 1
Take 1 → Remaining: 0

Answer: 6 coins.
⚠ Doesn't always give optimal for all coin systems (e.g., coins: 9, 6, 1 and amount 11 → greedy gives 9+1+1 = 3 but optimal is 6+6 = 2).

Coin Change Problem

24 May 2025 14:24

Given a set of coin denominations coins[] and a target amount amount, use the **minimum number of coins** to make the amount.
You can use each coin unlimited times.

The denominations are standard and canonical (like Indian or US currency: [1, 2, 5, 10, 20, 50, 100

You want to give change using the fewest coins possible.
Coins available: 25, 10, 5, 1.

example:
Visualization for target amount 63:

Take 25 → Remaining: 38
Take 25 → Remaining: 13
Take 10 → Remaining: 3
Take 1 → Remaining: 2
Take 1 → Remaining: 1
Take 1 → Remaining: 0

Answer: 6 coins.
Doesn't always give optimal for all coin systems (e.g., coins: 9, 6, 1 and amount 12 → greedy gives 9+1+1+1 = 12 coins but optimal is 6+6 = 2 coins).

- **Time: O(nlogn) (highest to lowest denomiations**
  - **O(n log n + amount/mincoins) for sorting**
  - **Worst-case inner loop: if using all 1s → O(amount)**
- **Space: O(amount) for storing result coins (worst-case)**

n = 6
coins = [1, 2, 5, 10, 20, 50]
amount = 93

Output:
50 20 20 2 1

Approach
---
1. Sort the coins in descending order.
2. Initialize:
    - remaining_amount = total_amount
    - coin_list = [] (to store selected coins)
3. Loop through each coin c in the sorted list:
    - While remaining_amount>=c, pick the coin:

        - remaining_amount -= c
        - Add c to coin_list
4. Repeat until the amount becomes 0.

Coin Exchange using DP

27 May 2025 10:01

Key Idea:

- Use a DP array dp where dp[i] represents the minimum coins needed to make amount i.
- Initialize dp[0] = 0 (0 coins needed to make amount 0).
- Initialize all other values as a large number (amount + 1) to represent infinity (impossible state).

For each amount i from 1 to target:

- Try every coin denomination c:
  - If c ≤ i, update dp[i] by comparing current value dp[i] and dp[i - c] + 1.
  - dp[i - c] + 1 means: use one coin c plus minimum coins needed for i - c.
- Take the minimum over all coins.

After filling dp[], check dp[amount]:

- If still infinity (greater than amount), return -1 (no solution).
- Otherwise, dp[amount] is the minimum coins needed.

Think like this:

- To get amount 11, try using a coin c:
  - That coin covers c units.
  - We still need to cover the leftover 11 - c units.
- If we know the minimum coins to cover 11 - c (dp[11 - c]), then total is dp[11 - c] + 1.
- Check all coins and choose the minimum.

- coins[] = {9, 6, 1}
- amount = 11

DP Stores minimum coins required for all sub-amounts, ensuring the globally optimal solution.

☑ **Initialization:**

- dp[0] = 0
- All others initialized to amount + 1 = 12 → acts as infinity.

So initially:

dp = [0, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12]

☑ **Fill DP Table:**

We process i = 1 to 11
For each i, try every coin c such that c <= i.

Step-by-step Update:

Let’s trace for each i:

i = 1
Check coins:
- 9 > 1 → skip
- 6 > 1 → skip
- 1 ≤ 1 → update
  → dp[1] = min(dp[1], dp[0] + 1) = min(12, 0 + 1) = 1

dp = [0, 1, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12]

i = 2
- 1 ≤ 2 → dp[2] = min(12, dp[1] + 1) = 1 + 1 = 2

dp = [0, 1, 2, 12, 12, 12, 12, 12, 12, 12, 12, 12]

i = 3
- 1 ≤ 3 → dp[3] = dp[2] + 1 = 2 + 1 = 3

dp = [0, 1, 2, 3, 12, 12, 12, 12, 12, 12, 12, 12]

i = 4
→ dp[4] = dp[3] + 1 = 4

dp = [0, 1, 2, 3, 4, 12, 12, 12, 12, 12, 12, 12]

i = 5
→ dp[5] = dp[4] + 1 = 5

dp = [0, 1, 2, 3, 4, 5, 12, 12, 12, 12, 12, 12]

i = 6
- 6 ≤ 6 → dp[6] = min(12, dp[0] + 1) = 1
- 1 ≤ 6 → dp[6] = min(1, dp[5] + 1) = min(1, 6) = 1

dp = [0, 1, 2, 3, 4, 5, 1, 12, 12, 12, 12, 12]

i = 7
→ dp[7] = min(dp[6] + 1 = 2)

dp = [0, 1, 2, 3, 4, 5, 1, 2, 12, 12, 12, 12]

i = 8
→ dp[8] = dp[7] + 1 = 3

dp = [0, 1, 2, 3, 4, 5, 1, 2, 3, 12, 12, 12]

i = 9
- 9 ≤ 9 → dp[9] = min(12, dp[0] + 1) = 1
- 6 ≤ 9 → dp[9] = min(1, dp[3] + 1 = 4) = 1

dp = [0, 1, 2, 3, 4, 5, 1, 2, 3, 1, 12, 12]

i = 10
→ dp[10] = dp[9] + 1 = 2

dp = [0, 1, 2, 3, 4, 5, 1, 2, 3, 1, 2, 12]

i = 11
→ dp[11] = dp[10] + 1 = 3

dp = [0, 1, 2, 3, 4, 5, 1, 2, 3, 1, 2, 3]

Amount	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
dp[]	0	1	2	3	4	5	1	2	3	1	2	3

Time Complexity:
- O(n * amount) where n is number of coins.

Space Complexity:
- O(amount) for dp and parent.

Final Answer:
dp[11] = 3
(We can make 11 using (9 + 1 + 1))

Fractional Knapsack Problem

26 May 2025 15:30

Greedy Approach (Optimal for fractional)
Idea:

- Compute **value per weight**
- Sort items by **value per weight** descending
- Pick full items till capacity is left
- Pick fraction of next item if needed

```
val[] = [60, 100, 120]
wt[] = [10, 20, 30]
capacity = 50
```

- Sort items by profit/weight ratio ↓: [Item 1, Item 2, Item 3] (descending)
- Take full Item 1 → cap=40, val=60
- Take full Item 2 → cap=20, val=60+100=160
- Take 20/30 of Item 3 → val += (2/3)*120 = 80

Total = 60 + 100 + 80 = 240.0

```
val = [500]
wt = [30]
capacity = 10
Value/Weight = 500/30 = 16.6667

→ take 10/30 of it:
→ value = (10/30)*500 = 166.6667
```

Min fib terms to sum upto K

30 May 2025 12:09

Key Idea:
Always subtract the **largest Fibonacci number ≤ k** from k, and repeat. and keep track of the count of numbers considered.
Because **Fibonacci numbers grow exponentially**, so **larger ones cover more value in fewer terms**.
You can always build any number using **minimum terms** by **always choosing the largest possible Fibonacci number at each step**.

1. Generate all Fibonacci numbers ≤ k
2. Start from the largest Fibonacci number
3. While k > 0:
   - Pick the largest Fibonacci number f ≤ k
   - Subtract f from k
   - Increment count
4. Return the count

- **Time:** O(n) where n = number of Fibonacci numbers ≤ k
- **Space:** O(n) for Fibonacci list

k = 7:
Fibonacci list: [1, 2, 3, 5]
Initially: i = 3 (points to 5)
First iteration:
- k = 7, fib[3] = 5
- 5 ≤ 7 → **okay**, skip the while

Now we subtract 5 → k = 2, count++
Second iteration:
- k = 2, fib[3] = 5 → too big
- Enter the while:
  - fib[3] = 5 > 2 → i-- → i = 2
  - fib[2] = 3 > 2 → i-- → i = 1
  - fib[1] = 2 ≤ 2 → stop

Now we subtract 2 → k = 0, count++
final count is 2

- while (i >= 0 && fib.get(i) > k) i--

- This **moves the pointer i to the largest Fibonacci number ≤ k.**
- You are starting from the **largest Fibonacci number** in the list (fib.get(i)), and moving left until you find one **that can be subtracted from k** (i.e., ≤ k).

Connect N ropes with a Minimum Cost

30 May 2025 12:28

Given an array **arr[]** of rope lengths, connect all ropes into a single rope with the minimum total cost. The cost to connect two ropes is the **sum** of their lengths.

Examples:

Input: *arr[] = [4, 3, 2, 6]*

Output: *29*

Explanation: *We can connect the ropes in following ways.*

1) First connect ropes of lengths 2 and 3. Which makes the array [4, 5, 6]. Cost of this operation 2 + 3 = 5.

2) Now connect ropes of lengths 4 and 5. Which makes the array [9, 6]. Cost of this operation 4 + 5 = 9.

3) Finally connect the two ropes and all ropes have connected. Cost of this operation 9 + 6 =15. Total cost is 5 + 9 + 15 = 29. This is the optimized cost for connecting ropes.

Other ways of connecting ropes would always have same or more cost. For example, if we connect 4 and 6 first (we get three rope of 3, 2 and 10), then connect 10 and 3 (we get two rope of 13 and 2). Finally we connect 13 and 2. Total cost in this way is 10 + 13 + 15 = 38.

Key Idea:

Combining the two smallest ropes early keeps the cost lower

1. *Push all rope lengths into a **Min-Heap***
2. *While the heap has more than one element:*
    - *Pop the two smallest ropes a and b*
    - *Combine them → new rope = a + b*
    - *Add the cost to total*
    - *Push new rope back into the heap*
3. *Return total cost*

- ***Time:*** *O(n log n)*
  Each insertion and extraction in heap takes O(log n), and we do this n-1 times.
- ***Space:*** *O(n) for priority queue*

[4, 3, 2, 6]
Initial Min-Heap: [2, 3, 4, 6]

- *Pop 2 + 3 = 5, cost = 5 → Heap: [4, 6, 5]*
- *Pop 4 + 5 = 9, cost = 9 → Heap: [6, 9]*
- *Pop 6 + 9 = 15, cost = 15 → Heap: [15]*

Total cost = 5 + 9 + 15 = 29

- ***Internally, the structure is a binary heap, not a sorted array***
- ***Ordering inside the heap (e.g., 4, 6, 5) is based on heap rules, not value order***

After inserting 4, 6, and 5 into a min-heap, it might look like:

```
      4
     / \
    6   5
```

Here:

- *4 is the smallest → root*
- *6 and 5 are children (still satisfies heap property)*

How Min-Heap Works Internally:

It stores elements in binary tree fashion with this rule:

- *Parent ≤ children*

So:

- *Heap: [4, 6, 5] is valid (root is 4)*
- *Heap: [4, 5, 6] is also valid*

Both satisfy the min-heap property.

Don’t expect a min-heap to be sorted.

Instead, always rely on:

- *poll() → gives smallest element*
- *peek() → gives current minimum*

Buy Maximum Stocks if i stocks can be bought on i-th day

30 May 2025 12:43

- You have prices of stocks for **N days**: arr[i] is price on day i (1-based).
- On day i, you can buy **at most i stocks.**
- You have k money in total.
- Buy **maximum number of stocks** with money k while respecting the day limits.

Input *: price[] = { 10, 7, 19 },*
k = 45.
Output *: 4*

A customer purchases 1 stock on day 1 for 10 rs,
2 stocks on day 2 for 7 rs each and 1 stock on day 3 for 19 rs.
Therefore total of
*10, 7 * 2 = 14 and 19 respectively. Hence,*
total amount is 10 + 14 + 19 = 43
Max number of stocks purchased is 4.

Key Idea (Greedy):

- *To maximize stocks,* ***buy the cheapest stocks first****.*
- *But you can buy* ***only up to i stocks on day i****.*
- *So sort days by price, then buy as many as you can on the cheapest days first.*

Input:

Day	Price
1	10
2	7
3	19

Money (k) = 45

Step 1: Sort days by price:

Day	Price
2	7
1	10
3	19

Step 2: Buy stocks day by day (cheapest first):

- Day 2 (price=7, max buy=2):
  - How many can we buy? Minimum of max stocks on day and what money allows.
  - Money allows: 45 / 7 = 6 (can buy up to 6 stocks if no limit)
  - Max stocks allowed on day 2: 2
  - so buy min(6,2)=2

    - So buy 2 stocks → cost = 2 * 7 = 14
    - Money left = 45 - 14 = 31
    - Stocks bought = 2
  - Day 1 (price=10, max buy=1):
    - Money allows: 31 / 10 = 3
    - Max stocks allowed on day 1: 1
    - Buy 1 stock → cost = 1 * 10 = 10
    - Money left = 31 - 10 = 21
    - Stocks bought so far = 2 + 1 = 3
  - Day 3 (price=19, max buy=3):
    - Money allows: 21 / 19 = 1
    - Max stocks allowed on day 3: 3
    - Buy 1 stock → cost = 1 * 19 = 19
    - Money left = 21 - 19 = 2
    - Stocks bought so far = 3 + 1 = 4

Result: maximum stocks = 4

Why sorting by price is important?
If you buy expensive stocks first, you'll run out of money quickly and buy fewer stocks. Buying cheaper stocks first lets you buy more stocks overall.

Algorithm

- Create an array of (price, day) pairs.
- Sort the pairs by **price ascending**.
- For each day in sorted order:
  - Calculate how many stocks you can buy: buy = min(day, k / price)
  - Update k -= buy * price
  - Add buy to total_stocks
- Return total_stocks

Complexity:

- Sorting: O(N log N)
- Buying stocks: O(N)
- Space: O(N) for auxiliary array

binary search with first and last occurrence

24 May 2025 14:26

Given an array of integers nums sorted in increasing order, find the starting and ending position of a given target value.

If target is not found in the array, return [-1, -1].

You must write an algorithm with O(log n) runtime complexity.

Example 1:
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
Example 2:
Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]

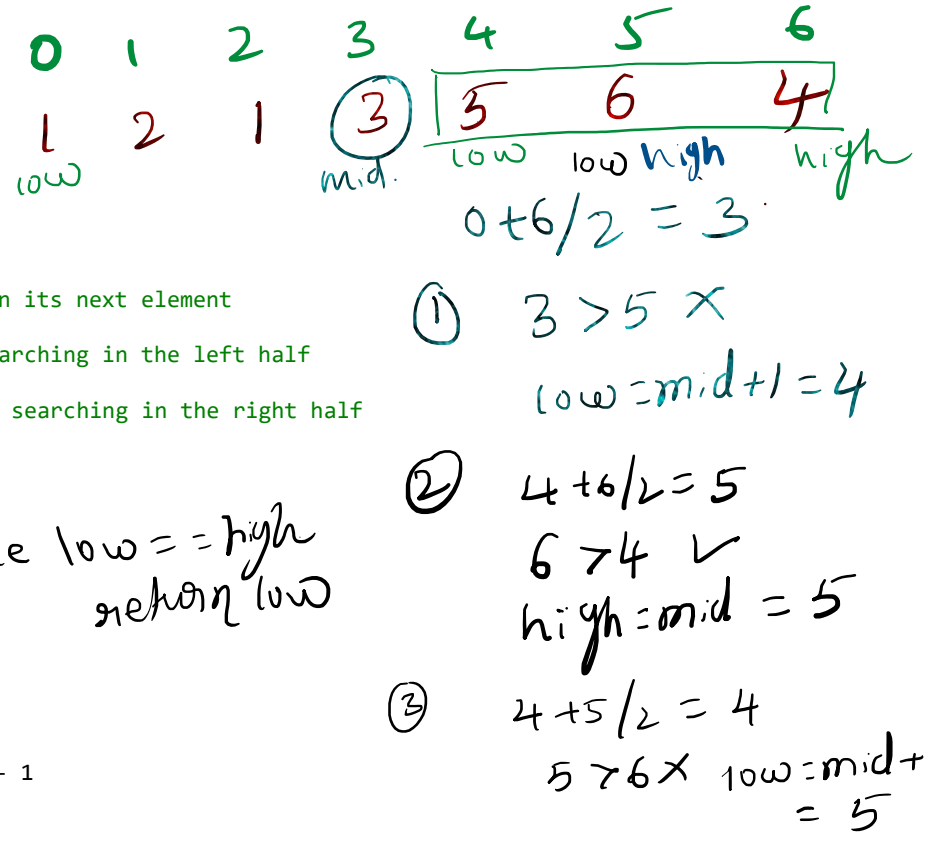
Peak Element

02 June 2025 12:58

A peak element is an element that is strictly greater than its neighbors.
Given a **0-indexed** integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.
You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.
You must write an algorithm that runs in O(log n) time.

Example 1:
Input: nums = [1,2,3,1]
Output: 2
Explanation: 3 is a peak element and your function should return the index number 2.
Example 2:
Input: nums = [1,2,1,3,5,6,4]
Output: 5
Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

```
int low = 0
int high = n - 1

    while (low < high) {
        int mid = (low + high) / 2;

        // Check if mid is greater than its next element
        if (nums[mid] > nums[mid + 1])
            high = mid  // Continue searching in the left half
         else
            low = mid + 1  // Continue searching in the right half


    return low;
```



In binary search for a value:
- If a[mid] > target, you do: high = mid - 1
- Because you know mid is not the answer

But in **peak search**:
- If a[mid] > a[mid + 1], it means you are in the **decreasing slope** of a **bitonic/peak region**
  → So the **peak might be at mid itself**, or somewhere to the **left**
  → Thus, you do: high = mid (not mid - 1)

Pivot Element

02 June 2025 12:58

```
Pivot element is the only element in input array which is smaller than it's
previous element.
A pivot element divides a sorted rotated array into two monotonically increasing
array.
Find such pivot element using recursive binary search.
input=7
4 5 6 7 1 2 3
output=Pivot element is: 1
*/

        int pivot = findPivot(arr, 0, n - 1);
        System.out.println("Pivot element is: " + pivot);



    public static int findPivot(int[] arr, int low, int high)
        if (low>high)
            return arr[0]   // Array not rotated, return first element

        if (high == low)
            return arr[low]  // Only one element left

        int mid = (low + high)

        // Check if mid+1 is the pivot
        if (mid < high && arr[mid] > arr[mid + 1])
            return arr[mid + 1]

        // Check if mid is the pivot
        if (mid > low && arr[mid] < arr[mid - 1])
            return arr[mid]


        if (arr[low] < arr[mid])
            return findPivot(arr, mid + 1, high) // Search in the right half
         else
            return findPivot(arr, low, mid - 1) // Search in the left half
```

aggressive cows

24 May 2025 14:26

You are given an array with unique elements of stalls[], which denote the position of a **stall**. You are also given an integer **k** which denotes the number of aggressive cows. Your task is to assign **stalls** to **k** cows such that the **minimum distance** between any two of them is the **maximum** possible.

Examples :

Input: stalls[] = [1, 2, 4, 8, 9], k = 3
Output: 3
Explanation: The first cow can be placed at stalls[0],
the second cow can be placed at stalls[2] and
the third cow can be placed at stalls[3].
The minimum distance between cows, in this case, is 3, which also is the largest among all possible ways.
Input: stalls[] = [10, 1, 2, 7, 5], k = 3
Output: 4
Explanation: The first cow can be placed at stalls[0],
the second cow can be placed at stalls[1] and
the third cow can be placed at stalls[4].
The minimum distance between cows, in this case, is 4, which also is the largest among all possible ways.

Input: stalls[] = [2, 12, 11, 3, 26, 7], k = 5
Output: 1
Explanation: Each cow can be placed in any of the stalls, as the no. of stalls are exactly equal to the number of cows.
The minimum distance between cows, in this case, is 1, which also is the largest among all possible ways.

Naive APproach

- try **all possible minimum distances d** from 1 up to the maximum possible distance (max(stalls) - min(stalls)).
- For each distance d, try to place all cows in stalls so that the distance between any two cows is at least d.
- The **largest d for which placement is possible** is the answer.

Time:
O(max_dist * n)
(since max_dist ranges from 1 to (max-min), and for each, we do a linear scan)

stalls = [1, 2, 4, 8, 9]
k = 3 (number of cows)

Sort first step: {1,2,4,8,9}
max_distance = stalls[max] - stalls[min] = 9 - 1 = 8

Step 2: Try each distance from 1 to 8 (inclusive)

Distance d	**Is it possible to place 3 cows with at least d?**	**How cows are placed?**	**Result**
1	Yes	Cows at [1, 2, 4]	Possible
2	Yes	Cows at [1, 4, 8]	Possible
3	Yes	Cows at [1, 4, 8]	Possible
4	Yes	Cows at [1 4, 8]4-1=3	not Possible
5	No	Try [1, 8, ?] (9-8=1 < 5 no)	Not Possible
6	No	Try [1, 8, ?] (9-8=1 < 6 no)	Not Possible
7	No	Try [1, 8, ?] (9-8=1 < 7 no)	Not Possible
8	No	Only 1 cow can be placed at 1	Not Possible

Approach 2 (Optimal - Binary Search on Answer):

1. **Sort the stall positions.**
2. **Binary Search**:
   - low = 1
   - high = max_distance = last - first
3. For a mid value (distance), **check if it's possible** to place k cows such that minimum distance between any two is at least mid.
4. If yes → try bigger distance.
   If no → try smaller.

```
Arrays.sort(stalls)

       int low = 1;
       int high = stalls[n - 1] - stalls[0]
       int ans = 0

       while (low <= high) {
           int mid = (low + high) / 2

           if (is_possible(stalls, n, k, mid))
               ans = mid;
               low = mid + 1;
           else
               high = mid - 1;

       }

            print(ans)
   }

 boolean is_possible(int[] stalls, int n, int cows, int dist) {
       int count = 1
       int last_pos = stalls[0]

       for (int i = 1; i < n; i++)
           if (stalls[i] - last_pos >= dist) {
               count++
               last_pos = stalls[i]
           }


       return count >= cows
   }
```

Sum of all Subarrays

05 November 2024 01:28

The total number of subarrays in an array of size N is N * (N + 1) / 2.
elements should be contigous.

(empty subarray is not allowed)
For an array [1, 2, 3], subarrays are: [1], [2], [3], [1, 2], [1, 2, 3], [2,3]

- [1] → 1
- [2] → 2
- [3] → 3
- [1, 2] → 1+2=3
- [2, 3] → 2+3=5
- [1, 2, 3] → 1+2+3=6

1+2+3+3+5+6=20

1 appears in subarrays: [1], [1, 2], [1, 2, 3]
Contribution of 1 = 1 * 3 = 3

2 appears in subarrays: [2], [1, 2], [2, 3], [1, 2, 3]
Contribution of 2 = 2 * 4 = 8

3 appears in subarrays: [3], [2, 3], [1, 2, 3]
Contribution of 3 = 3 * 3 = 9

Hence sum of all subarrays can be said as: 3 + 8 + 9 = 20.

```
n = arr.length
long totalsum = 0

for i from 0 to n - 1
    long res = arr[i] * (i + 1) * (n - i)
    totalsum = totalsum + res
end for

print totalsum
```

- (i + 1) → Number of starting indices from which a subarray that includes arr[i] can begin.
- (n - i) → Number of ending indices at which a subarray that includes arr[i] can end.

Let's take a small array:
arr = [3, 1, 2]
n = 3
All possible subarrays are:

```
[3]
[3, 1]
[3, 1, 2]
[1]
[1, 2]
[2]
And their sums:
3 + 4 + 6 + 1 + 3 + 2 = 19
So total sum = 19
```

Sum of all Subsequences(or subsets same)

05 November 2024 01:28

The total number of subsequences possible is equal to 2^n. (including empty subsequence)
elements need not to be contigous.

Note: only subsequence and subset can have empty array.)

A subsequence is formed by including or excluding each element, maintaining the order.
For array [1, 2]:

- Subsequences = [], [1], [2], [1,2]
- Sum of subsequences = 0 + 1 + 2 + (1+2) = 6

For an array [1, 2, 3], subsequences are:
[]
[1]
[2]
[3]
[1, 2]
[1, 3]
[2, 3]
[1, 2, 3]

Naive Approach

Use recursion or backtracking to explore all combinations (include/exclude)

```
import java.util.*;

public class Main {
    static int total = 0;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for(int i = 0; i < n; i++)
            a[i] = sc.nextInt();

        generate(0, 0, a, n);
        System.out.println(total);
    }

    static void generate(int i, int sum, int[] a, int n) {
        if(i == n) {
            total += sum;
            return;
        }
        generate(i + 1, sum + a[i], a, n);
        generate(i + 1, sum, a, n);
    }
}
```

In an array subsequence, each element appears in 2^n-1 subsequences.

first calculate array sum: which is 1+2+3=6
count of each element: 2^3-1=2^2=4

print(sum*countofeachelement) 6*4=24

```
sum = 0
for num in arr
    sum = sum + num


long res = (long) Math.pow(2, n - 1);
or
long res= 1L << (n-1))

print (sum * res)
```

```
import java.util.*;

public class subset_sum_of_all_subsets {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        int total_sum = 0;
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
            total_sum += a[i];
        }
        int power = 1;
        for (int i = 1; i < n; i++) {
            power *= 2;
        }
        int ans = power * total_sum;
        System.out.println(ans);
    }
}
```

The **only difference between subsets and subsequences** is the **order and position** of elements.

Characteristic	**Subset**	**Subsequence**
Order of Elements	The order of elements does not matter. A subset consists of any **combination** of elements chosen from the original set, regardless of their position. Example: For set s = {1, 2, 3}, subset {3, 2} is equivalent to {2, 3}.	The elements must appear in the **same** relative order as in the original sequence, though they do not need to be **contiguous**. Example: For sequence s = {1, 2, 3}, subsequence {3, 2} is **invalid**, but {1, 3} or {2, 3} are valid.
Relationship with Original Set/Sequence	Defined in the context of sets, duplicates are irrelevant.	Defined in the context of sequences, duplicates are preserved.
Size and Selection	Formed by selecting **any combination** of elements from the set.	Formed by **deleting specific elements** without rearranging the rest.
Contextual Usage	Primarily used in **set theory** and **combinatorics**.	Primarily used in **algorithms, string matching, and sequence alignment**.
Example	For set s = {1, 2, 3}, subset {3,2} is valid because order does not matter.	For subsequence s = {1, 2, 3}, subsequence {3, 2} is invalid because order is not preserved.

Longest consecutive subsequence

03 June 2025 14:43

Given an array of n integers, find the **length of the longest subsequence** of consecutive integers (they **can appear in any order**).

- Duplicates and order don’t matter.
- Subsequence (not necessarily contiguous in the array).

Input: arr = [2, 6, 1, 9, 4, 5, 3]
Output: 6
Explanation: The subsequence is [1, 2, 3, 4, 5, 6]

Approach 1: Brute Force
Idea:
For each element, try to find the **next consecutive number** (x+1, x+2, ...) by checking if it's in the array.
Steps:
1. For every element, keep increasing and check if element+1, element+2... exist in array.
2. Count how far you can go.

1. Initialize variables

- max_len = 1 → keeps track of longest subsequence length

2. Outer loop (i = 0 to n-1)
Start with each element arr[i] and assume it to be the **starting point** of a potential sequence.

- Let x = arr[i]
- Initialize count = 1 to count this as the first element in sequence

3. Inner loop (j = 0 to n-1)
Search for x+1 in the array:

- If arr[j] == x+1, it means next number in sequence is found
- Then:
  - Increment count
  - Update x = x + 1
  - Reset j = -1 to **restart** search for the next x+1

4. Update the maximum length
After the inner loop ends:

- If count > max_len, update max_len = count

5. Repeat outer loop for all i
6. Return max_len

Visual Trace for Input: [2, 6, 1, 9, 4, 5, 3]

i	arr[i]	Found Sequence	Count
0	2	2, 3, 4, 5, 6	5
1	6	6	1
2	1	1, 2, 3, 4, 5, 6	6 ☑
3	9	9	1
4	4	4, 5, 6	3
5	5	5, 6	2
6	3	3, 4, 5, 6	4

Time: O(n^2)
Space: O(1)

Approach	Time	Space	Remarks
Brute	O(n²)	O(1)	Slow for large input
Sorting	O(n log n)	O(1)	Better, but needs sorting
HashSet	O(n)	O(n)	Best in time, needs space

approach 2 using sorting

Idea:

- If we sort the array, all consecutive elements will appear **next to each other**.
- We can then iterate through the array and count the length of consecutive increasing numbers.
- Sorting puts numbers in order.
- Skip **duplicates** — they shouldn't reset the count.
- If arr[i] == arr[i-1] + 1, it's part of the same sequence.
- If arr[i] != arr[i-1] + 1, the sequence is **broken** → reset count.

- **Sort the array**
  - So that consecutive numbers are adjacent.
- **Initialize variables**
  - curr_len = 1 → Current sequence length
  - max_len = 1 → Max sequence length
- **Loop from index 1 to n-1**
  - If arr[i] == arr[i-1] + 1 → consecutive → increment curr_len
  - If arr[i] == arr[i-1] → duplicate → skip (no change)
  - Else → not consecutive → reset curr_len = 1
- **Update max length after each step**
  - max_len = max(max_len, curr_len)
- **Return max_len**

[2, 6, 1, 9, 4, 5, 3]

- Sorted: [1, 2, 3, 4, 5, 6, 9]

i	arr[i]	arr[i-1]	Relation	curr_len	max_len
1	2	1	+1	2	2
2	3	2	+1	3	3
3	4	3	+1	4	4
4	5	4	+1	5	5
5	6	5	+1	6	6
6	9	6	not +1	1	6

approach3: optimized using hashset

- For each number, check **only if it's the start** of a sequence (i.e., num - 1 is **not** in the set).
- Then, expand the sequence forward (num + 1, num + 2, ...) until it's broken.
- Keep track of the longest such sequence.
- Every element is inserted and removed **at most once** in the set.
- No sorting or nested loops.
- **Time: O(n), Space: O(n)**

- **Add all elements to a HashSet**
  - So we can check existence in O(1) time.
- **Initialize max_len = 0**
- **Loop over each number num in the array**:
  - If num - 1 is **not** in the set → num is the **start of a new sequence**
  - Set current = num
  - Count how many current + 1, current + 2, etc. are in the set
  - Track length of this sequence using count
- **Update max_len with the maximum of current and previous max**
- **Return max_len**

Loop Over Each Element and Apply Logic

➤ **i = 0, arr[i] = 2**

- Is 2 - 1 = 1 in the set? → YES
- → So 2 is NOT the start of a sequence → skip

➤ **i = 1, arr[i] = 6**

- Is 6 - 1 = 5 in the set? → YES
- → Not a start → skip

➤ **i = 2, arr[i] = 1**

- Is 1 - 1 = 0 in the set? → NO

- → So 1 **is the start** of a new sequence

current = 1
count = 1
Loop and check for current + 1 while it’s in the set:

Step	current	Exists in set?	count
1	2	yes	2
2	3	yes	3
3	4	yes	4
4	5	yes	5
5	6	yes	6
6	7	NO	stop

☑ So, this sequence: [1, 2, 3, 4, 5, 6]
Update:

max_len = max(0, 6) = 6

Maximum Subarray Sum (Kadane's Algorithm)

05 November 2024 01:25

```
/*
Given an array of integers (can be positive, negative, or zero), find the
contiguous subarray which has the maximum sum, and return that sum.

Input: arr[] = {2, 3, -8, 7, -1, 2, 3}
Output: 11
Explanation: The subarray {7, -1, 2, 3} has the largest sum 11. */
```

Naive Approach

- Initialize res = -∞ (or Integer.MIN_VALUE in Java)
- Loop i from 0 to n - 1:
  - Initialize sum = 0
  - Loop j from i to n - 1:
    - Add arr[j] to sum
    - Update res = max(res, sum)
- After both loops, res will contain the maximum subarray su

Optimal Approach: Kadane’s Algorithm

The idea of Kadane's algorithm is to traverse over the array from left to right and for each element, find the maximum sum among all subarrays ending at that element. The result will be the maximum of all these values.

- Time: O(n)
- Space: O(1)

Idea:
While traversing, keep a running curr_sum.

- If curr_sum becomes negative, reset it to 0.
- Track the max seen so far using max_sum.

```
n = scanner.nextInt()

for i from 0 to n - 1
    arr[i] = scanner.nextInt()
end for

res = arr[0]
temp = arr[0]
```

```
for i from 1 to n - 1
    temp = max(arr[i], temp + arr[i])
    res = max(res, temp)


output res
```

```
Input: {2, 3, -8, 7, -1, 2, 3}
i    arr[i] curr_sum max_sum
0    2      2        2
1    3      5        5
2    -8     -3       5
3    7      7        7
4    -1     6        7
5    2      8        8
6    3      11       11
```

☑ Final Answer = 11

```
Index:    0   1   2   3   4   5   6
Array:   [2,  3, -8,  7, -1,  2,  3]
                     ^^^^^^^^^^^^^
                Max Subarray → Sum = 11
```

subarray with 0 sum

03 June 2025 13:01

Given an array of **positive and negative integers**, check if **there exists a subarray** (of size ≥ 1) **whose sum is 0.**

☑ **Sample Inputs:**
1. {4, 2, -3, 1, 6} → ✓ true → subarray [2, -3, 1]
2. {4, 2, 0, 1, 6} → ✓ true → element 0 itself
3. {-3, 2, 3, 1, 6} → ❌ false

Approach 1: Brute Force (Naive)

Check **sum of all subarrays**. If any subarray sum = 0, return true.

```
 boolean found = false;
        for(int i = 0; i < n; i++) {
            int sum = 0;
            for(int j = i; j < n; j++) {
                sum += a[j];
                if(sum == 0) {
                    found = true;
                    break;
                }
            }
            if(found)
                break;
        }
        System.out.println(found);
```

Time: O(n²), Space: O(1)

Prefix Sum + HashSet (Optimal)
A prefix sum, also known as a cumulative sum, is an array where each element represents the sum of all previous elements in the original array, up to and including that element. It's a powerful technique for efficiently calculating the sum of subarrays within an array.

If at any point the **prefix sum repeats**, or is 0, then a subarray with 0 sum exists.
Reason:
If prefix_sum[j] == prefix_sum[i], then subarray from (i+1 to j) has sum 0.

Time: O(n)
Space: O(n)

{4, 2, -3, 1, 6}

i	a[i]	prefix sum	seen	prefix already seen?
0	4	4	{4}	No
1	2	6	{4, 6}	No
2	-3	3	{4, 6, 3}	No
3	1	4	4 already seen	✓ Yes → return true

→ So subarray [2, -3, 1] has sum 0. (only existence we are checking we cant verify through logic)

Prime number problems

04 June 2025 14:45

Print prime numbers upto given N
Print prime numbers upto given count
Print prime numbers from given range

sieve of eratosthenesis

24 May 2025 14:26

- **Create a boolean array prime[0...n]**
  - Initialize all values as true
  - prime[i] = true means we initially assume i is prime
- **Set prime[0] = false and prime[1] = false**
  - Because 0 and 1 are **not** prime
- **Loop from p = 2 to √n**
  - If prime[p] == true:
    - It means p is a prime number
    - Mark all multiples of p as **not prime**
      (i.e., set prime[i] = false for all i = p*p, p*p+p, ..., ≤ n)
- **After the loop**, all indexes i where prime[i] == true are prime numbers

```
prime[] = [F, F, T, T, T, T, T, T, T, T, T]
Index      0  1  2  3  4  5  6  7  8  9 10
p = 2: mark 4, 6, 8, 10 as false

p = 3: mark 9 as false

Final:

prime[] = [F, F, T, T, F, T, F, T, F, F, F]
Primes:          2  3     5     7
```

Pivot Index Prefix Sum

02 June 2025 13:32

The **efficient algorithm** avoids recomputing left and right sums every time by:

- Precomputing the **total sum**.
- Tracking the **left sum** while iterating.
- Then computing the **right sum** using:

  right_sum = total - left_sum - nums[i]

  This is mathematically correct because:

  left_sum + nums[i] + right_sum = total

  → right_sum = total - left_sum - nums[i]

  So, instead of recomputing left and right sums for every index, we use just a running left sum and a constant-time expression for right sum.

  Naive Approach

  For each index i, calculate:
  - left_sum = sum from index 0 to i-1
  - right_sum = sum from index i+1 to n-1

```
for (int i = 0; i < n; i++) {
    int left_sum = 0;
    for (int j = 0; j < i; j++)
        left_sum += nums[j];

    int right_sum = 0;
    for (int j = i + 1; j < n; j++)
        right_sum += nums[j];

    if (left_sum == right_sum)
        return i;
}
return -1;
```

  Optimized Approach

```
int total = 0;
        for(int i = 0; i < n; i++)
            total += nums[i]

        int left_sum = 0
        for(int i = 0; i < n; i++)
            int right_sum = total - left_sum - nums[i];
            if(left_sum == right_sum) {
                System.out.println(i)
                return
            }
            left_sum += nums[i]
        }
```

```
        System.out.println(-1)
    }
```

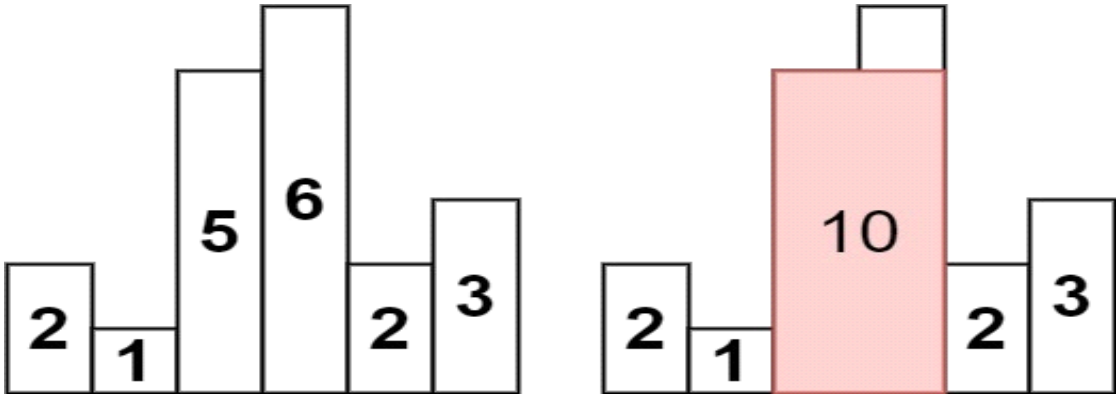
Input: nums = [1, 7, 3, 6, 5, 6]
Expected Output: 3

i	nums[i]	left_sum	right_sum = total - left_sum - nums[i]	left_sum == right_sum
0	1	0	28 - 0 - 1 = 27	no
		left_sum = 0 + 1 = 1		
1	7	1	28 - 1 - 7 = 20	no
		left_sum = 1 + 7 = 8		
2	3	8	28 - 8 - 3 = 17	no
		left_sum = 8 + 3 = 11		
3	6	11	28 - 11 - 6 = 11	YES, return index 3

Largest Rectangle in Histogram

03 June 2025 10:11

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram*.



Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

1. Naive Approach – Brute Force

Time O(n^2) → two nested loops for each bar (in worst case)

Space O(1)

For each bar:

- Expand **left** and **right** as far as the bars are **greater than or equal** to the current bar's height.
- Compute area = height of current bar × width
- Keep track of the **maximum** area found

- The current bar of height heights[i] can only be part of a rectangle as long as **all bars to the left and right** are **greater than or equal** to it.
- If any bar is **shorter**, then it breaks the rectangle.

If you're at index i, and heights[i] = 5,
then you can include as many consecutive bars on the **left and right** that are ≥ 5.

Index: 0 1 2 3 4 5
heights = [2, 1, 5, 6, 2, 3]
Let's choose i = 2, so:

heights[2] = 5

▶ Step 1: Expand Left
Start at i = 2, move left:

i = 1 → heights[1] = 1 < 5 → STOP

So left boundary = 2

▶ Step 2: Expand Right
Start at i = 2, move right:

i = 3 → heights[3] = 6 ≥ 5 → OK

i = 4 → heights[4] = 2 < 5 → STOP

So right boundary = 3
▨ Rectangle formed:
Bars: [5, 6]
Indices: 2 3
Height: 5
Width: 2 → (3 - 2 + 1)
Area: 5 × 2 = 10

Optimized Approach

- for **each bar**, we want to find:
  - The **first smaller bar on the left** → tells us how far we can go left
  - The **first smaller bar on the right** → tells us how far we can go right
- Using these two, we compute:

width = right_index - left_index - 1
area = height * width

- To do this efficiently in **O(n)** time, we use a **monotonic increasing stack.**
-

Instead of checking for left and right for each bar separately (which is brute force), we use a **Monotonic Stack** to do this in one pass:
Stack maintains indices of increasing heights.
Why increasing?

- So, when we encounter a **lower height**, we know:
  - We've found the **right boundary** for taller bars.
  - We can now **calculate area** for those taller bars that can't go further.

Let's say heights = [2,1,5,6,2,3].
1. While iterating from **left to right**:
   - Keep pushing index into stack **if height increases.**
   - If height **decreases**, pop from stack and compute:
     - **height** = height of bar at popped index.
     - **width** = current index − index of new stack top − 1
2. After iterating, clean up remaining stack elements.

Step 1: Initialization

- Let stack be an empty stack storing **indices** of bars.
- Let max_area = 0

Step 2: Iterate over the heights
For each i from 0 to n-1:

1. While the **stack is not empty** and heights[i] < heights[stack.peek()]:
    - It means the current bar breaks the increasing trend.
    - Pop the top index from the stack → top = stack.pop()
    - Compute height = heights[top]
    - Compute width:
        - If stack is empty: width = i
        - Else: width = i - stack.peek() - 1
    - Compute area = height × width
    - Update max_area
2. Push current index i into the stack.

◇ **Step 3: Clean up remaining stack**

After reaching end of the array (i == n), some bars might still be in the stack.
While stack is not empty:

- Pop top = stack.pop()
- Compute height = heights[top]
- Compute width:
    - If stack is empty: width = n
    - Else: width = n - stack.peek() - 1
- Compute area = height × width
- Update max_area

Stack = index positions

i = 0

- h[0] = 2 → stack is empty → push 0
  stack = [0]

i = 1

- h[1] = 1 < h[stack.peek()] → pop
  → top = 0, h[0] = 2, width = 1
  → area = 2×1 = 2
  max_area = 2, push 1
  stack = [1]

i = 2

- h[2] = 5 ≥ h[1] → push
  stack = [1, 2]

i = 3

- h[3] = 6 ≥ h[2] → push
  stack = [1, 2, 3]

i = 4

- h[4] = 2 < h[3]
  → pop 3 → height = 6, width = 1 → area = 6
  → pop 2 → height = 5, width = 2 → area = 10
  max_area = 10, push 4
  stack = [1, 4]

and so on.

Time Complexity = O(n)

- n pushes + n pops = O(n)
- Everything else inside the loop is constant time.
- Stack stores at most n indices → **O(n)** space in worst case (e.g., increasing array)

Kth Smallest element using Priority queue (max heap)

04 June 2025 15:29

Given an n x n matrix where each of the rows and columns is sorted in **ascending** order, and an integer k, return the **kth smallest** element in the matrix.

Sample input=
1 5 9
10 11 13
12 13 15
k = 8

Sample output=
13

Naive approach

Idea:
Flatten the matrix into a 1D array, sort it, and return the k-th element.

- **Time Complexity:**
  - Flattening: O(n^2)
  - Sorting: O(nlogn)
  - Overall: **O(n^2 log n)**
- **Space Complexity: O(n^2)**

Better Approach (Using max Heap)

WHY Max Heap?

- You want to keep the **smallest k elements** from the matrix.
- In order to do that, you:
  - Insert each element into a max heap (so the **largest among the k smallest is on top**)
  - If the heap size exceeds k, **remove the largest** – i.e., discard the one that **can't be among the k smallest.**
- At the end, the largest element in the heap (the top) is **the k-th smallest overall.**

- Traverse the entire matrix (n x n elements).
- For each element:
  - Push into a **max heap**
  - If heap size exceeds k, **remove the top (maximum)** element.
- After traversing all n² elements:
  - Heap contains **exactly the k smallest elements** from the matrix.
  - **Top of heap = k-th smallest**

Given matrix:
10 20 30 40
15 25 35 45
24 29 37 48
32 33 39 50
And k = 3
We want to find the **3rd smallest element.**

output=20

☑ **Step-by-Step Max Heap Simulation**

We initialize:

- A **max heap** (priority queue with reverse order)
- Insert elements one by one
- If size > k, remove the **largest element** from heap

Heap state after each insertion:

Step	Inserted	Heap (top is largest)	Explanation
1	10	[10]	size ≤ k
2	20	[20, 10]	size ≤ k
3	30	[30, 10, 20]	size = k
4	40	[30, 10, 20]	remove 40 → not inserted
5	15	[20, 10, 15]	remove 30 → 30 > 15
6	25	[20, 10, 15]	remove 25 → 25 > 20
7	35	[20, 10, 15]	remove 35 → 35 > 20
8	45	[20, 10, 15]	remove 45 → 45 > 20
9	24	[20, 10, 15]	remove 24 → 24 > 20
10	29	[20, 10, 15]	remove 29 → 29 > 20
11	37	[20, 10, 15]	remove 37 → 37 > 20
12	48	[20, 10, 15]	remove 48 → 48 > 20
13	32	[20, 10, 15]	remove 32 → 32 > 20
14	33	[20, 10, 15]	remove 33 → 33 > 20
15	39	[20, 10, 15]	remove 39 → 39 > 20
16	50	[20, 10, 15]	remove 50 → 50 > 20

☑ **Final Heap:**

[20, 10, 15]

- Top of heap (largest among k smallest) = **20**
- So, the **3rd smallest element is 20**

☑ **Diagram of Heap Evolution**

Let’s visualize heap as a **binary tree (max heap)** at each main step:

```
After inserting 10, 20, 30:
       30
      /  \
    10   20
Insert 40:
       30
      /  \
    10   20
→ remove 40 → ignored

Insert 15:
       20
      /  \
    10   15
→ 30 is removed to keep size = k
```

Heap always keeps the **k smallest,** and throws away anything bigger than current top.

Kth smallest using Binary Search

05 June 2025 11:38

10 20 30 40
15 25 35 45
24 29 37 48
32 33 39 50
Let k = 3

☑ Step 1: Initial range

low = matrix[0][0] = 10
high = matrix[3][3] = 50
We binary search on this value range [10, 50]

☑ Step 2: Binary Search Loop
➤ Iteration 1:

mid = (10 + 50) / 2 = 30
Now count how many elements ≤ 30:

Start from bottom-left: (row = 3, col = 0)
→ matrix[3][0] = 32 > 30 → move up
→ matrix[2][0] = 24 ≤ 30 → count += 3 (rows 0 to 2) → col++
→ matrix[2][1] = 29 ≤ 30 → count += 3 → col++
→ matrix[2][2] = 37 > 30 → move up
→ matrix[1][2] = 35 > 30 → move up
→ matrix[0][2] = 30 ≤ 30 → count += 1 → col++
→ matrix[0][3] = 40 > 30 → move up → out of bounds

☑ Total count = 3 + 3 + 1 = 7
→ 7 ≥ 3 → So 30 might be the answer
→ Update: high = 30

➤ Iteration 2:
i
mid = (10 + 30) / 2 = 20
Count elements ≤ 20:

Start at (3,0) = 32 > 20 → move up
(2,0) = 24 > 20 → move up
(1,0) = 15 ≤ 20 → count += 2 (rows 0,1) → col++
(1,1) = 25 > 20 → move up
(0,1) = 20 ≤ 20 → count += 1 → col++
(0,2) = 30 > 20 → move up → out of bounds

☑ Total count = 2 + 1 = 3
→ 3 ≥ 3 → 20 might be the answer
→ Update: high = 20

Iteration 3:
mid = (10 + 20) / 2 = 15
Count elements ≤ 15:
Start at (3,0) = 32 > 15 → up
(2,0) = 24 > 15 → up
(1,0) = 15 ≤ 15 → count += 2 → col++
(1,1) = 25 > 15 → up
(0,1) = 20 > 15 → up → out of bounds
☑ Total count = 2
→ 2 < 3 → Need more → low= mid+1= 16

Initial range: [10 ..... 50]

→ mid = 30 → count = 7<3 false → high = 30
→ mid = 20 → count = 3<3 false → high = 20
→ mid = 15 → count = 2<3 true → low = 16
→ mid = 18 → count = 2<3 true → low = 19
→ mid = 19 → count = 2<3 true → low = 20

Now low == high → stop.

return low as the kth smallest element

- **Time:** O(n * log(max - min))
  - log(max - min) = binary search steps on value range
  - Each step takes O(n) to count elements ≤ mid
- **Space:** O(1) → constant

Matrix Value	Compared to mid	Action	Why?
≤ mid	Add row+1 to count	Move right (col++)	All above are ≤ mid
> mid	Do not count	Move up (row--)	Need smaller values

count < k
- This means **there are fewer than k elements** that are ≤ mid.
- So, mid is **too small** to be the k-th smallest.
- We need **larger** values.
- Hence, do: low = mid + 1

- If count >= k, we **can't discard mid**
- But we want to **try to find a smaller value that still gives us ≥ k elements**

- Hence we do: high = mid

Condition	Meaning	Action
count < k	Not enough small elements	Go right → low = mid + 1
count >= k	Enough small elements (maybe too many)	Go left → high = mid

- The binary search runs on the value range:
  [min_element, max_element] = [matrix[0][0], matrix[n-1][n-1]]
- So the number of iterations is:
  $\log_2$(max - min)

- Each time we check a mid value
- So the binary search cuts the **value range,** not the index space

Let’s denote this as:
- d = max_element - min_element
  → **Binary Search Iterations = O(log d)**

2. Counting Elements ≤ mid
- For each mid, we start from **bottom-left** (row = n-1, col = 0)
- At most, we take n steps in rows and n steps in columns
- So **each count operation is O(n)**

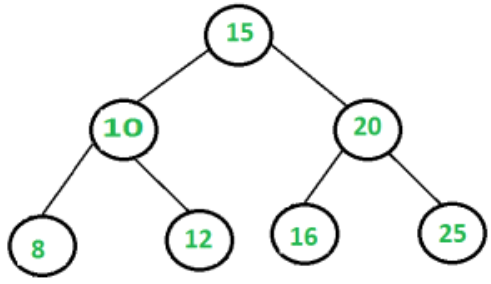
Final Time Complexity:
O(n * log(max - min))
Where:
- n = number of rows (or columns)
- max = largest element in matrix
- min = smallest element in matrix

Space Complexity
- We use **no extra data structures**
- Only a few pointers and variables

Space = O(1)

sum of all nodes in Binary Tree

04 June 2025 15:17



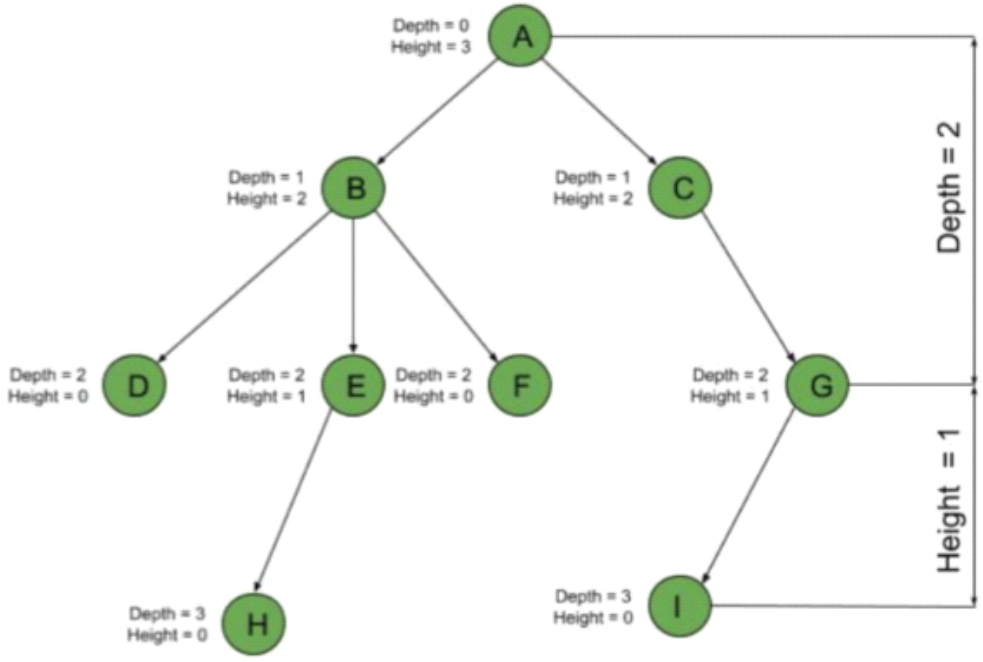
Output=106

```
 int compute_sum(Node root) {
        if (root == null) return 0;
        return root.data + compute_sum(root.left) + compute_sum(root.right)
    }
```

```
if tree is skewed stack approach is better

Stack<Node> stack = new Stack<>()
stack.push(root)
while (!stack.isEmpty()) {
    Node node = stack.pop()
    sum += node.data
    if (node.right != null) stack.push(node.right)
    if (node.left != null) stack.push(node.left)
}
```

The depth of a node is the number of edges present in path from the root node of a tree to that node.

The height of a node is the number of edges present in the longest path connecting that node to a leaf node.

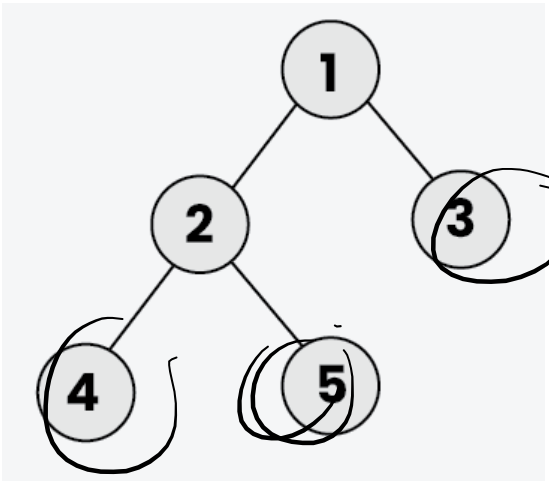


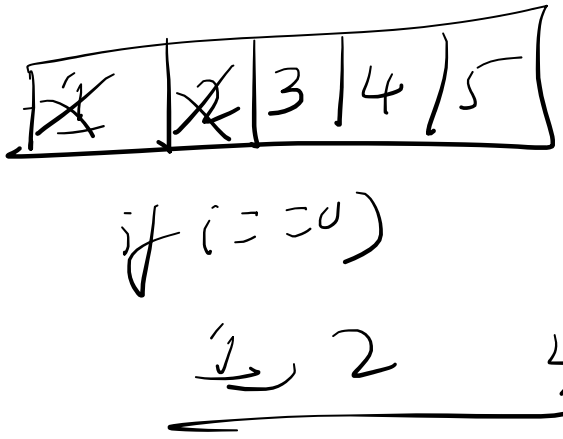
left view of b tree

03 July 2025 15:45

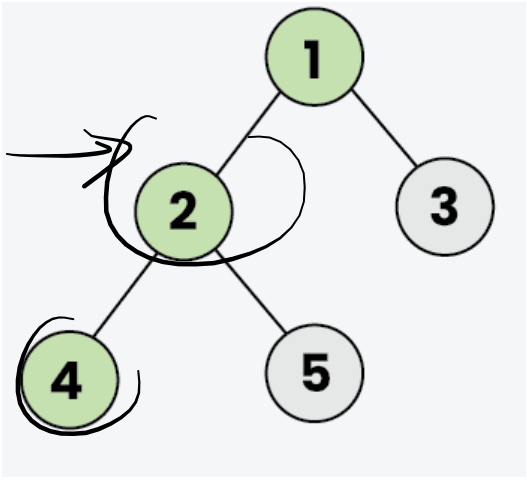
You are given the **root** of a binary tree. Your task is to return the ***left view*** of the binary tree. The ***left view*** of a binary tree is the set of nodes visible when the tree is **viewed** from the **left side**.
If the tree is empty, return an **empty list**.

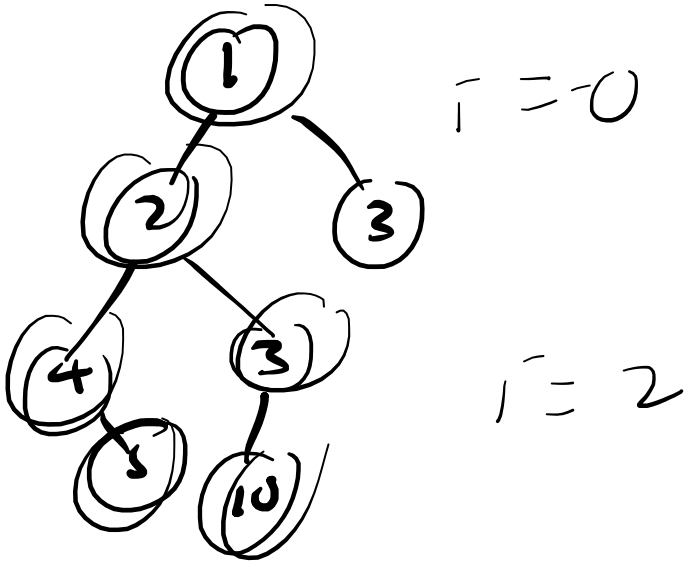
Input: root[] = [1, 2, 3, 4, 5, N, N]





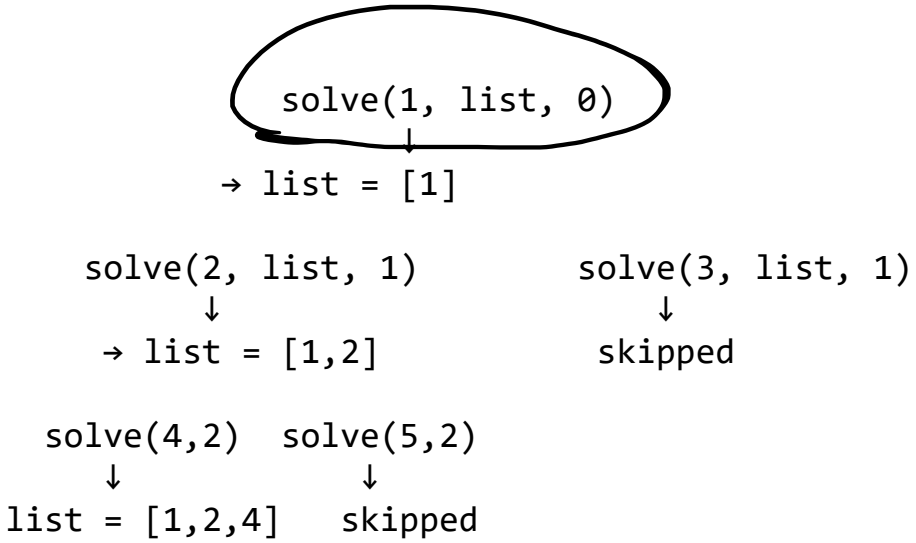
Output: [1, 2, 4]
Explanation: From the left side of the tree, only the nodes 1, 2, and 4 are visible.





- **Initialize** an empty list ans to store the result (left view).
- Start a **recursive function** solve(rootnode, level=0):
  - If node is null, return.
  - If ans.size() == level, it means **this is the first node at this level**, so:
    - Add node.data to ans.
- **Recur to the left child** first: solve(node.left, level + 1)

- **Then recur to the right child**: solve(node.right, level + 1)
- After recursion completes, return ans.

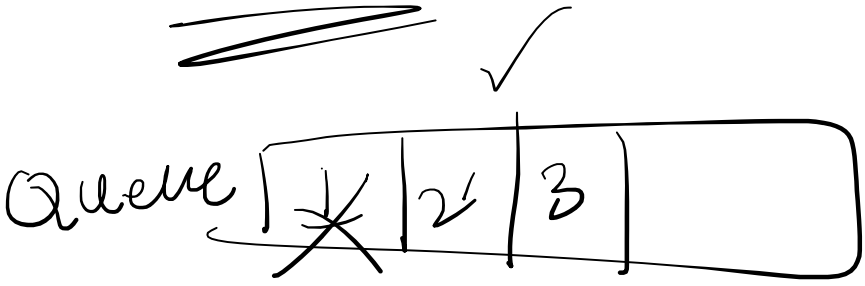


if(list.size() == level)

add node to the list

Solve(node.left, list, level+1)
Solve(node.right, list level+1)

BFS



While(empty)
{
int S = q.size

) for (0 to 5)

node t temp = q.poll()

if(temp.left != null)

q.add(t.left)

if(q.add(t.right)

if(i==0)

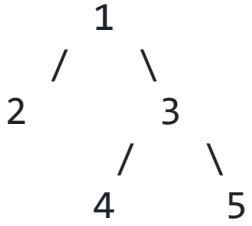
right view of b tree

04 June 2025 15:17

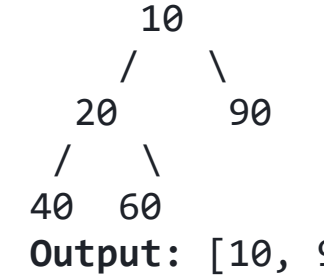
Given a Binary Tree, Your task is to return the values visible from **Right view** of it.

Right view of a Binary Tree is set of nodes visible when tree is viewed from **right** side.

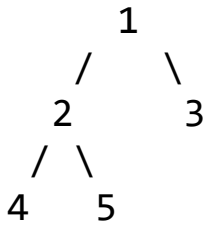
Examples :
Input: root = [1, 2, 3, 4, 5]



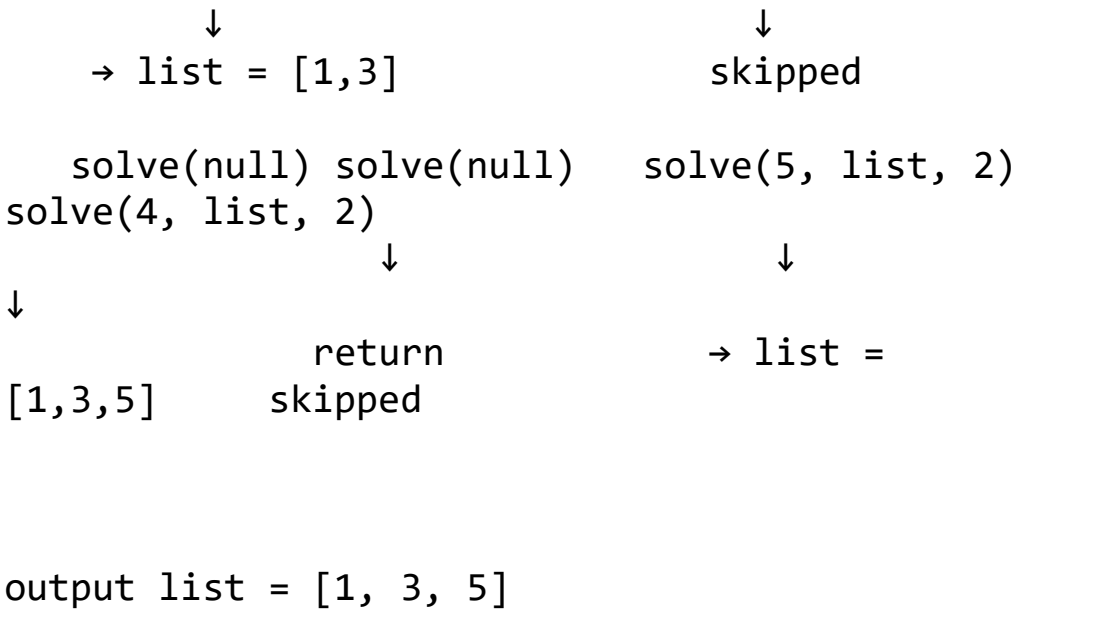
Output: [1, 3, 5]
Input: root = [10, 20, 90, 40, 60]



Output: [10, 90, 60]



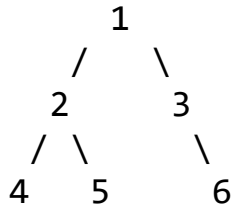
```
        solve(1, list, 0)
                ↓
          → list = [1]

    solve(3, list, 1)        solve(2, list, 1)
```

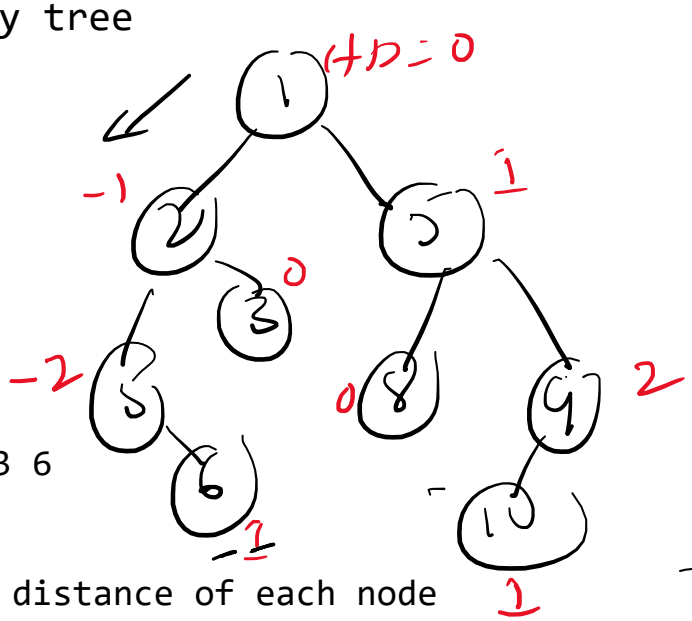


output list = [1, 3, 5]

top view of the binary tree

04 July 2025 00:15



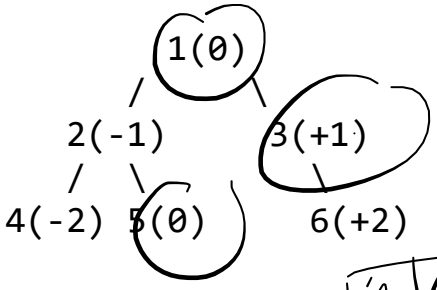


Top View Output: 4 2 1 3 6

step 1: find horizontal distance of each node

HD means **how far a node is from the root**, left or right:

- Root node → HD = 0
- Go left → HD - 1
- Go right → HD + 1



step 2:

Traverse the Tree

- Start with root at HD = 0
- Add it to the queue
- For each node:
  - If ~~its~~ HD is **not in the map**, add it (means first time we're visiting this column)
  - Add left child to queue with HD - 1
  - Add right child to queue with HD + 1

step 3:

Once the traversal is done:

- The map has one node per HD
- We **sort the HDs from smallest to largest**
- Print the node data in that order

e

```
class Solution {
    static class Pair {
        Node node;
        int hd;
        Pair(Node n, int h) {
            node = n;
            hd = h;
        }
    }



static ArrayList<Integer> topView(Node root) {
        ArrayList<Integer> result = new ArrayList<>();
        if (root == null) return result;

        TreeMap<Integer, Integer> map = new TreeMap<>();
        Queue<Pair> q = new LinkedList<>();
        q.add(new Pair(root, 0));

        while (!q.isEmpty()) {
            Pair temp = q.poll();
            int hd = temp.hd;
            Node curr = temp.node;

            if (!map.containsKey(hd)) {
                map.put(hd, curr.data);
            }

            if (curr.left != null)
               q.add(new Pair(curr.left, hd - 1));
            if (curr.right != null)
               q.add(new Pair(curr.right, hd + 1));
        }

        for (int val : map.values()) {
            result.add(val);
        }
        return result;
    }
}
```

Check BST

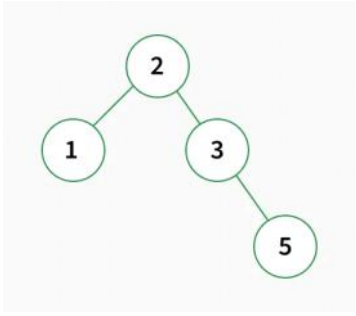
04 June 2025 15:20

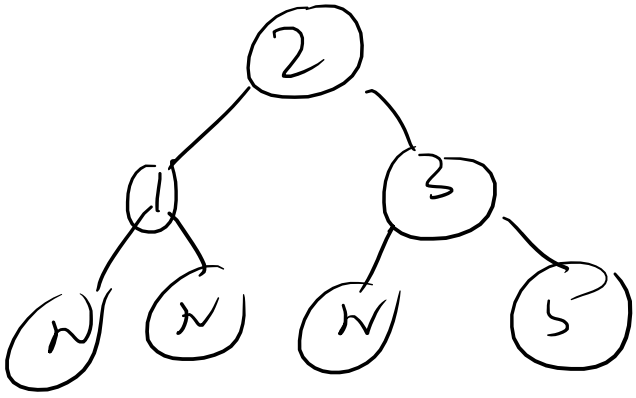
(

Given the root of a binary tree. Check whether it is a BST or not.
Note: We are considering that BSTs can not contain duplicate Nodes.
A **BST** is defined as follows:

- The left subtree of a node contains only nodes with keys **less than** the node's key.
- The right subtree of a node contains only nodes with keys **greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

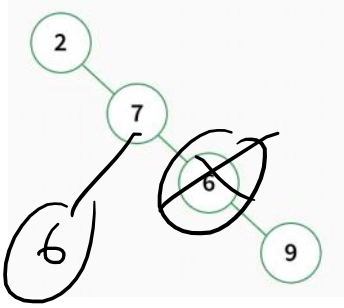
Input: root = [2, 1, 3, N, N, N, 5]





Output: true
Explanation: The left subtree of every node contains smaller keys and right subtree of every node contains greater keys. Hence, the tree is a BST.

Input: root = [2, N, 7, N, 6, N, 9]



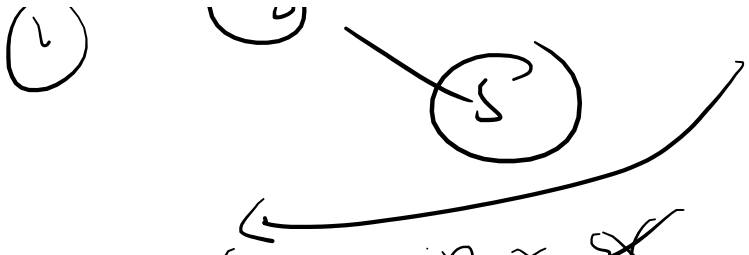
Output: false
Explanation: Since the node t

```
boolean isBST(Node root) {

        if(root == null)
        {
            return true
        }
        if(root.left != null && root.data < findmax(root.left).data)
```

```
        {
            return false
        }
        if(root.right != null && root.data > findmin(root.right).data)
        {
            return false
        }

        return (isBST(root.left) && isBST(root.right))
    }

    Node findmin(Node root)
    {
        if(root == null)
        {
           return null
        }

        while(root.left != null)
        {
            root = root.left
        }
        return root
    }
     Node findmax(Node root)
    {
        if(root == null)
        {
           return null
        }

        while(root.right != null)
        {
            root = root.right
        }
        return root
    }
}
```

Better approach

```
------------------
 if (is_bst(root, Integer.MIN_VALUE, Integer.MAX_VALUE)) {
            System.out.println("YES")
        } else {
            System.out.println("NO")
        }

boolean is_bst(node root, int min, int max) {
        if (root == null) return true
        if (root.data <= min || root.data >= max) return false
        return is_bst(root.left, min, root.data) && is_bst(root.right, root.data, max);
    }
```



S(1, min, 2)

S(3, 2, max)

1 <= min X

1 >= 2 X

3 <= 2 X

3 >= max X

S(5, 3, max)

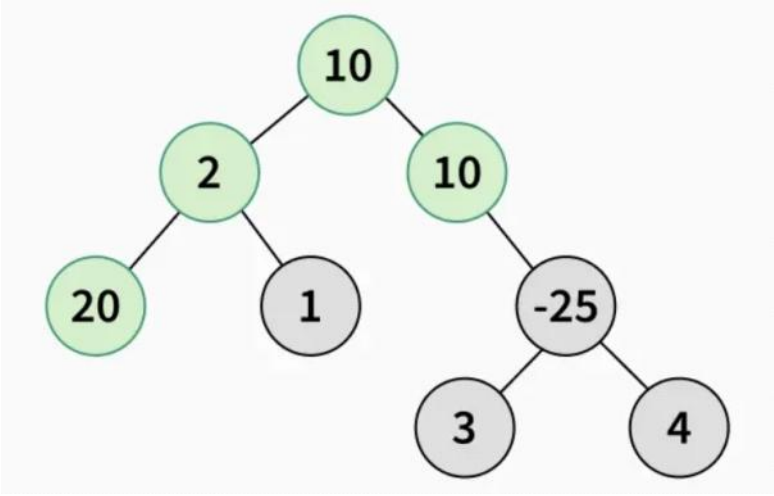
5 <= 3 X

5 >= max X

Maximum Path sum

03 June 2025 14:43

Input: root[] = [10, 2, 10, 20, 1, N, -25, N, N, N, N, 3, 4]
Output: 42
Explanation:



Max path sum is represented using green colour nodes in the above binary tree.

Given a node n, and:

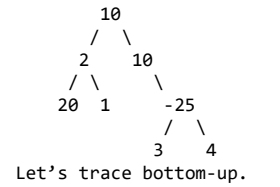
- left = max gain from left subtree
- right = max gain from right subtree

We compute:

- curr_sum = left + right + n.val → this is the **best sum at this node**
- The function **returns** n.val + max(left, right) → because only **one path can be extended** upward

We also maintain a **global max** to store the best answer across all nodes.

```
 static int max_path_sum(Node root) {
        if (root == null) return 0
        int left = Math.max(0, max_path_sum(root.left))
        int right = Math.max(0, max_path_sum(root.right))
        max_sum = Math.max(max_sum, root.val + left + right)
        return root.val + Math.max(left, right)
    }
```



Let’s trace bottom-up.

Node -25:
left = 3, right = 4

path sum through node = -25 + 3 + 4 = -18 → update max if better

return to parent = -25 + max(3, 4) = -21

Node 10 (right of root):
right = -21

path = 10 + -21 = -11

return = 10 + max(0, -21) = 10

Node 2:
left = 20, right = 1

local path = 2 + 20 + 1 = 23

return = 2 + max(20, 1) = 22

Root 10:
left = 22, right = 10

path = 10 + 22 + 10 = 42

return = 10 + max(22, 10) = 32

So:

Final max path = 42

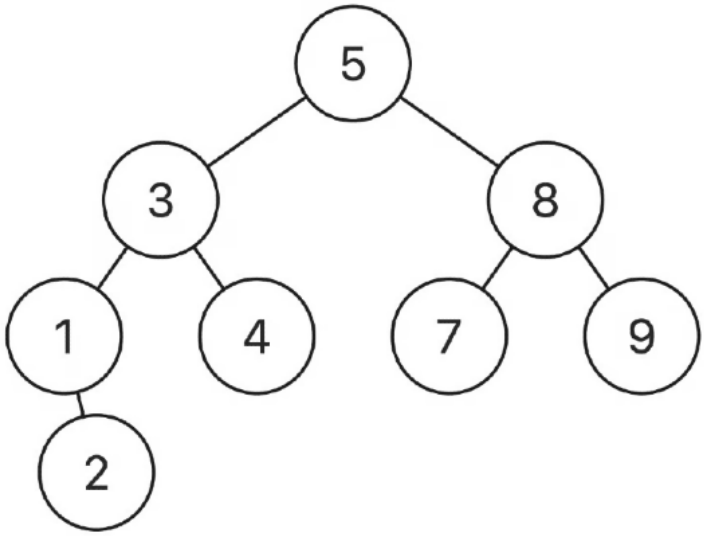
Lowest common ancestor of given two nodes BSTppa

04 June 2025 15:20

Given a binary search tree (BST) where all node values are *unique*, and two nodes from the tree p and q, return the lowest common ancestor (LCA) of the two nodes.

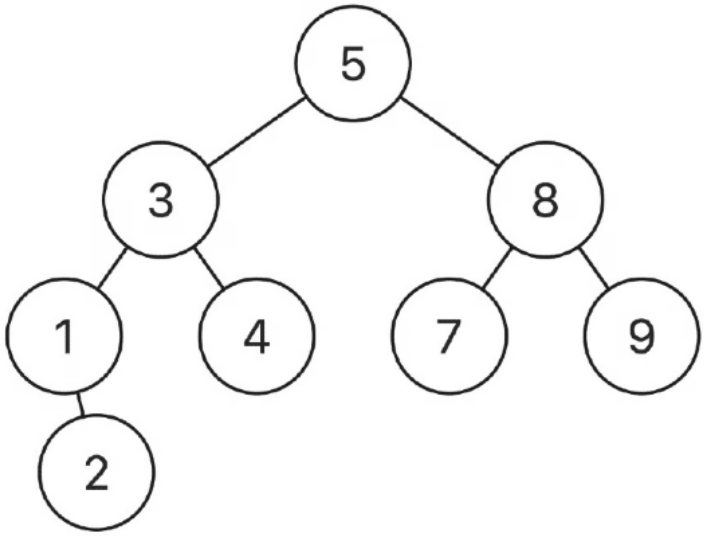
The lowest common ancestor between two nodes p and q is the lowest node in a tree T such that both p and q as descendants. The ancestor is allowed to be a descendant of itself.

Example 1:



Input: root = [5,3,8,1,4,7,9,null,2], p = 3, q = 8
Output: 5

Example 2:



Input: root = [5,3,8,1,4,7,9,null,2], p = 3, q = 4
Output: 3

```
        20
       /  \
     10    30
    / \     \
   5  15     35
```

LCA(5, 15) → 10

LCA(5, 35) → 20

LCA(30, 35) → 30

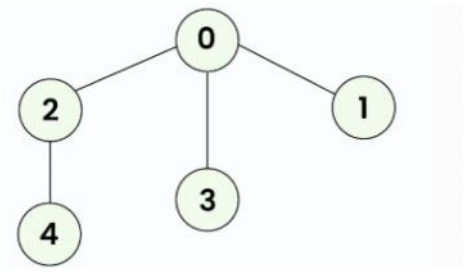
```
 Node lca(Node root, int n1, int n2) {
        while (root != null) {
            if (n1 < root.val && n2 < root.val)
                root = root.left;
            else if (n1 > root.val && n2 > root.val)
                root = root.right;
            else
                return root;
        }
        return null;
    }
```

BFS and DFS traversal

04 June 2025 15:34

Given a **connected undirected graph** containing **V** vertices, represented by a 2-d adjacency list **adj[][]**, where each adj[i] represents the list of vertices connected to vertex i. Perform a **Breadth First Search (BFS)** traversal starting from vertex 0, visiting vertices from left to right according to the given adjacency list, and return a list containing the BFS traversal of the graph.
Note: Do traverse in the **same order** as they are in the given **adjacency list**.

Input: adj[][] = [[2, 3, 1], [0], [0, 4], [0], [2]]



create adjacency list of this graph

0 → 1 2 3
1 → 0
2 → 0 4
3 → 0
4 → 2

Output: [0, 2, 3, 1, 4]
Explanation: Starting from 0, the BFS traversal will follow these steps:
Visit 0 → Output: 0
Visit 2 (first neighbor of 0) → Output: 0, 2
Visit 3 (next neighbor of 0) → Output: 0, 2, 3
Visit 1 (next neighbor of 0) → Output: 0, 2, 3,
Visit 4 (neighbor of 2) → Final Output: 0, 2, 3, 1, 4

```
public ArrayList<Integer> bfs(ArrayList<ArrayList<Integer>> adj) {
        int n=adj.size()
        int[] vis=new int[n+1]
        vis[0]=1
        Queue<Integer> q=new LinkedList<>()
        q.offer(0)
        ArrayList<Integer> bfs=new ArrayList<>()
        while(!q.isEmpty()){
```

e

```
            int node=q.poll()
            bfs.add(node)
            for(int it:adj.get(node)){
                if(vis[it]!=1){
                    vis[it]=1
                    q.offer(it)
                }
            }
        }
        return bfs

    }
}
```

DFS traversal

Output: [0, 2, 4, 3, 1]
Explanation: Starting from 0, the DFS traversal proceeds as follows:
Visit 0 → Output: 0
Visit 2 (the first neighbor of 0) → Output: 0, 2
Visit 4 (the first neighbor of 2) → Output: 0, 2, 4
Backtrack to 2, then backtrack to 0, and visit 3 → Output: 0, 2, 4, 3
Finally, backtrack to 0 and visit 1 → Final Output: 0, 2, 4, 3, 1

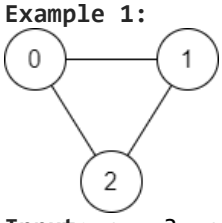
```
public ArrayList<Integer> dfs(ArrayList<ArrayList<Integer>> adj) {
        boolean vis[]=new boolean[adj.size()]
        vis[0]=true
        ArrayList<Integer> list=new ArrayList<>()
        helper(0,vis,adj,list)
        return list
    }

    private void helper(int node,boolean vis[],ArrayList<ArrayList<Integer>> adj,ArrayList<Integer> list){
        vis[node]=true
        list.add(node)

        for(int i : adj.get(node))
        {
            if(!vis[i])
                helper(i,vis,adj,list)
        }
    }
}
```

Check if there is a path from src to dest

04 June 2025 15:20

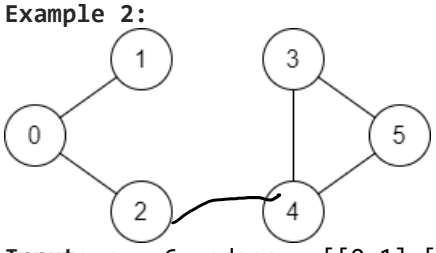
There is a **bi-directional** graph with n vertices, where each vertex is labeled from 0 to n - 1 (**inclusive**). The edges in the graph are represented as a 2D integer array edges, where each edges[i] = [u_i, v_i] denotes a bi-directional edge between vertex u_i and vertex v_i. Every vertex pair is connected by **at most one** edge, and no vertex has an edge to itself.
You want to determine if there is a **valid path** that exists from vertex source to vertex destination.
Given edges and the integers n, source, and destination, return true *if there is a* ***valid path*** *from* source *to* destination*, or* false *otherwise*.

Example 1:



Input: n = 3, edges = [[0,1],[1,2],[2,0]], source = 0, destination = 2
Output: true
Explanation: There are two paths from vertex 0 to vertex 2:
- 0 → 1 → 2
- 0 → 2

Example 2:



Input: n = 6, edges = [[0,1],[0,2],[3,5],[5,4],[4,3]], source = 0, destination = 5
Output: false
Explanation: There is no path from vertex 0 to vertex 5.

Approach 1 using DFS

```
 int source = sc.nextInt()
        int destination = sc.nextInt()

        boolean[] visited = new boolean[n]
        boolean result = dfs(source, destination, adj, visited)

        System.out.println(result)
    }

    public static boolean dfs(int curr, int dest, ArrayList<ArrayList<Integer>> adj, boolean[] visited) {
        if (curr == dest) return true

        visited[curr] = true

        for (int neighbor : adj.get(curr)) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, dest, adj, visited)) return true
            }
        }

        return false
```

```
    }
}
```

approach 2 using BFS

- Start from the source node
- Use a **queue** to explore neighbors level by level
- Mark nodes as **visited** to avoid cycles
- If you reach destination, return true
- If BFS completes and you never reach destination, return false

```
 boolean result = bfs(source, destination, adj, n);

        System.out.println(result)
    }

    public static boolean bfs(int source, int destination, ArrayList<ArrayList<Integer>> adj, int
n) {
        boolean[] visited = new boolean[n];
        Queue<Integer> q = new LinkedList<>();

        q.add(source);
        visited[source] = true;

        while (!q.isEmpty()) {
            int node = q.poll()

            if (node == destination) return true

            for (int neighbor : adj.get(node)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true
                    q.add(neighbor)
                }
            }
        }

        return false
    }
}
```

```
    public boolean validPath(int n, int[][] edges, int source, int destination) {
        ArrayList<ArrayList<Integer>> graph=new ArrayList<>()
        for(int i=0;i<n;i++){
            graph.add(new ArrayList<>());
        }
        for(int[] edge:edges){
            int u=edge[0],v=edge[1];
            graph.get(u).add(v);
            graph.get(v).add(u);
        }
        boolean[] vis=new boolean[n];
        return dfs(graph,vis,source,destination);
    }
    public boolean dfs(ArrayList<ArrayList<Integer>> graph,boolean[] vis,int src,int tar){
```

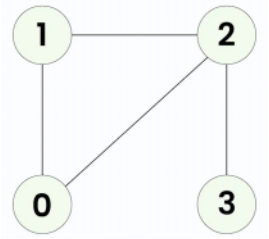
```
        if(src==tar)
        return true;
        vis[src]=true;
        for(int neigh:graph.get(src)){
            if(!vis[neigh]){
            boolean an= dfs(graph,vis,neigh,tar);
            if(an)
            return true;
            }
        }
        return false;
    }
}
```

Detect cycle in an undirected graph

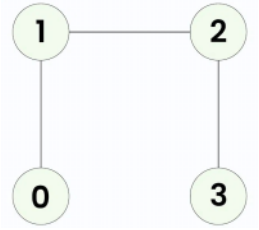
04 June 2025 15:56

Given an **undirected graph** with **V** vertices and **E** edges, represented as a 2D vector **edges[][]**, where each entry **edges[i] = [u, v]** denotes an edge between vertices **u** and **v**, determine whether the graph contains a **cycle** or not.

Examples:
Input: V = 4, E = 4, edges[][] = [[0, 1], [0, 2], [1, 2], [2, 3]]
Output: true
Explanation:



1 -> 2 -> 0 -> 1 is a cycle.
Input: V = 4, E = 3, edges[][] = [[0, 1], [1, 2], [2, 3]]
Output: false
Explanation:

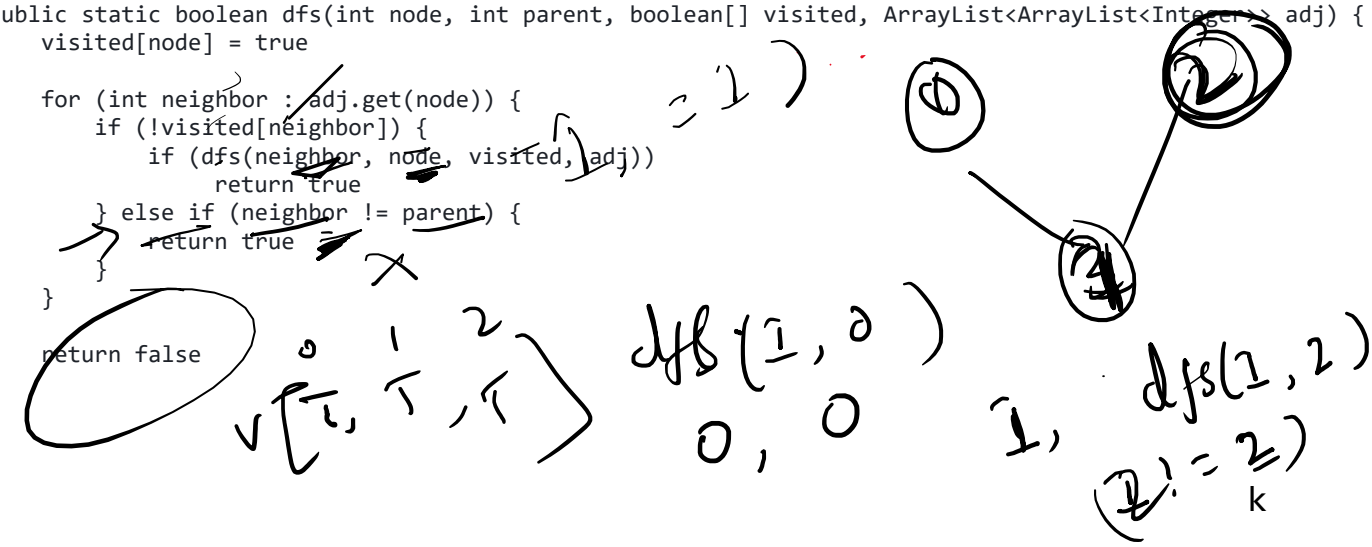


No cycle in the graph.

- Use a **visited array** to track visited nodes.
- For each unvisited node, do DFS.
- In DFS:
  - For every neighbor of the current node:
    - If it's **not visited**, recursively call DFS.
    - If it’s **visited and not the parent**, cycle exists.

```
        boolean[] visited = new boolean[v];
        boolean has_cycle = false;

        for (int i = 0; i < v; i++) {
            if (!visited[i]) {
                if (dfs(i, -1, visited, adj)) {
                    has_cycle = true;
                    break;
                }
            }
        }

        System.out.println(has_cycle);
    }

    public static boolean dfs(int node, int parent, boolean[] visited, ArrayList<ArrayList<Integer>> adj) {
        visited[node] = true
```

```
}

public static boolean dfs(int node, int parent, boolean[] visited, ArrayList<ArrayList<Integer>> adj) {
    visited[node] = true

    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) {
            if (dfs(neighbor, node, visited, adj))
                return true
        } else if (neighbor != parent) {
            return true
        }
    }

    return false
}
}
```



Dijkstras Algorithm

04 June 2025 15:55

https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/

number of provinces

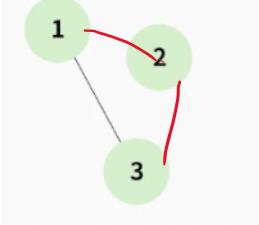
04 June 2025 15:34

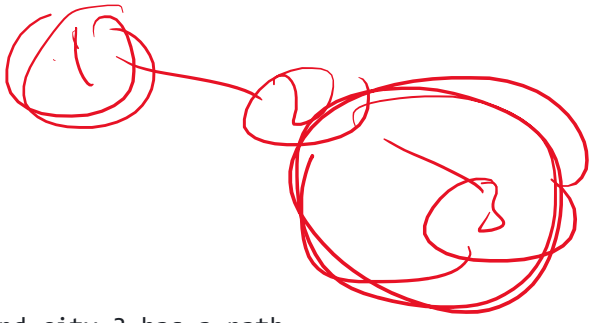
Given an **undirected** graph with **V** vertices. We say two vertices u and v belong to a single province if there is a path from u to v or v to u. Your task is to find the number of provinces.

Note: A province is a group of **directly** or **indirectly connected** cities and no other cities outside of the group.

Example 1:

Input:[[1, 0, 1],[0, 1, 0],[1, 0, 1]]





Output:

2

Explanation:

The graph clearly has 2 Provinces [1,3] and [2]. As city 1 and city 3 has a path between them they belong to a single province. City 2 has no path to city 1 or city 3 hence it belongs to another province.

```
boolean[] visited = new boolean[v];
        int count = 0;

        for (int i = 0; i < v; i++) {
            if (!visited[i]) {
                dfs(i, visited, adj, v);
                count++;
            }
        }

        System.out.println(count);
    }

    public static void dfs(int node, boolean[] visited, int[][] adj, int v) {
        visited[node] = true;
        for (int j = 0; j < v; j++) {
            if (adj[node][j] == 1 && !visited[j]) {
                dfs(j, visited, adj, v);
            }
        }
    }
}
```

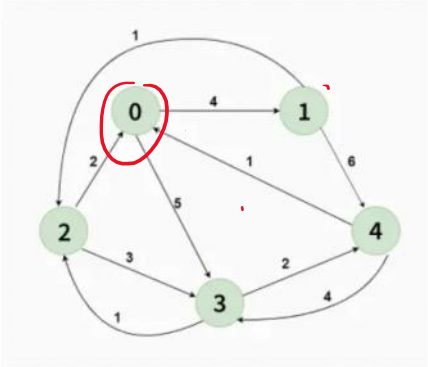
Floyd warshall algorithm

04 July 2025 16:45

```
for (int k = 0; k < v; k++) {
            for (int i = 0; i < v; i++) {
                for (int j = 0; j < v; j++) {

      if (dist[i][k] < inf && dist[k][j] < inf)
                        dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
```

dist[1][2] = min(dist[1][2], dist[1][0] + dist[0][2])
= min(∞, 4+2)
min(∞, 6)
= 6

Input: *dist[][] = [[0, 4, 10^8, 5, 10^8],*
[10^8, 0, 1, 10^8, 6],
[2, 10^8, 0, 3, 10^8],
[10^8, 10^8, 1, 0, 2],
[1, 10^8, 10^8, 4, 0]]



dist[4][1] = min(
dist[4][1], dist[4][0] + dist[0][1]
min(6, 1+4)
= 5